COMP34312: Mathematical Topics in Machine Learning

Last update: February 17, 2026

Gavin Brown *Gavin.Brown@manchester.ac.uk*
Anirbit Mukherjee *Anirbit.Mukherjee@manchester.ac.uk*

This course introduces the **theoretical foundations of machine learning**. We aim at students who want to move beyond just using models, *to understanding why they work*. This will be a rigorous mathematical and conceptual treatment that may require developing your knowledge of probability theory, linear/matrix algebra, and calculus. We will not teach this mathematics for its own sake, but rather to frame the understanding of Machine Learning as a field.

You'll learn how we formalize learning problems, interpret performance through the lens of probability theory, and reason about why some models generalize while others do not. We focus on foundational ideas—risk, generalization, capacity, convergence, and trade-offs—that underlie all learning systems, regardless of the algorithm used. By the end, you'll be equipped to think critically about model behavior, and understand what statistical learning theory can (and cannot) guarantee.

This could be seen as preparing you to engage with modern research literature, as an enabler toward a research career, either in industry or academia.

Lecture in weeks 1-6: Tuesday, 11am - 1pm. Simon Building, Theatre D.
Lecture in weeks 7-11: Same day, same time. Mansfield Cooper Building, G19.

In most (not all) weeks, the first hour will be quizzes/reviews, and the second will be new material. We will use time flexibly to give you breaks as needed.

Assessment: 80% closed-book exam, 20% coursework (see Canvas site for details).

This notes pack is intended to be self-contained, i.e. contains everything you need to know.

The session on...	is about...
Tuesday 3 February 2026	1. The Foundations of Supervised Learning
10 February	2. The Probabilistic View of Learning
17 February	3. The Problem with Learning a Model
24 February	4. Combining Multiple Models
3 March	5. Understanding the Sources of Risk
10 March	6. How to be Probably Approximately Correct
17 March	7. Some more mathematics ... that you need to know
24 March	8. A Formal Introduction to the Gradient Descent Algorithm
31 March	NO LECTURE – EASTER BREAK
7 April	NO LECTURE – EASTER BREAK
14 April	NO LECTURE – NON-TEACHING PERIOD
21 April	9. Gradient Descent in *Overparameterized* Linear Regression
28 April	10. A Measure of Capacity : Rademacher Complexity
5 May	11. Elementary Examples of Bounding Rademacher Complexity
Friday 8 May	**Coursework: Group Poster Presentation**
19 May	NO LECTURE – REVISION WEEK
26 May	NO LECTURE – EXAMS BEGIN

Contents

1 The Foundations of Supervised Learning **5**

1.1 What is supervised learning? . . . 5

1.2 A reminder: Overfitting and Underfitting . . . 10

1.3 Population Risk and Empirical Risk . . . 11

1.4 The "Bayes model" . . . 12

1.5 Summary . . . 12

2 The Probabilistic View of Learning **16**

2.1 The Principle of Maximum Likelihood . . . 16

2.2 How does all this probability theory relate to good old machine learning?! . . . 18

2.3 Deriving an asymmetric loss for predicting house prices . . . 20

2.4 Relating all this back to the concept of 'Risk' . . . 21

2.5 Summary . . . 22

3 The Problem with Learning a Model **25**

3.1 Our training set is finite, and has hidden sources of randomness . . . 25

3.2 Bias and Variance: A Metaphor . . . 26

3.3 From Metaphors to Mathematics: A formal definition of bias & variance . . . 27

3.4 The bias/variance trade-off can help understand overfitting . . . 28

3.5 The decomposition holds for many losses, but not all losses . . . 29

3.6 Summary . . . 29

4 Combining Multiple Models **34**

4.1 A simple ensemble learning algorithm . . . 34

4.2 Ensembles can guarantee a reduction in loss . . . 36

4.3 Revisiting bias/variance in the ensemble setting . . . 37

4.4 Theory for Majority Voting ensembles . . . 40

4.5 Summary . . . 41

5 Understanding the Sources of Risk **46**

5.1 What causes risk? . . . 46

5.2 Let's expand our function class . . . 49

5.3 The bias/variance trade-off cannot explain ALL overfitting: The Double-Descent Phenomenon . . . 50

5.4 Summary . . . 51

6 How to be Probably Approximately Correct **54**

6.1 54

6.2 54

6.3 54

7 Some more mathematics ... that you need to know **55**

7.1 Introduction to Lipschitzness, Convexity and Smoothness . . . 56

7.2 Introduction to Convergence . . . 62

7.3 Introduction to Supremums and Infimums . . . 64

7.4 Introduction to Spectral Norms of Matrices . . . 65

7.5 Introduction to Rank of Matrices . . . 66

8 A Formal Introduction to the Gradient Descent Algorithm **71**

8.1 Introduction to Proof of Convergence of G.D. & More Demonstrations . . . 71

9 Gradient Descent in *Overparameterized* Linear Regression **76**

10 A Measure of Capacity : Rademacher Complexity **82**

10.1 Expectation Over Data of the Worst Generalization Gap & Rademacher Complexity . . . 84

11 Elementary Examples of Bounding Rademacher Complexity **90**

11.1 Rademacher Complexity for Linear Predictors . . . 91

11.2 Proof of Lemma 11.2 . . . 94

11.3 Looking Ahead With Neural Nets . . . 95

A Supervised Learning Basics **96**

A.1 Supervised Learning basics: over-fitting and under-fitting . . . 96

A.2 Loss functions: squared and cross-entropy . . . 98

A.3 Gradient Descent as a way of minimizing loss . . . 100

A.4 Decision trees . . . 102

A.5 Linear Regression . . . 103

B Probability Theory Basics **109**

B.1 Random Variables . . . 109

B.2 Joint and Conditional Random Variables . . . 111

B.3 Expectation and Variance . . . 112

B.4 The Law of Large Numbers and the Central Limit Theorem . . . 114

Notation used in this module

There will be a *lot* of mathematical notation over the next 11 weeks. We have tried to collect the majority of this notation here, for quick reference. At the beginning of the module, you won't know the meaning of all the terms, but they will get slowly used over the weeks, so have patience, and don't panic.

$\boldsymbol{x}$	Input vector
$\boldsymbol{y}$	Output/target vector
d	Dimension of input $\boldsymbol{x}$ (i.e. number of features)
k	Dimension of labels $\boldsymbol{y}$ (e.g. number of classes)
$p(\boldsymbol{x}, \boldsymbol{y})$	A probability distribution over example pairs $(\boldsymbol{x}, \boldsymbol{y})$
$\mathbb{E}_{\boldsymbol{xy}}[\cdots]$	Expectation (i.e. average) over possible examples drawn from $p(\boldsymbol{x}, \boldsymbol{y})$
$\mathcal{S}$	A data set $\mathcal{S} \coloneqq \{(\boldsymbol{x}_i, \boldsymbol{y}_i) \mid i = 1, \ldots, n\}$, where each $(\boldsymbol{x}, \boldsymbol{y}) \sim p(\boldsymbol{x}, \boldsymbol{y})$
$\mathbb{E}_{\mathcal{S}}[\cdots]$	Expectation (i.e. average) over possible datasets $\mathcal{S}$
$f(\boldsymbol{x})$	A model predicting $\boldsymbol{y}$, evaluated at a point $\boldsymbol{x}$.
$\ell(\boldsymbol{y}, f(\boldsymbol{x}))$	A (non-negative) loss function, evaluated at a point $\boldsymbol{x}, \boldsymbol{y}$.
$R(f) \coloneqq \mathbb{E}_{\boldsymbol{xy}}\left[\ell(\boldsymbol{y}, f(\boldsymbol{x}))\right]$	Population risk of a model
$\hat{R}(f, \mathcal{S}) \coloneqq \frac{1}{n}\sum_{i=1}^{n}\left[\ell(\boldsymbol{y}_i, f(\boldsymbol{x}_i))\right]$	Empirical risk of a model, evaluated on a set $\mathcal{S}$
Ω	The space of all possible (measurable) functions.
$\mathcal{F} \subset \Omega$	A function class, as a subset of all possible functions.
$f_{erm} \coloneqq \operatorname*{arginf}_{f \in \mathcal{F}}\left[\hat{R}(f, \mathcal{S})\right]$	Empirical risk minimizer in $\mathcal{F}$
$f^* \coloneqq \operatorname*{arginf}_{f \in \mathcal{F}}\left[R(f)\right]$	Population risk minimizer in $\mathcal{F}$.
$y^* \coloneqq \operatorname*{arginf}_{f \in \Omega}\left[R(f)\right]$	Population risk minimizer in Ω, known as the Bayes-optimal model.
$\hat{\mathcal{R}}_n(\mathcal{H})$	Empirical Rademacher complexity of a function class $\mathcal{H}$
$\mathcal{R}_n(\mathcal{H})$	Averaged Rademacher complexity of a function class $\mathcal{H}$
	Given a $\mathcal{F}$ and ℓ as above our typical instance of $\mathcal{H}$ will be the corresponding "loss class", $\mathcal{H} = \{(\boldsymbol{x}, y) \mapsto \ell(y, f(\boldsymbol{x})) \in [0, \infty) \mid f \in \mathcal{F}\}$. Though we would also be interested in the Rademacher complexity of $\mathcal{F}$ and accordingly $\hat{\mathcal{R}}_n(\mathcal{F})$ and $\mathcal{R}_n(\mathcal{F})$ will be defined.
$F : \mathbb{R}^p \to \mathbb{R}^q$	An arbitrary function which we may use with gradient descent.

1 The Foundations of Supervised Learning

Outline: In this first week we review the essential elements of supervised learning. Our goal is to formalise what it really means to "learn" from data.

We will mostly use concepts you are familiar with from previous courses, but we'll provide a more formal mathematical notation. Toward the end we'll introduce a couple of new ideas.

This week is mostly revision on ideas you know already. For a deeper treatment, see Appendix A.

1.1 What is supervised learning?

Supervised learning provides the formal setting in which we study the behaviour of predictive models. There are **four essential elements** that define any supervised Machine Learning scenario.

The 4 "essential elements" of ML: Data. Model. Loss. Algorithm.

The **data** is a set of examples specifying how we'd like our final system to behave, e.g. if we want a system that can recognize images of cats, we provide images of cats, and also images of things that are not cats, along with labels attached to each image saying what it is.

The **model** is what makes predictions for us. You will have heard of 'neural networks'—this is just one of many many possible models. Other important ones are decision trees, logistic regression, etc. We will treat them in *abstract*, they are all just mathematical functions with parameters.

The **loss** is a mathematical function used to assess the performance of our model. It takes predictions from the model, and scores how well the model makes predictions on the data.

The **algorithm** is the procedure that we use to optimise the parameters of the model, based on what the loss says. Most algorithms are *iterative*, in that they update parameters by a small amount, then re-evaluate the loss, and repeat. A family of such algorithms on which we will focus are called 'gradient descent' methods, but there are many many variations, and many learning algorithms that do not use gradient descent at all.

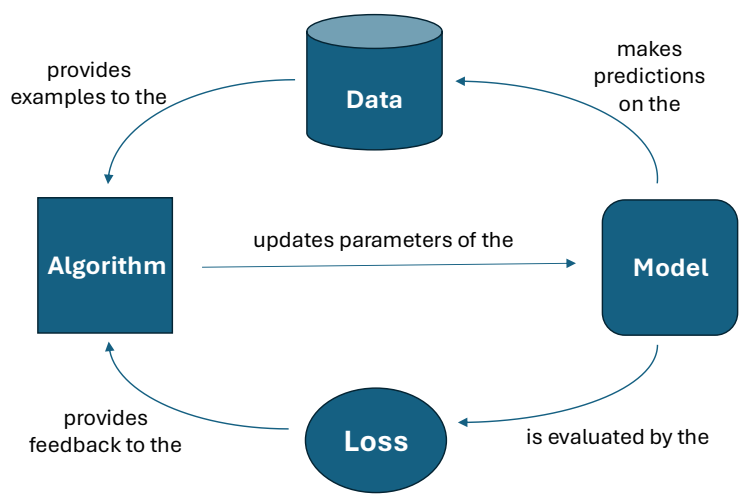


Figure 1: Relationships between the 4 essential elements of a supervised learning system.

Throughout the course, these four ideas will be central to our discussions. They are best understood using the language of probability theory, calculus, and linear algebra. By defining these at the outset, we prepare ourselves more challenging (and interesting) ideas, like generalization, risk, and capacity. This week establishes the formal vocabulary for the remainder of the course. Let's go....

Data

You were probably taught that an ML dataset comes in a 'spreadsheet' structure, with rows/columns. This is not the only way data comes, but it's convenient to start our journey. Each row is called an *example*. For each, there are *feature* columns that describe it, plus a final one: the *label* of the example.

We now define a notation that will be used through the course. Each example (sometimes called a 'data-point') consists of a vector $\boldsymbol{x} \in \mathbb{R}^d$, i.e. a d-dimensional vector of real-valued numbers. The other part of an example is the label. The label is $y \in \{1, 2, 3, ..., k\}$, assuming there are k possible labels. However we might have a *real-valued* label (e.g. height of a person), which would be written $y \in \mathbb{R}$. We could also have a vector of outputs, e.g. predicting label *probabilities*, for which we will clarify notation as we go on. In general we will specify some *input space* $\mathcal{X}$, and some *label space* $\mathcal{Y}$, where each of our examples is $(\boldsymbol{x}, \boldsymbol{y}) \in \mathcal{X} \times \mathcal{Y}$. The supervised learning task is to take a new $\boldsymbol{x}$ and predict what the correct label y should be.

A supervised dataset with n examples and d features is written in matrix form as

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,d} \\ x_{2,1} & x_{2,2} & \dots & x_{2,d} \\ \vdots & \vdots & & \vdots \\ x_{n,1} & x_{n,2} & \dots & x_{n,d} \end{bmatrix} \in \mathbb{R}^{n \times d}, \qquad \boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n.$$

Each row $(\boldsymbol{x}_{i,1}, \dots, \boldsymbol{x}_{i,d})$ is a feature vector $\boldsymbol{x}_i \in \mathcal{X}$, and $y_i \in \mathcal{Y}$ is its corresponding label. Notice how the notation allows us to refer to the full set of labels as a *column* vector $\boldsymbol{y}$, and the inputs as a matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$.

Probability theory is fundamental to Machine Learning. We will use this as our 'language' to describe the data. We assume each example / data-point is an independent *observation* from an unknown probability distribution $p(\boldsymbol{x}, y)$. Our data-set, denoted $\mathcal{S}$, is then a *sample* (i.e. multiple observations) drawn from $p(\boldsymbol{x}, y)^n$. Notice the notation for 'to the power n' on the distribution—this means we draw n data-points independently from the same distribution. Formally, this is known as the *independent and identically distributed* (i.i.d.) assumption, and it underpins most (if not all) algorithms you will have heard about.

> The i.i.d. (independent and identically distributed) assumption means we assume our dataset consists of n examples/data-points each sampled independently from the same unknown distribution.

Our data-set (sometimes called a 'sample') can now described as $\mathcal{S} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n \sim p(\boldsymbol{x}, y)^n$. You've been taught the concept of a *training* set, and a *testing* set. These are then simply $\mathcal{S}^{train}$ and $\mathcal{S}^{test}$. We may have these provided, or perhaps we split some larger dataset into two parts, i.e. $\mathcal{S} = \{\mathcal{S}^{train} \cup \mathcal{S}^{test}\}$. We use these in what's commonly called the supervised learning 'pipeline', illustrated below.

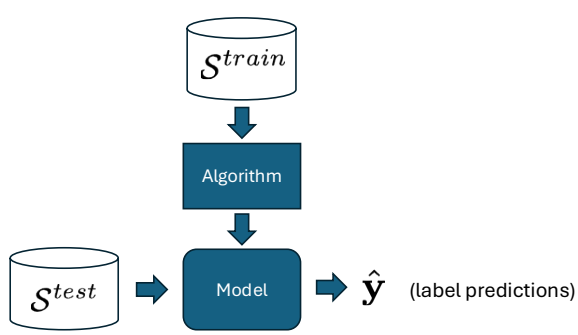


Figure 2: Flow of data in the standard supervised learning pipeline.

Our overall objective in supervised learning is to use $\mathcal{S}^{train}$ to learn a **model** that can predict well on *any future dataset* that might be drawn from the unknown distribution $p(\boldsymbol{x}, y)$. We use the testing set $\mathcal{S}_m^{test}$ as a proxy for this, but have to accept that it (and the training set itself) may not be perfectly representative of the underlying distribution. **This brings us to the idea of a "model".**

Models

You will have heard about different models already, like logistic regression, neural networks, naive Bayes, and more. We will treat all of these as abstract mathematical concepts, i.e. they are just 'some function' that maps from inputs $\boldsymbol{x}$ to predicted labels y. Formally, we denote this as a function $f(\boldsymbol{x};\boldsymbol{w}) : \mathcal{X} \to \mathcal{Y}$, where $\boldsymbol{w}$ are the *parameters* of the function, defining how the mapping works.

To establish notation, we will review a few details. Assume we have d real-valued features, and a task where we have to predict a real-valued quantity, like someone's blood pressure or height, we can just specify a single *real-valued output*, i.e. $f(\boldsymbol{x};\boldsymbol{w}) : \mathbb{R}^d \to \mathbb{R}$. One simple model for this is a *linear regression*, specified as,

$$f(\boldsymbol{x};\boldsymbol{w}) \;=\; \boldsymbol{w}^T\tilde{\boldsymbol{x}}. \tag{1}$$

where $\tilde{\boldsymbol{x}} = [1\ \boldsymbol{x}]^T$. This 'padding' with a constant 1 enables what is called a 'bias' term.

If this is not familiar, you should go back to the course pre-requisites e.g. COMP24112 notes.

The output of a linear regression ranges from $-\infty$ to ∞, and in fact we have to be careful with our algorithms, to ensure we don't get those infinities, i.e. numerical under/overflow. However, lots of tasks don't need that range, and might require for example just predicting a binary label, e.g. 'cat' vs 'not-a-cat'. In this case, we may just want a predicted *probability*.

One model that can do this is *logistic regression*. This applies a *logistic function*, $\sigma(z) := 1/(1+\exp(-z))$, over the top of the linear regression,

$$f(\boldsymbol{x};\boldsymbol{w}) \;=\; \frac{1}{1+\exp\Big(-\boldsymbol{w}^T\tilde{\boldsymbol{x}}\Big)}. \tag{2}$$

This mathematical form is also called the *sigmoid*, and you'll see it again later in the course. A logistic regression produces an output in $(0,1)$, i.e. varying *between* 0 and 1, but not[1] hitting the extreme 0/1. This is also sometimes known as a single 'neuron', since the $\boldsymbol{x}_j$ are like input 'signals' from other neurons, the $\boldsymbol{w}_j$ are weightings, and the neuron 'fires' an output $f(\boldsymbol{x};\boldsymbol{w}) \in (0,1)$. Another type of model, which you may know as a 'neural network', is just a directed graph made up of lots of these things.

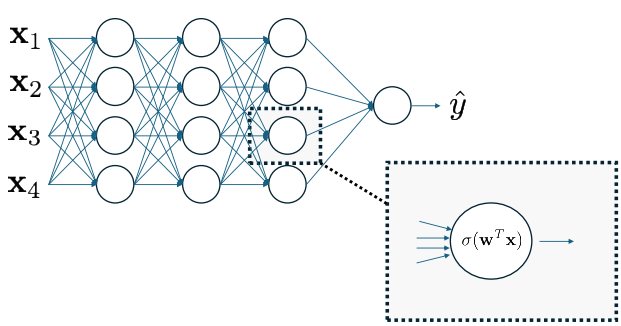


Figure 3: A "multi-layer perceptron" (one type of neural network) is just a directed graph of logistic regressions. There are many types of neural networks, that use functions other than the logistic.

> There are many types of models beyond neural networks, which are the current fashionable trend. Decision trees, probabilistic models, support vector machines, nearest-neighbour models, and many more. They all have strengths and weaknesses. For this course we treat them all abstractly as functions $f(\boldsymbol{x};\boldsymbol{w})$. The theoretical frameworks that we will discuss apply to **all** of them.

In *all* of these models, the behavior is *entirely determined* by the values of the parameters $\boldsymbol{w}$. If we can find 'good' parameters, then our model will perform well, being able to predict well on new examples $(\boldsymbol{x}, y)$. If we have 'bad' parameters, our model will fail, and predict badly. The challenge is now, how to *measure* how good/bad are a given set of parameters. **This brings us to the idea of a loss function.**

[1] If we wanted to specify that it can hit the extremes, we'd use square brackets, $[0,1]$.

Loss functions

A loss function quantifies the 'cost' we pay for a particular prediction, given the target. Imagine we have a new testing example $(\boldsymbol{x}, y)$, consisting of an input $\boldsymbol{x}$, and a true label y. The label is a real-valued quantity, and for this example we will assume $y = 1$. A natural choice here is the *squared loss.*

> Note: in the following, for a compact notation, we will write $f(\boldsymbol{x}) = f(\boldsymbol{x}; \boldsymbol{w})$, i.e. we drop the explicit dependence on the parameters. We just assume a function with some parameters hidden inside it.

Definition 1 (Squared loss)**.** *For a target $y \in \mathbb{R}$, and prediction $f(\boldsymbol{x}) \in \mathbb{R}$, the squared loss is,*

$$\ell(y, f(\boldsymbol{x})) \;=\; (y - f(\boldsymbol{x}))^2 \tag{3}$$

We can visualise this:

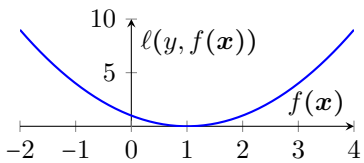


Figure 4: Squared loss $\ell(y, f(\boldsymbol{x})) = (y - f(\boldsymbol{x}))^2$ for target $y = 1$.

So, if we had set our parameters in such a way that $f(\boldsymbol{x}) = 1$, then our 'loss' on this example is zero. As the prediction gets worse, the loss[2] increases. The squared loss is suitable when we're predicting $y \in \mathbb{R}$. Let's now imagine we want to recognise images of cats. In our dataset, each image has $y = 1$ if it's a cat, otherwise $y = 0$. Our model produces a probability in $(0, 1)$, an estimate of $p(y = 1 \mid \boldsymbol{x})$, i.e. the *conditional probability* of it being a cat, given image $\boldsymbol{x}$. In this case a sensible loss is the *binary cross-entropy:*

Definition 2 (Cross-entropy loss)**.** *For a target $y \in \{0, 1\}$, and prediction $f(\boldsymbol{x}) \in (0, 1)$, the cross-entropy is,*

$$\ell(y, f(\boldsymbol{x})) \;=\; -\Big(y \ln f(\boldsymbol{x}) + (1 - y) \ln(1 - f(\boldsymbol{x}))\Big) \tag{4}$$

And, visualised...

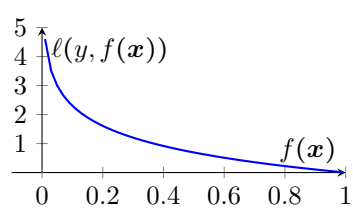


Figure 5: Cross-entropy for $y = 1$, i.e. $\ell(1, f(\boldsymbol{x})) = -\ln f(\boldsymbol{x})$. If the label was $y = 0$, this would become $\ell(0, f(\boldsymbol{x})) = -\ln(1 - f(\boldsymbol{x}))$, i.e. flipping horizontally, the same shape but going "uphill" from left to right.

There are **many many many** loss functions, that have pros and cons for different scenarios. We'll meet some more next week. The thing they have in common is: they **quantify** how bad our model is. And for any future examples, we want to minimise loss. To do that, we need one final element—an **algorithm** that can optimise our parameters, so we get the smallest possible loss.

[2] You may have met squared loss before with a constant $\frac{1}{2}$ in front, i.e. $\frac{1}{2}(y - f(\boldsymbol{x}))^2$. What matters is where the loss reaches it's minimum value. Multiplying by a constant does not move the minima, just scaling everything up/down by that constant.

Algorithms

The final element in a supervised learning system is the *learning algorithm*. The algorithm is the procedure that adjusts the parameters of a model, using the feedback provided by the loss. Its role is to *search* through the space of possible functions until it finds one that works well on the training data.

A model specifies a family of possible predictors, $f(\boldsymbol{x}; \boldsymbol{w})$, where $\boldsymbol{w} \in \mathbb{R}^p$ is the parameter vector. Each choice of $\boldsymbol{w}$ gives a different mapping from $\boldsymbol{x}$ to predictions of y. The algorithm takes the training set $\mathcal{S}^{train} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ and uses the loss $\ell(y, f(\boldsymbol{x}; \boldsymbol{w}))$ to decide how to modify $\boldsymbol{w}$ so that the model's predictions improve. After running the algorithm, we obtain a *trained function* $f(\boldsymbol{x}; \hat{\boldsymbol{w}})$ where $\hat{\boldsymbol{w}}$ is the parameter vector chosen by the algorithm based on the data.

Most modern learning algorithms are *iterative*. The basic idea is that they:

1. evaluate how well the model performs on data, with current parameters $\boldsymbol{w}$
2. compute some form of feedback using the loss function,
3. update the parameters in a direction that reduces the loss.

This process is repeated until the loss is deemed low enough for the problem at hand. After this, the trained model is returned and can be evaluated on the test set. The most widely used family of such methods is called *gradient descent*—this will be a focal point from week 8 onwards.

The learning algorithm uses the training set and the loss function to guide a search through a space of possible predictors, ultimately selecting a single predictor that we will use in deployment.

You can think about moving through the 'space' of possible predictors as below.

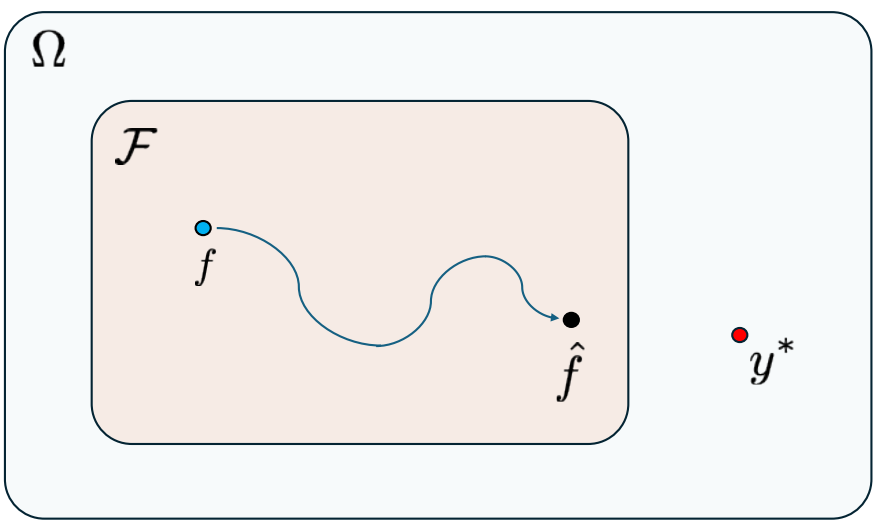


Figure 6: The blue box, marked top-left with Ω ("omega"), is the overall space of all possible predictors. The inner pink box $\mathcal{F}$ is the class of functions that can be represented by the particular model family we've committed to, for example neural networks of a particular fixed size/architecture. Before we learn a good set of parameters, we start at some (not so good) model f. Then, the process of learning can be thought of as iteratively updating the model parameters—moving through this space, to find a better (trained) model $\hat{f}$. We can also consider the hypothetical **best possible predictor**, marked y^*. This is the theoretical gold standard, and has a special name, the "Bayes model". We will formally define this later.

1.2 A reminder: Overfitting and Underfitting

Now we've got our 4 essential elements, we can review an important issue, that I hope you remember.

Overfitting and **underfitting** are two ways that a model can *fail* to capture the true structure of the data.

Underfitting is when the model is too simple to solve the task at hand. Even with plenty of training data, it cannot achieve a small loss because its function space is not big/expressive enough. In practice, this shows up as high training error and high test error: the model simply cannot fit what is actually there.

Overfitting is the opposite problem. Here the model is expressive enough to fit the training data extremely well, but it is so good that it captures idiosyncrasies and rare coincidences in the limited training data, that do not generalise to future observations. Training error becomes misleadingly small, and test error increases.

A **well-fitted** model sits between these extremes: expressive enough to capture the true signal, but constrained enough to avoid chasing noise. Understanding this balance is central to learning theory, because it reveals how model capacity, data size, and optimisation interact to determine generalisation performance.

Figure 7: If we choose a very simple model (left) it will likely not capture the underlying function, resulting in an **under-fitted** model. If we choose a very complex model (right) it will likely capture noise in the data, that we really want it to ignore, resulting in an **over-fitted** model.

As mentioned, the way we diagnose over/underfitting is by looking at performance on data as we increase the complexity of the model. Below is a plot as we train successively more complex *decision trees* on a problem. The faint red lines are repeated runs of the algorithm, and bold lines show the mean. The testing error first goes down, and then slowly drifts up, whilst training error continues downward. As can be seen from the plot, for this problem we'd ideally stop at depth 5, and deploy that model—it doesn't get the lowest training error, but does perform the best on the testing data.

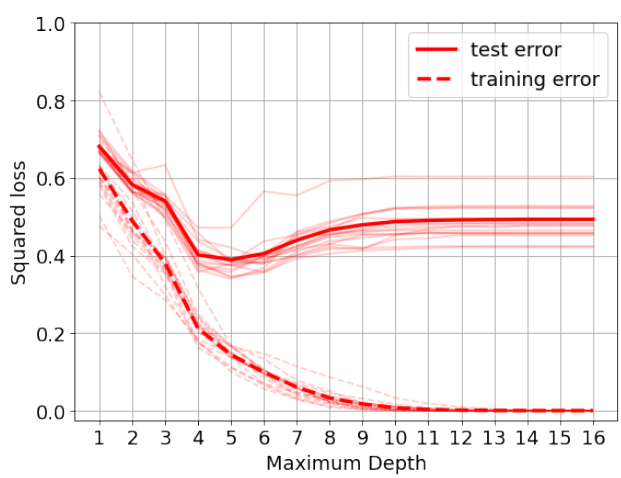


Figure 8: Increasing the depth of a decision tree to observe overfitting.

1.3 Population Risk and Empirical Risk

Thus far, we've reviewed things you (should) know already. Now, we'll introduce a couple of new concepts.

We know that we don't just want to minimise loss on one example—we want the loss to be minimal for *all future examples*. How do we even begin describing "all future examples"? Now, you will see the importance of our assumption that the data is sampled i.i.d. from $p(\boldsymbol{x}, y)$. We can use probabilistic notions to describe the *average* loss over possible observations drawn from the true data-generating distribution. We use specific terminology, borrowed from statistics, the *"population risk"*.

Definition 3 (Population risk)**.** *Given a distribution $p(\boldsymbol{x}, y)$, a model that provides predictions as $y \leftarrow f(\boldsymbol{x})$, and a loss $\ell(y, f(\boldsymbol{x}))$ evaluating the quality of a prediction, the population risk is*

$$R(f) \coloneqq \mathbb{E}_{\boldsymbol{xy}}\Big[\ell(y, f(\boldsymbol{x}))\Big] = \int \int \ell(y, f(\boldsymbol{x}))\ p(\boldsymbol{x}, y) d\boldsymbol{x} dy \tag{5}$$

Notice here that we are using the concept of *expectation*, denoted $\mathbb{E}$. Formally, the notation $\mathbb{E}_{\boldsymbol{xy}}$ should be read as the 'expectation with respect to the true data distribution'. Informally, you can think of this as the loss on average across 'all possible' datapoints.

The **population risk** is the expected loss, with respect to the true (unknown) distribution $p(\boldsymbol{x}, y)$.

Now, that's somewhat abstract. We can't write an algorithm to solve the learning problem if it involves an unknown distribution $p(\boldsymbol{x}, y)$. Instead we will *approximate* this with the average loss over a dataset $\mathcal{S}$.

Definition 4 (Empirical risk)**.** *We assume a dataset $\mathcal{S} = \{(\boldsymbol{x}, y_i)\}_{i=1}^n$, each drawn i.i.d. from $p(\boldsymbol{x}, y)$. We have a model that provides predictions as $y \leftarrow f(\boldsymbol{x})$, and a loss $\ell(y, f(\boldsymbol{x}))$ evaluating the quality of a prediction. Then the* empirical risk *is defined:*

$$\hat{R}(f, \mathcal{S}) \coloneqq \frac{1}{n} \sum_{i=1}^{n} \ell\Big(y_i, f(\boldsymbol{x}_i)\Big). \tag{6}$$

The **empirical risk** is the arithmetic mean of the loss, over a set of examples $\{(\boldsymbol{x}, y_i)\}_{i=1}^n$.

This is an approximation to the population risk: if we increase the value n, i.e. make our dataset larger, then the numerical value $\hat{R}$ should converge to the true R. To believe this, that the **empirical risk** can be a reasonably good approximation of the **population risk**, we rely on the **law of large numbers**. You don't need to know the details of this. But, if you want to learn more, it is reviewed in appendix B.4.

The difference between these two is clearly important. So important, that it has a special name.

Definition 5 (Generalisation gap)**.** *The* "generalisation gap" *is defined as the empirical risk minus the population risk.*

$$\Delta_f \ = \ \hat{R}(f, \mathcal{S}) - R(f) \tag{7}$$

The **generalisation gap** is the difference $\hat{R} - R$, i.e. the empirical risk minus the population risk.

We might imagine that our set $\mathcal{S}$ is a training set. If we use a very complex model, we may be able to get the empirical (training) risk small. However, this may result in overfitting—indicated by a *negative* gap.

We now see that learning is not simply about making the empirical risk small; it is about ensuring that the empirical risk is a good proxy for the population risk. The difference between these two quantities is the generalisation gap, and everything that follows in this course is ultimately about understanding when this gap is small, when it might be large, and how it depends on the model we choose.

1.4 The "Bayes model"

We've just defined a very important expression, the population risk, in Definition 3. We know that this is the ultimate expression we'd like to get as small as possible. The hypothetical predictor that minimizes this is called the **Bayes model**. You will remember that you have actually seen this earlier today, illustrated in Figure 6. Now, we have the mathematics do formally define it.

Definition 6 (Bayes model)**.** *The Bayes model is the hypothetical model that minimizes population risk.*

$$y^* := \arg\min_{f \in \Omega} R(f) \tag{8}$$

where Ω is the set of all measurable functions $\mathcal{X} \to \mathcal{Y}$.

The **Bayes Model** is the hypothetical predictor that minimizes the population risk. It is also sometimes called the *Bayes-optimal predictor*, or just the *Bayes predictor*.

The is the theoretical gold standard: the model that makes predictions with the lowest possible risk. While unattainable in practice, it serves as a useful concept when discussing the performance of learned models.

Now, what is really interesting is... in certain cases, we can write down a mathematical expression for the Bayes model, even though we can't actually compute it with a learning algorithm. For example, with squared loss, the Bayes model is the *expected value of the label* at the point $\boldsymbol{x}$.

$$y^*(\boldsymbol{x}) := \mathbb{E}_{y|\boldsymbol{x}}[y]. \tag{9}$$

What the expression means in practice is: we take the input vector $\boldsymbol{x}$, and we *average* the possible labels y, weighted by their probability of occurring under the true (unknown) distribution. As such, the expression $\mathbb{E}_{y|\boldsymbol{x}}[y]$ is also sometimes called the **expected label**.

The Bayes model under squared loss is the **expected label**, $\mathbb{E}_{y|\boldsymbol{x}}[y]$.

The **proof of this** is not mandatory to learn/understand, but you may find it interesting... it's at the end of this chapter, if you want to take a look. Since we have a definition of this hypothetical model in Equation 9, we can discuss the population risk of this hypothetical model.

Definition 7 (Bayes risk, squared loss)**.** *The Bayes risk, given $\ell(y, f) = (y - f)^2$, and an unknown $p(\boldsymbol{x}, y)$ is,*

$$R(y^*) := \mathbb{E}_{\boldsymbol{xy}}\Big[(y - \mathbb{E}_{y|\boldsymbol{x}}[y])^2\Big] \tag{10}$$

Notice this is just the Bayes model $\mathbb{E}_{y|\boldsymbol{x}}[y]$ plugged into the squared loss, with an expectation over $p(\boldsymbol{x}, y)$.

The **Bayes risk** is the risk of the Bayes-optimal model.

1.5 Summary

To sum up, supervised learning is about choosing a model that not only fits the examples we are given, but also makes good predictions on new data drawn from the same unknown source. We some new ideas introduced this week: empirical risk, population risk, the generalization gap, and the theoretical ideal, the Bayes Model. These give us a way to talk about learning in a formal manner. They also highlight a key challenge: we only ever see a finite sample, yet to avoid overfitting, we must think about performance on the whole probability distribution of possible observations.

In the next section, we take a closer look at this probabilistic view. We will see how loss functions are not arbitrary choices—they actually arise from simple probabilistic assumptions built into the problem itself—and how thinking in terms of probability helps us understand what it means to "learn" from limited data.

Foundations of Supervised Learning — Practice Exam Questions

Each week we provide you a few practice questions, representative of what may be in the final written exam. The exam will be in Canvas, so all questions match this style. Answers provided separately.

If you can answer all of **Level 1**... you're on track, but doing the bare minimum to pass.
If you can answer all of **Level 2**... you're doing well, more than enough to pass.
If you can answer all of **Level 3**... well ahead, into first class territory. I expect 20% of the class here.

Level 1

1. In supervised learning, the four essential elements are *data*, *model*, *loss*, and _____.

2. **True or False.** Underfitting occurs when a model is too simple to achieve low training error.

3. Which component of a learning system assigns a numerical score to a prediction?

   (a) Data
   (b) Model
   (c) Loss
   (d) Algorithm

4. Match each term to its description.

Term	Description
Data	Updates model parameters
Model	Produces predictions
Loss	Evaluates prediction quality
Algorithm	Provides examples

5. Overfitting typically corresponds to low training error and _____ test error.

Level 2

6. The *population risk* of a model is the _____ loss with respect to the true data distribution.

7. Why can population risk not be computed directly?

   (a) The loss function is unknown
   (b) The model is random
   (c) The true data distribution is unknown
   (d) The dataset is finite

8. A model has population risk 0.35 and empirical risk 0.25. Compute the generalisation gap.

9. **True or False.** Empirical risk is a random quantity because it depends on the sampled dataset.

10. **True or False.** The *generalisation gap* is the difference between expected risk and population risk.

Level 3

11. Which statement best characterises the Bayes model?
    (a) It is the trained model with lowest empirical risk
    (b) It minimises population risk over a fixed function class
    (c) It minimises population risk over all measurable functions
    (d) It is the model returned by gradient descent

12. **True or False.** The Bayes model is the optimal model based on the training set $\mathcal{S}$.

13. A model has generalisation gap $\Delta_f < 0$. Does this indicate that the empirical risk is an overestimate or an underestimate of the population risk?

14. A regression model produces predictions

$$f(\boldsymbol{x}_1) = 2, \quad f(\boldsymbol{x}_2) = 3, \quad f(\boldsymbol{x}_3) = 5,$$

for labels

$$y_1 = 1, \quad y_2 = 2, \quad y_3 = 4.$$

Using $\ell(y, f(\boldsymbol{x})) = (y - f(\boldsymbol{x}))^2$, compute the empirical risk $\hat{R}(f, \mathcal{S})$.

15. At a fixed input x, the conditional distribution of y is

$$\mathbb{P}(y = 0 \mid \boldsymbol{x}) = 0.2, \qquad \mathbb{P}(y = 2 \mid \boldsymbol{x}) = 0.8.$$

Compute the Bayes model $y^*(\boldsymbol{x})$ under squared loss.

Proofs
You will not be expected to reproduce any proof in the exam.
However, certain questions may be easier if you understand the proof.

Proof of the Bayes model for Squared loss

Theorem 1.1. *The population risk minimizer (also known as the Bayes model) for the squared loss is the expected label, given an input:*

$$y^* \;:=\; \arg\inf_z \mathbb{E}_{\boldsymbol{x}}\Big[\mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}\Big[(y-z)^2\Big]\Big] \;=\; \mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}[y]. \tag{11}$$

Proof. For any predictor $f(\boldsymbol{x})$, the population risk with squared loss can be written,

$$R(f) = \mathbb{E}_{\boldsymbol{x}}\Big[\mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}\Big[(y-f(\boldsymbol{x}))^2\Big]\Big]. \tag{12}$$

Our strategy will be to show that for every $\boldsymbol{x}$, the minimum value of the loss is obtained at $f(\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}[y]$. Take the inner expectation in Equation 12 and expand the square:

$$\mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}\Big[(f(\boldsymbol{x})-y)^2\Big] \;=\; \mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}\Big[f(\boldsymbol{x})^2 + y^2 - 2yf(\boldsymbol{x})\Big] \tag{13}$$

To find the minimum, we differentiate with respect to $f(\boldsymbol{x})$, and set to zero:

$$\frac{\partial}{\partial f(\boldsymbol{x})}\mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}\Big[f(\boldsymbol{x})^2 + y^2 - 2yf(\boldsymbol{x})\Big] \;=\; \mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}\Big[2f(\boldsymbol{x}) - 2y\Big] \;=\; 0 \tag{14}$$

Now by linearity of expectation,

$$\mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}\Big[2f(\boldsymbol{x}) - 2y\Big] \;=\; 2f(\boldsymbol{x}) - 2\mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}[y] \;=\; 0. \tag{15}$$

Solving for $f(\boldsymbol{x})$, we find that $f(\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}[y]$, which proves the theorem. □

2 The Probabilistic View of Learning

Outline: This week you learn the deeper meaning behind the notion of *risk* introduced last week. We will see that loss functions are a natural consequence of probabilistic assumptions that we make about the world. Overall we establish probability theory as the language in which learning is defined.

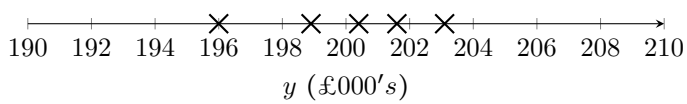
Reminder... you may need to brush up on your probability theory
This course is about the theoretical foundations of Machine Learning. The language in which this is expressed is primarily *probability theory*. We expect you to come with a certain level of understanding, but only that taught at A-level, and in the first year of CS at Manchester, in your *Mathematics for Computer Science* course. Some material that may help you with all this is provided in Appendix B.

2.1 The Principle of Maximum Likelihood

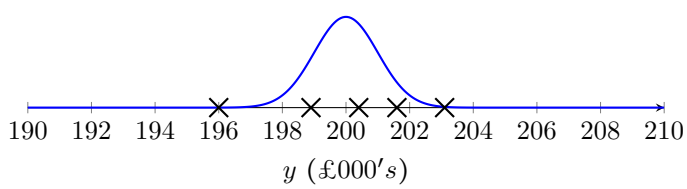
In this subsection we'll cover a topic that *seems* a little disconnected from your current understanding of ML, but in fact it's leading to a deeper connection. Stick with me, you'll understand in 2 pages time.

Let's imagine we are provided with a dataset of pairs $(\boldsymbol{x}, y)$, which come from some underlying process. **We'll use a running example of house sales**. The vector $\boldsymbol{x}$ contains characteristics like the number of bedrooms, size of garden, age of property, etc. And, y is the price for which the house was sold. Now, selling houses is not a simple process—two houses, with *exactly* the same specification $\boldsymbol{x}$, might sell for different prices, due to *unobserved* factors like how desperate the sellers are to move, or the negotiating skills of the buyers. For this reason, it is natural to treat the data probabilistically.

To make this idea concrete, let's pretend we have 5 houses, with *identical* specification $\boldsymbol{x}$. We sell them, and the sold prices $\{y_1, \cdots, y_5\}$ in £000's are $\{196.0, 198.9, 200.4, 201.6, 203.1\}$, and plotted below:



If we saw a house with this exact $\boldsymbol{x}$ again, we'd have some uncertainty about the price. To represent this formally, we will assume each point y_i is an observation from a conditional probability distribution, $p(y \mid \boldsymbol{x})$. If we say $p(y \mid \boldsymbol{x})$ is Gaussian, what would be a plausible location for it on this axis? i.e. what mean μ is most likely to have generated those points? I hope you're now saying to yourself "about $\mu = 200$", like this:



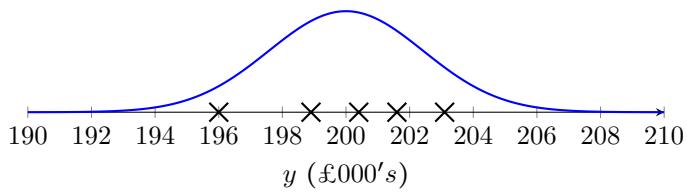
We can now evaluate the *likelihood* of each y_i under that Gaussian, by simply plugging it into the formula, as below. For example $p(y = 196 \mid \mu = 200, \sigma = 1) \approx 0.0001$, or 0.0001338 to be more precise.

y_i	$p(y_i \mid \boldsymbol{x}; \mu = 200, \sigma = 1)$
196.0	0.0001
198.9	0.2178
200.4	0.3683
201.6	0.1109
203.1	0.0033

$$p(y_i \mid \boldsymbol{x}; \mu = 200, \sigma = 1) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(y_i - \mu)^2}{2\sigma^2}\right)$$

Note that these values **are not probabilities** (i.e. numbers in $[0, 1]$), but instead the value of the probability density evaluated at different points, which we refer to as a *likelihood*.

The Gaussian above has mean parameter $\mu = 200, \sigma = 1$, but we could also change σ. Below is $\sigma = \sqrt{5.908}$.



These values didn't come out of nowhere: they are $\mu = \frac{1}{5}\sum_{i=1}^{5} y_i = 200$ and $\sigma = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(y_i - \mu)^2} = \sqrt{5.908}$.

With $\sigma = 1$, the leftmost point, $y_1 = 196$, was quite unlikely. With $\sigma = \sqrt{5.908}$, it is much more likely. In fact, this setting $\mu = 200, \sigma = \sqrt{5.908}$ is *most* likely to have generated the data, i.e. *maximum likelihood*.

y_i	$p(y_i \mid \boldsymbol{x}; \mu = 200, \sigma = \sqrt{5.908})$
196.0	0.0423
198.9	0.1482
200.4	0.1619
201.6	0.1322
203.1	0.0728

$$p(y_i \mid \boldsymbol{x}; \mu = 200, \sigma = \sqrt{5.908}) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(y_i - \mu)^2}{2\sigma^2}\right)$$

This principle, of choosing parameters for a probability distribution such that it is the most likely configuration to generate our observed data, is key to understanding the probabilistic view of learning.

Now, we have 5 *likelihood values*, one for each data point. We'd like a summary number that combines these. Since these are assumed to be independent and identically distributed events, we take the product:

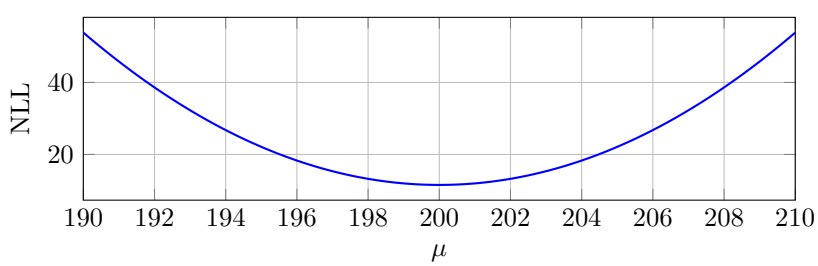
$$\mathcal{L}(\theta \mid \mathcal{S}) = \prod_{i=1}^{n} p(y_i \mid \boldsymbol{x}_i; \theta). \tag{16}$$

where $\theta = \{\mu, \sigma\}$ in the case of our Gaussian, but the principle is more generally applicable. For historical reasons, this is referred to as the **likelihood of the parameters, given the data**, denoted $\mathcal{L}(\theta \mid \mathcal{S})$. This may seem backwards (and it kind of is, a little) but it's established terminology, so do try to internalise it.

Now, as good Computer Scientists, we know that a product of these very small numbers will be an even smaller number. And, we'll quickly get beyond machine precision if we have much more than 5 data points. So, in practice we take the **logarithm** of this, and furthermore we usually **negate** it, turning the maximisation problem into a minimisation one. This gives us an expression called the **negative log-likelihood.**

$$NLL(\theta \mid \mathcal{S}) \;:=\; -\ln\left(\prod_{i=1}^{n} p(y_i \mid \boldsymbol{x}_i; \theta)\right) \tag{17}$$

The NLL of our 5 points, for $\mu = 200, \sigma = 1$ is about 19.36, but for $\sigma = \sqrt{5.908}$, it's about 11.53, much lower. Equally, the NLL changes as we vary μ, and it minimized at $\mu = 200$. The full plot is below.



I hope you're now starting to see the connection... we will make it clear in the next subsection.

2.2 How does all this probability theory relate to good old machine learning?!

Let's review terminology. The *true and unknown distribution* behind the data is $p(y \mid \boldsymbol{x})$. We hypothesize what the form of that might be, coming up with a model distribution $p(y \mid \boldsymbol{x}; \theta)$. We then try to find settings for θ that get it as close as possible to the true distribution.

To take a concrete example, we will say $p(y \mid \boldsymbol{x}; \theta)$ is Gaussian again, so $\theta = \{\mu, \sigma\}$, but assume a fixed $\sigma = 1$. To get the mean μ, we will have a function $\mu \leftarrow f(\boldsymbol{x})$. So, our model distribution will be $p(y \mid \boldsymbol{x}; f(\boldsymbol{x}))$. For our toy data above, this $f(\boldsymbol{x})$ should just return 200, but in general, it will *predict* what the mean should be, given the input $\boldsymbol{x}$. Given this, let's write down the explicit formulae for the NLL in our simple 5 data point example.

$$NLL(\theta \mid \mathcal{S}) \;=\; -\ln\left(\prod_{i=1}^{5} p(y_i \mid \boldsymbol{x}_i; f(\boldsymbol{x}_i))\right) \tag{18}$$

where

$$p(y_i \mid \boldsymbol{x}_i; f(\boldsymbol{x}_i)) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(y_i - f(\boldsymbol{x}_i))^2}{2\sigma^2}\right). \tag{19}$$

We now plug these together, and we get

$$\begin{aligned}
NLL(\theta \mid \mathcal{S}) &= -\ln\left(\prod_{i=1}^{5} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(y_i - f(\boldsymbol{x}_i))^2}{2}\right)\right) && (20)\\
&= -\sum_{i=1}^{5} \ln\left(\frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(y_i - f(\boldsymbol{x}_i))^2}{2}\right)\right) && (21)\\
&= 5\ln\left(\sqrt{2\pi}\right) + \sum_{i=1}^{5} \frac{1}{2}\Big(y_i - f(\boldsymbol{x}_i)\Big)^2 && (22)\\
&= \sum_{i=1}^{5} \frac{1}{2}\Big(y_i - f(\boldsymbol{x}_i)\Big)^2 + const && (23)
\end{aligned}$$

This is the sum of squared losses, for our model $f(\boldsymbol{x})$ as it predicts what the mean should be. Remember, we want to *minimize NLL*. The constant here makes no difference to the problem, so we can ignore it. **What we see here is that maximizing likelihood is equivalent to minimizing squared loss**.

> Assuming the label y comes from a Gaussian with fixed variance, the problem of maximizing likelihood is equivalent to minimizing the sum of squared loss across the dataset.

Furthermore in practice, we will **average** rather than **sum** the terms. This is because the sum might be problematic, as the value of the NLL will depend on the size of the dataset. So we reach our final expression,

$$\max_{\theta} \mathcal{L}(\theta \mid \mathcal{S}) \longleftrightarrow \min_{f} \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, f(\boldsymbol{x}_i)) \tag{24}$$

which states that maximizing the likelihood of the parameters given the data is equivalent to minimizing an average loss across the dataset. You might think this a bit convoluted, but in fact it's taught us something deep: **the loss function is a direct consequence of a particular assumption about the way the data was generated.** Going back to the phrasing in the outline section of this chapter:

> Loss functions are not just arbitrary inventions. In fact, they are a natural consequence of probabilistic assumptions that we make about the world.

So, when we use squared loss to fit a model, what we are doing is equivalent to predicting the mean of a Gaussian distribution, with fixed variance.

What if we take another learning problem, with a different distribution?

Binary Classification via Bernoulli Likelihood

Let's say we want to predict not the price, but instead a simple Yes/No, for whether or not the house will sell in the next month. We have a historical dataset $\mathcal{S}$, where $\boldsymbol{x}_i$ is again the house characteristics, but $y_i = 1$ if the house was on the market less than 4 weeks, and $y_i = 0$ if it took longer to sell.

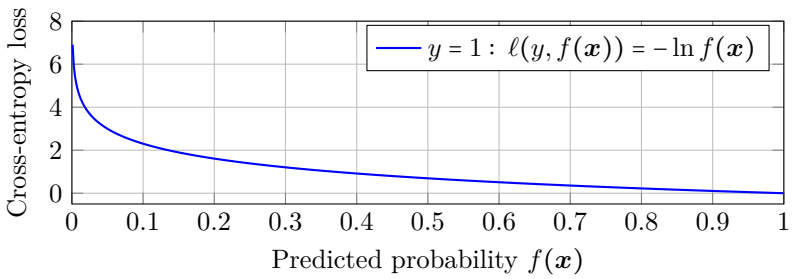
A Bernoulli distribution has just one parameter, $\mu \in (0, 1)$, and takes the form:

$$p(y \mid \mu) = \mu^y (1-\mu)^{(1-y)} \tag{25}$$

We again assume we predict the parameter μ using a model $f(\boldsymbol{x})$. We take the negative log likelihood:

$$\begin{aligned} NLL(\mu \mid \mathcal{S}) &= -\ln\left(\prod_{i=1}^{n} f(\boldsymbol{x}_i)^{y_i}(1-f(\boldsymbol{x}_i))^{(1-y_i)}\right) && (26) \\ &\propto \frac{1}{n}\sum_{i=1}^{n}\left[-\Big(y_i \ln f(\boldsymbol{x}_i) + (1-y_i)\ln(1-f(\boldsymbol{x}_i))\Big)\right]. && (27) \end{aligned}$$

This is the **binary cross-entropy**, commonly used for training neural networks.



Learning a model by minimizing binary cross-entropy is equivalent to maximizing likelihood where $p(y \mid \boldsymbol{x}; \theta)$ is a Bernoulli distribution.

The whole pipeline: from probabilistic assumptions to loss functions

Rather than predicting a single value of y given $\boldsymbol{x}$, the model is assumed to predict the *parameters* of a conditional probability distribution. Concretely, we consider a model $\theta \leftarrow f(\boldsymbol{x})$, i.e. the model predicts a value θ which is used as the parameter of a probability distribution.

The connection between probabilistic modelling and empirical risk minimization can be summarized as the following sequence of steps:

1. ***Specify a conditional distribution*** $p(y \mid x; \theta)$ ***to model how outputs are generated from inputs.***
2. ***Let the predictive model predict its parameters, i.e.*** $\theta = f(\boldsymbol{x})$***.***
3. ***Let the loss function*** $\ell\big(y, f(\boldsymbol{x})\big)$ ***be the negative log-likelihood:***

$$\ell\big(y, f(\boldsymbol{x})\big) = -\ln p\big(y \mid \boldsymbol{x}; f(\boldsymbol{x})\big).$$

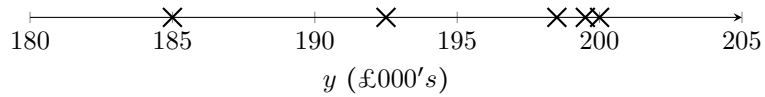
4. ***Train the model by minimizing the empirical average of this loss:***

$$\min_f \frac{1}{n}\sum_{i=1}^{n} \ell\big(y_i, f(\boldsymbol{x}_i)\big).$$

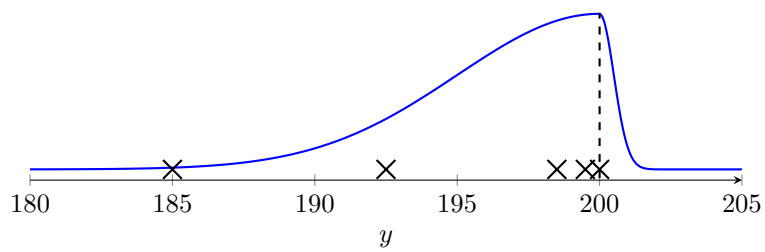
This is the general conceptual pipeline, demonstrating how deeply intertwined probability theory is with Machine Learning. We've seen it invoked for two relatively simple cases of Gaussian and Bernoulli, which underlie the squared loss and cross-entropy loss, respectively. Let's see one more, a bit more complex.

2.3 Deriving an asymmetric loss for predicting house prices

We consider how to predict the house price, but under certain market conditions. Let's pretend we observe some data, again for a fixed $\boldsymbol{x}$, with $\{y_1, \cdots, y_5\} = \{185.0, 192.5, 198.5, 199.5, 200.0\}$.



In a harsh market, houses tend to sell for slightly *less* than their asking price. So, a Gaussian isn't such a good modelling choice, as it's symmetric. Instead, a better fit might be a *split normal distribution:*



A split normal is a Gaussian with different variances on each side of the mode, marked with the dashed line. The probability density function of a split normal is:

$$p(y \mid \boldsymbol{x}; \theta) = \frac{\sqrt{2}}{\sqrt{\pi}\left(\sigma_R + \sigma_L\right)} \exp\left(-\frac{\left(y - m\right)^2}{2\,\sigma(m)^2}\right),$$

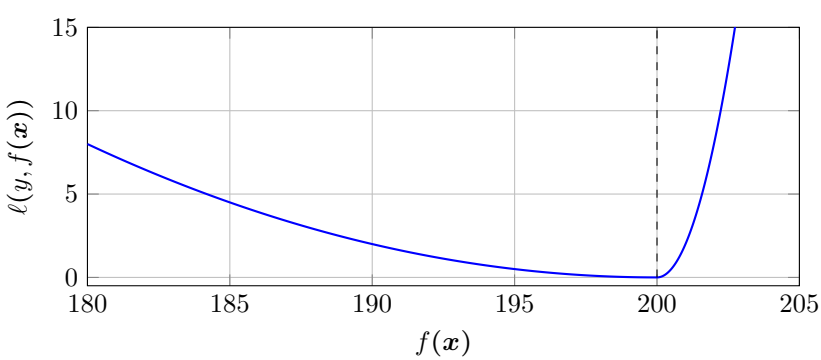
where the parameters $\theta = \{m, \sigma_R, \sigma_L\}$ are the mode m, and the variance parameters σ_R, σ_L, which depend on whether the y is on the left/right of the mode.

$$\sigma(m) = \begin{cases} \sigma_L, & y < m, \\ \sigma_R, & y \geq m. \end{cases}$$

Here we will assume we know $\sigma_L = 5$ and $\sigma_R = 0.5$. This is perfectly plausible to get via expert advice—you can imagine asking an estate agent/realtor what the typical distribution of sale prices might be, for a given house, with them "eyeballing" the distribution above until it looks right.

To get our loss, we take the NLL of the PDF, with $f(\boldsymbol{x})$ used to predict the mode:

$$\ell\big(y, f(\boldsymbol{x})\big) \;=\; -\ln p(y \mid \boldsymbol{x}; \theta) \;=\; \frac{1}{2\sigma(f(\boldsymbol{x}))^2}\big(y - f(\boldsymbol{x})\big)^2 + \ln\left(\frac{\sqrt{\pi}\left(\sigma_L + \sigma_R\right)}{\sqrt{2}}\right). \tag{28}$$



So we've *derived* an *asymmetric* loss, that is suitable for our problem—heavily penalising any prediction above the observed y_i, and more tolerant below. In practice the expression above can be further simplified by dropping constants—see if you can figure out which parts.

Let's review how we did this—we followed our pipeline as specified earlier:

1. ***Specify a conditional distribution*** $p(y \mid \boldsymbol{x}; \theta)$ ***to model how outputs are generated from inputs.***
   - Done. A split normal.
2. ***Let the predictive model predict its parameters, i.e.*** $\theta = f(\boldsymbol{x})$***.***
   - For simplicity, we assume we know σ_L, σ_R, and just predict the mode, so $m = f(\boldsymbol{x})$.
3. ***Let the loss function*** $\ell\big(y, f(\boldsymbol{x})\big)$ ***be the negative log-likelihood.***
   - $\ell\big(y, f(\boldsymbol{x})\big) = -\ln p(y \mid \boldsymbol{x}; f(\boldsymbol{x})) = \frac{1}{2\sigma(f(\boldsymbol{x}))^2}\big(y - f(\boldsymbol{x})\big)^2 + \ln\left(\frac{\sqrt{\pi}\left(\sigma_L + \sigma_R\right)}{\sqrt{2}}\right)$.
4. ***Train the model by minimizing the empirical average of this loss***

$$\min_f \frac{1}{n}\sum_{i=1}^{n} \ell\big(y_i, f(\boldsymbol{x}_i)\big).$$

In the probabilistic view of supervised learning, we assume a conditional probability model $p(y \mid \boldsymbol{x}; \theta)$ that describes how targets y are generated from inputs $\boldsymbol{x}$. The negative log-likelihood of this conditional distribution gives a loss function, which we then use to train a model for the parameters.

Any probability distribution can be used with NLL to derive a loss,
but not every loss comes from the NLL of a probability distribution.

When a loss can be written as the NLL of a probability distribution, it is called a *likelihood-based* loss. Not all losses have such an interpretation: for example, the 0/1 loss used in classification does not correspond to the NLL of any probability distribution, and is therefore not likelihood-based.

2.4 Relating all this back to the concept of 'Risk'

Last week we introduced two important quantities: empirical risk and population risk. We can now reinterpret both of these using the probabilistic framework developed in this section. When a loss function is the negative log-likelihood of a conditional distribution $p(y \mid \boldsymbol{x}; \theta)$, the empirical risk can be written as

$$\hat{R}(f, S) = -\frac{1}{n}\sum_{i=1}^{n} \ln p(y_i \mid \boldsymbol{x}_i; f(\boldsymbol{x}_i)) = \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, f(\boldsymbol{x}_i)).$$

This is simply the average negative log-likelihood evaluated on the observed dataset. The population risk of our model has an analogous expression:

$$R(f) = -\mathbb{E}_{\boldsymbol{x}y}[\ln p(y \mid \boldsymbol{x}; f(\boldsymbol{x}))] = \mathbb{E}_{\boldsymbol{x}y}[\ell(y, f(\boldsymbol{x}))]\,.$$

Here, the expectation is taken with respect to the true (and unknown) data-generating distribution, $p(y \mid \boldsymbol{x})$. In theory, we could also talk about the risk of this *true* distribution. $-\mathbb{E}_{\boldsymbol{x}\boldsymbol{y}}[\ln p(y \mid \boldsymbol{x})]$. The difference between these two can be written as:

$$\Delta = \underbrace{\Big(-\mathbb{E}_{\boldsymbol{x}y}[\ln p(y \mid \boldsymbol{x}; f(\boldsymbol{x}))]\Big)}_{\text{Risk of model distribution}} - \underbrace{\Big(-\mathbb{E}_{\boldsymbol{x}y}[\ln p(y \mid \boldsymbol{x})]\Big)}_{\text{Risk of true distribution}} = \mathbb{E}_{\boldsymbol{x}}\left[\mathbb{E}_{y|\boldsymbol{x}}\Big[\ln \frac{p(y \mid \boldsymbol{x})}{p(y \mid \boldsymbol{x}; f(\boldsymbol{x}))}\Big]\right] \tag{29}$$

This is the "excess" risk, that the model distribution gives us, on top of that of the true distribution.

There is a term here is known as the *Kullback-Leibler* divergence between the distributions, which we have averaged over all $\boldsymbol{x}$. Let's go a bit deeper into this.

We will take an example of a *categorical distribution*, this is a generalization of the Bernoulli to multiple outcomes (not just binary) also known as a *Multinoulli* distribution.

First, let's expand the expression above:

$$D_{KL}\big(p(y \mid \boldsymbol{x}) \,\|\, p_f(y \mid \boldsymbol{x}; f(\boldsymbol{x}))\big) = \mathbb{E}_{y|\boldsymbol{x}}\left[\ln \frac{p(y \mid \boldsymbol{x})}{p(y \mid \boldsymbol{x}; f(\boldsymbol{x}))}\right] \tag{30}$$

$$= \sum_{c=1}^{k} p(y = c \mid \mathbf{x})\left[\ln \frac{p(y \mid \mathbf{x})}{p(y \mid \mathbf{x}; f(\mathbf{x}))}\right] \tag{31}$$

Let's imagine our true distribution is $p(y \mid \boldsymbol{x}) = [0.1, 0.1, 0.8]$, and our model is $p(y \mid \boldsymbol{x}; f(\boldsymbol{x})) = [0.1, 0.3, 0.6]$. Then, the *excess risk* of the model over the true distribution, or the KL-divergence between them, is:

$$D_{KL}(p \,\|\, p_f) = \left(0.1 \times \ln \frac{0.1}{0.1}\right) + \left(0.1 \times \ln \frac{0.1}{0.3}\right) + \left(0.8 \times \ln \frac{0.8}{0.6}\right) \approx 0.120284 \tag{32}$$

The expression above reveals something deeper than a numerical calculation. When the loss is the negative log-likelihood of a conditional model $p(y \mid \boldsymbol{x}; f(\boldsymbol{x}))$, the population risk can be decomposed as

$$R(f) = -\mathbb{E}_{\boldsymbol{x}y}[\ln p(y \mid \boldsymbol{x})] + \mathbb{E}_{\boldsymbol{x}}\big[D_{\mathrm{KL}}\big(p(y \mid \boldsymbol{x}) \,\|\, p(y \mid \boldsymbol{x}; f(\boldsymbol{x}))\big)\big].$$

The first term depends only on the true data-generating distribution. It does not depend on our model, and therefore cannot be reduced by learning. It represents the intrinsic uncertainty of the world. In analogy with last week, this plays the role of the irreducible risk.

The second term is the part we can influence. It is the expected Kullback–Leibler divergence between the true conditional distribution and the one predicted by our model. This quantity is always non-negative, and it is zero if and only if the two conditional distributions match exactly.

Thus, under likelihood-based losses, learning can be interpreted as distribution matching. Minimizing population risk is equivalent to minimizing an expected KL divergence to the true conditional distribution. The quality of a learned model is therefore governed by how close its predicted distribution is, on average over inputs x, to the true data-generating mechanism.

2.5 Summary

Loss functions can appear to be arbitrary design choices, but they are not. Within a probabilistic view of learning, losses arise naturally as the negative log-likelihood of an assumed conditional distribution $p(y \mid \boldsymbol{x})$. This provides a principled justification for common losses such as squared error and cross-entropy, and allows domain assumptions to be encoded directly into the learning objective.

In the next section, we will see that because the dataset itself is random, this KL divergence becomes a random quantity—leading us directly to an important new topic, the *bias–variance decomposition.*

The Probabilistic View of Learning — Practice Exam Questions

Each week we provide you a few practice questions, representative of what may be in the final written exam. The exam will be in Canvas, so all questions match this style. Answers provided separately.

If you can answer all of **Level 1**... you're on track, but doing the bare minimum to pass.
If you can answer all of **Level 2**... you're doing well, more than enough to pass.
If you can answer all of **Level 3**... well ahead, into first class territory. I expect 20% of the class here.

Level 1

1. When a loss function can be written as

$$\ell(y, f(\boldsymbol{x})) = -\ln p(y \mid \boldsymbol{x}; f(\boldsymbol{x}))$$

   it is called a _____-based loss.

2. True or False. Minimizing the negative log-likelihood of a Gaussian with an unknown and moving data-dependent variance is equivalent to minimizing squared loss.

3. Which of the following best describes *empirical risk*?
   (a) Expected loss under the true data-generating distribution
   (b) Loss of the Bayes-optimal predictor
   (c) Average loss over a finite dataset
   (d) Difference between training and test loss

4. Match each object to its role.

Object	Role
$A.\ p(y \mid \boldsymbol{x}; \theta)$	(1) Average loss over observed data
$B.\ \ell(y, f(\boldsymbol{x}))$	(2) Assumed conditional distribution
$C.\ \hat{R}(f, S)$	(3) Negative log-likelihood
$D.\ R(f)$	(4) Expected loss under the true distribution

5. True or False. The value of the empirical risk depends on which dataset was sampled.

Level 2

6. Explain briefly why population risk cannot be evaluated directly in practice.

7. True or False. If a loss is likelihood-based, then minimizing empirical risk is equivalent to maximizing the likelihood of the model parameters given the data.

8. A student claims: *"If we fix the model class and the algorithm, the learned predictor is fixed."*

   Is this statement correct? Give a one-sentence justification.

9. Which of the following best explains why the learned model should be viewed as a random variable?
   (a) The optimisation algorithm is iterative
   (b) The loss function is stochastic
   (c) The training dataset is a random sample
   (d) The model parameters are real-valued

10. Consider a model trained using squared loss. What probabilistic assumption about $p(y \mid \boldsymbol{x})$ is implicitly being made?

Level 3

11. Let a model be trained using a likelihood-based loss

$$\ell(y, f(\boldsymbol{x})) = -\ln p(y \mid \boldsymbol{x}; f(\boldsymbol{x})).$$

Write down an expression for the empirical risk $\hat{R}(f, \mathcal{S})$ in terms of the dataset $\mathcal{S} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$.

12. Consider two **Bernoulli** distributions:

- the true conditional distribution $p(y \mid \boldsymbol{x})$,
- the model distribution $p(y \mid \boldsymbol{x}; f(\boldsymbol{x}))$.

Explain why the quantity

$$R(f) = -\mathbb{E}_{\boldsymbol{x}y}[\ln p(y \mid \boldsymbol{x}; f(\boldsymbol{x}))]$$

is always non-negative, and state precisely conditions under which it is equal to zero.

13. Suppose a model is trained by minimizing empirical risk using a likelihood-based loss. Explain why, even if the optimisation algorithm is deterministic, the final predictor should still be regarded as a random object.

14. True or False (justify your answer): Population risk depends on the particular training dataset that was observed.

15. A student argues: *"If I choose a different conditional distribution $p(y \mid \boldsymbol{x}; \theta)$, but still minimize empirical risk, then I am solving the same learning problem."*

Is this statement correct? Give a concise explanation that explicitly refers to the role of probabilistic assumptions and the form of the empirical risk.

3 The Problem with Learning a Model

Outline: The problem with learning a model is... sometimes you can learn too well. What I mean is, that you can end-up *fine-tuning* the model too much, focusing on little things in your (relatively small) training set, that might not be real phenomena in the world. The technical name for this is **over-fitting**, as in we *over-fit* the model to the data. This week provides you with a formal mathematical grounding for these ideas: the *bias-variance* decomposition.

3.1 Our training set is finite, and has hidden sources of randomness

Imagine you have a training set $\mathcal{S} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$, and train a model with some learning algorithm. Now, recall that $\mathcal{S}$ is the result of some data-collection procedure. Maybe a person had to walk around measuring things, taking photos, or whatever else necessary to get this data to you. If that data-collection procedure had been (randomly) just a little different, then our training set would have been different too. Maybe, the actual data we have is not a good representation of the underlying process?

As you know, we assume $\mathcal{S}$ was *sampled* from a probability distribution $p(\boldsymbol{x}, y)^n$, and we could have instead obtained a slightly different $\mathcal{S}'$, drawn i.i.d. from the same distribution. We would like our learning algorithm to be resilient to small changes in its training data. To address this, we introduce the *expected population risk*, which explicitly averages over possible training sets.

Definition 8 (Expected Population Risk)**.** *Given a random training sample $\mathcal{S} \sim p(\boldsymbol{x}, y)^n$, the expected population risk of a trained model $f_{\mathcal{S}}(\boldsymbol{x})$ is defined,*

$$\mathbb{E}_{\mathcal{S}}\Big[R(f_{\mathcal{S}})\Big] = \mathbb{E}_{\mathcal{S}}\Big[\ \mathbb{E}_{\boldsymbol{xy}}\Big[\ \ell\Big(y, f_{\mathcal{S}}(\boldsymbol{x})\Big)\ \Big]\ \Big] \tag{33}$$

where you'll see additional notation $f_{\mathcal{S}}(\boldsymbol{x})$, making explicit that the model was trained on the random sample $\mathcal{S}$. You can think about this as the *average* population risk, over *all possible* $\mathcal{S}$, that might be drawn from $p(\boldsymbol{x}, y)^n$. Most of the time, we just say **expected risk**, i.e. without the prefix "population".

The **expected risk** of a model is the population risk averaged over all possible training sets of size n.

Why are we pretending to sample training sets? Because, the key question is not how well a single trained model performs, but how the learning algorithm might behave across possible training sets. If it returns us brilliant predictions no matter what training set it receives, well that's a valuable learning algorithm. If it only works well given a particular high quality training set, then perhaps... less valuable. We can also consider the notion of the *expected model*.

Definition 9. *The expected model is the hypothetical model that returns the prediction averaged over all possible training sets, $\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]$.*

The expected model represents the behavior of the algorithm when we average out the randomness in the source of training data. You could think of this informally like a 'committee' of all possible models. We introduce the expected model because the key ideas for this week are properties of the learning algorithm, not of any single trained model, or any single training set.

The **expected model** is the prediction obtained by averaging over all possible training sets of size n.

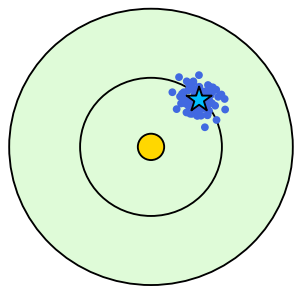
Now given all this, a natural question arises. If the learned model $f_{\mathcal{S}}$ depends on the particular training set $\mathcal{S}$, what aspects of its performance are due to *systematic* properties of the learning algorithm, and what aspects are due purely to the *randomness* of the data we happened to observe? These two components (systematic vs random) have special names: **bias** and **variance**.

3.2 Bias and Variance: A Metaphor

Before we get into the mathematics, we consider an intuitive (but obviously not precise) way to understand the concepts of bias and variance.

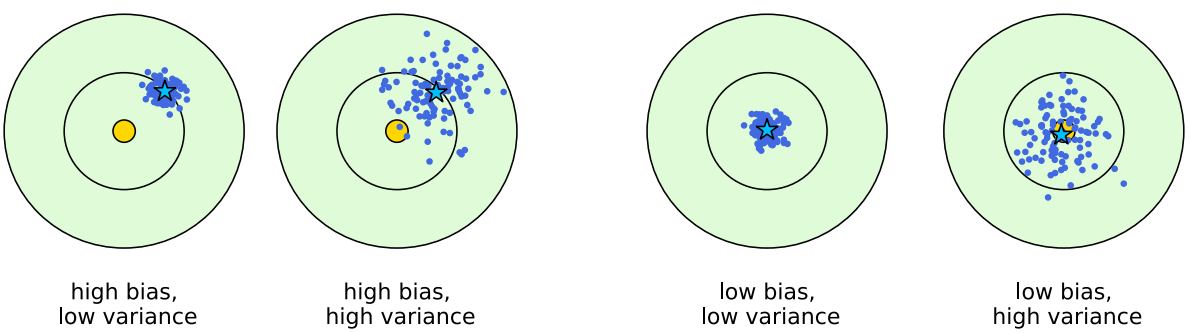
Imagine I train a model to throw darts at a dartboard. Let's assume I have some way to generate training data sets. I generate some training data $\mathcal{S}$, train a model, and make a prediction for some fixed test point $\boldsymbol{x}$. Then, I repeat this, but *with a different training set,* and try to predict that same test point again. In everything that follows, the test point $\boldsymbol{x}$ and its true label y are fixed. All randomness comes only from which training set $\mathcal{S}$ we happened to sample.

In the visualisation below, the yellow bullseye is the target, i.e. the y label value, for our fixed test point $\boldsymbol{x}$. The blue dots are predictions from models, each *trained from a different randomly generated training set.*



You can see variation in the prediction, due to changes in the training set provided. This spread is called model **variance**. It is the *random* component of learning, due to randomly sampled data. The center of the point-cloud is *offset from the target.* This is called model **bias**. It is the *systematic* part of the learning: being close to, or far away from, where it really should be. One way to think about it is that the bullseye is what the *Bayes-optimal* model[3] would predict. So the bias is the offset from the Bayes model prediction.

We can imagine the point-cloud anywhere on the dartboard, and having smaller/larger spread. This gives rise to four canonical scenarios, that we might expect to see.



We'd obviously like to get close to the bullseye, with low spread—i.e. low bias and low variance. The bias/variance are controlled by characteristics of the model and learning algorithm.

Of course, this dartboard is only a metaphor. It is useful for building intuition, but it does not yet tell us how to *measure* bias and variance precisely. Our goal now is to turn this qualitative picture into a quantitative statement. We will do this by *decomposing* the expected risk into a sum of two terms: one capturing the average prediction, and one capturing variability across training sets.

[3] Remember subsection 1.4?

3.3 From Metaphors to Mathematics: A formal definition of bias & variance

We now turn the intuitive dartboard picture into a precise mathematical statement. Throughout this section we restrict attention to the squared loss, since it admits a particularly clean decomposition. Similar decompositions exist for other losses, but the algebra is less transparent.

Recall that the training set $\mathcal{S} \sim p(\boldsymbol{x}, y)^n$ is random. As a consequence, the learned predictor $f_{\mathcal{S}}(\boldsymbol{x})$ is itself a random function. For a fixed *testing point* $\boldsymbol{x}$, all randomness in our prediction comes from this sampled training set $\mathcal{S}$. This variation is what we visualised/imagined as the spread of points hitting the dartboard. The center of the spread of points is the prediction from the *expected model*, $\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]$. The first mathematical result we want to consider is how the expected squared loss at $\boldsymbol{x}$ decomposes as as follows.

$$\mathbb{E}_{\mathcal{S}}\Big[(y - f_{\mathcal{S}}(\boldsymbol{x}))^2\Big] = \underbrace{(y - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})])^2}_{\text{bias at } \boldsymbol{x}} + \underbrace{\mathbb{E}_{\mathcal{S}}\Big[(f_{\mathcal{S}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})])^2\Big]}_{\text{variance at } \boldsymbol{x}}. \tag{34}$$

This is the exact mathematical analogue of the dartboard metaphor. The bias is the distance from the center of the point cloud to the bullseye. The variance is the spread of the points. The expected (average) distance from each point to the bullseye is equal to the sum of these two. If the algorithm always returned the same predictor regardless of $\mathcal{S}$, the variance term would be zero. If the expected prediction matched the target, then the bias would be zero.

Now, this metaphor really only holds for a single test point. In reality of course we will have a large set of test data. If we average the result above over all possible test points, we get the following more general noise/bias/variance decomposition.

Theorem 3.1 (Bias–Variance Decomposition for Squared Risk)**.** *Given a learning algorithm that produces a predictor $f_{\mathcal{S}}$ from a random training set $\mathcal{S}$, the expected risk satisfies*

$$\underbrace{\mathbb{E}_{\mathcal{S}}\Big[R(f_{\mathcal{S}})\Big]}_{\text{expected risk}} = \underbrace{\mathbb{E}_{\boldsymbol{xy}}\Big[\,(y - y^*(\boldsymbol{x}))^2\,\Big]}_{\text{noise (Bayes risk)}} + \underbrace{\mathbb{E}_{\boldsymbol{x}}\Big[(y^*(\boldsymbol{x}) - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})])^2\Big]}_{\text{bias}} + \underbrace{\mathbb{E}_{\boldsymbol{x}}\Big[\mathbb{E}_{\mathcal{S}}\left[(f_{\mathcal{S}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})])^2\right]\Big]}_{\text{variance}}. \tag{35}$$

The first term is the **noise**, also known as the Bayes risk. It is an irreducible quantity determined entirely by the conditional distribution $P(y \mid \boldsymbol{x})$ and cannot be reduced by any learning algorithm. This term exists even with infinite data and a perfect learning algorithm.

The second term is the **bias**. It measures the squared loss between the Bayes prediction and the average prediction of the learning algorithm. Bias captures systematic error introduced by model assumptions or algorithmic restrictions.

The third term is the **variance**. It quantifies how sensitive the learned model is to the particular training set it receives. Highly flexible models tend to fit each dataset very closely, leading to large variability across training sets and, consequently, high variance.

The dartboard metaphor illustrates bias and variance at a single input $\boldsymbol{x}$. The formal decomposition above aggregates this behaviour across all inputs, weighted by how frequently they occur in the data distribution.

Summary: Bias measures how far the average learned predictor is from the Bayes predictor.
Variance measures how much the learned predictor fluctuates across training sets.
Noise measures uncertainty in the labels that no algorithm can remove.

3.4 The bias/variance trade-off can help understand overfitting

Let's say we have some learning problem to solve. We build regression trees of increasing depth, trying to reduce the expected risk, that is, the risk *on average* across variations in our training set.

Plotted below, on the x-axis, we increase the depth of the tree—which makes it more complex, or more capable of fitting data. The faint red lines are 50 repeats of the model being trained from scratch with a newly sampled training set. The bold red line is the *average* of these, i.e. an estimate of the *expected risk* of the model. The orange/blue lines are the **variance**, and **bias**, and there is a third (black) line the **noise**.

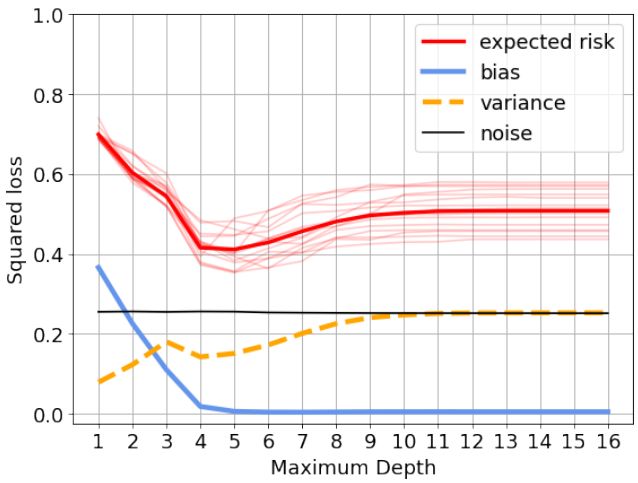


Figure 10: Building regression trees of increasing depth.

As tree depth increases, the model goes from very simple, to very complex. As this happens, the bias tends to *decrease*, and the variance tends to *increase*. You can see that this increasing variance is causing the red line to rise, and hence **is the cause of the over-fitting**.

This is commonly referred to as the *bias-variance trade-off*. The tree depth is our proxy for how *powerful* the model is, or how much *capacity* it has to learn. If it is too 'powerful' it tends to pick up on the noise present in the training data, and over-fit to it. Whilst we showed this for decision trees, the same principle applies for *all models*—though for some, the trade-off is not so straightforward.

Linear regression (high bias, low variance) If the true relationship is non-linear, linear regression may exhibit high bias, leading to underfitting. Variance is generally small in linear regression. You can control the trade-off with a technique called L2 weight regularization.

Decision trees (bias/variance managed by depth) Tree can have low bias, as they can represent complex relationships in the data. However, decision trees are prone to high variance, especially when they grow deep and capture noise in the training data. Techniques like tree "pruning" can be applied to control variance and avoid over-fitting.

k-Nearest Neighbors (bias/variance managed by k). The value of k defines the complexity of the k-nn model. With $k = 1$, it is very complex. With larger k, it becomes more simple. A k-NN can have thus have low bias with an appropriate setting. However, it may suffer from high variance, particularly in the presence of noisy data or irrelevant features.

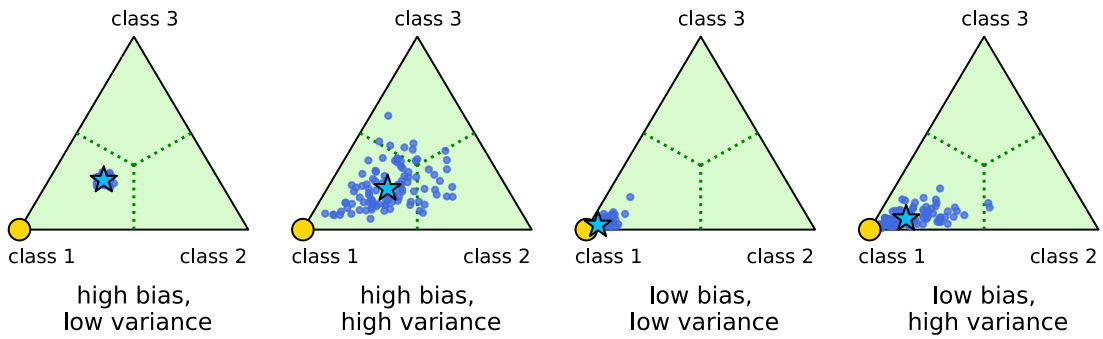
Deep Neural Networks (generally low bias, high variance, depending on regularization) Neural networks, especially deep ones, can model highly complex relationships and so are *low bias* models. However, they are prone to high variance, if the network is too large. Regularization techniques, such as "dropout", are often used to manage variance.

3.5 The decomposition holds for many losses, but not all losses

We've today focused on squared loss, but a similar decomposition also applies for cross-entropy, i.e.

$$\ell_{ce}(y, f(\boldsymbol{x})) \;=\; -\Big(y \ln f(\boldsymbol{x}) + (1-y)\ln(1-f(\boldsymbol{x}))\Big) \tag{36}$$

We won't go into the maths, but the same dartboard analogy applies, just in the *probability simplex* instead of the round dartboard. Illustrating this for the case of $k = 3$ classes, this would be:



Here, the blue dots are still predictions varying with training sample, but the blue star is no longer the *expected model*. Instead we have a more general concept called the *'centroid model'*, which respects the geometry of the cross-entropy loss function.

It is important to note that decompositions do not hold for ALL losses. For example, for the 0/1 loss, also known as the classification error. We assume a model outputting a discrete label $1, \ldots, k$, then the 0/1 loss is defined:

$$\ell_{0/1}(y, f(\boldsymbol{x})) = \begin{cases} 0, & \text{if } y = f(\boldsymbol{x}), \\ 1, & \text{if } y \neq f(\boldsymbol{x}). \end{cases} \tag{37}$$

i.e. the loss is 0 if the prediction is *exactly equal* to the label, and 1 otherwise.

The bias/variance decomposition does not hold for this loss. And, counterintuitive behaviours emerge—such as the variance increasing can actually cause the expected loss to *decrease*. Only very recently were the necessary and sufficient conditions identified for a decomposition to hold (Heskes, 2025). *There are still many open research issues.*

3.6 Summary

This week we've examined the bias-variance decomposition, a fundamental idea in ML, useful for understanding the nature of model fitting. We discussed how there can be a trade-off between these bias and variance terms, which is one cause of the phenomenon we know as *over-fitting*. But, as we will see in future weeks, the trade-off is not *always* so simple as we have seen today.

Next week, we take extend our bias/variance analysis beyond just single models, looking at how groups of models behave when used as a "committee", more commonly known as *ensemble learning*.

The Problem with Learning a Model — Practice Exam Questions

Each week we provide you a few practice questions, representative of what may be in the final written exam. The exam will be in Canvas, so all questions match this style. Answers provided separately.

If you can answer all of **Level 1**... you're on track, but doing the bare minimum to pass.
If you can answer all of **Level 2**... you're doing well, more than enough to pass.
If you can answer all of **Level 3**... well ahead, into first class territory. I expect 20% of the class here.

Level 1

1. When I say *expected* risk, does the word "expected" refer to $\mathbb{E}_{\mathcal{S}}$ or $\mathbb{E}_{\boldsymbol{xy}}$?
2. In the bias–variance decomposition for squared loss, the _____ term represents irreducible uncertainty in the labels, even when using the Bayes model.
3. What does the subscript notation on $f_{\mathcal{S}}(\boldsymbol{x})$ emphasize?
   (a) The model minimizes empirical risk
   (b) The model is Bayes-optimal for $\boldsymbol{x}$
   (c) The model depends on the empirical risk
   (d) The model depends on the particular sampled training set
4. In the dartboard metaphor, the spread of predictions corresponds to which quantity?
   (a) Spread of predictions around the Bayes model
   (b) Model variance with respect to training data
   (c) Empirical risk variance
   (d) Noise, with respect to the Bayes model
5. True or False. The noise term in the bias–variance decomposition depends on the learning algorithm.

Level 2

6. Which quantity is minimized by the Bayes model?
   (a) Empirical risk
   (b) Population risk
   (c) Expected empirical risk
   (d) Variance
7. For squared loss, the Bayes predictor satisfies
   $$y^*(\boldsymbol{x}) = \mathbb{E}_{y|\boldsymbol{x}}[y].$$
   If $P(y = 0 \mid \boldsymbol{x}) = 0.25$ and $P(y = 2 \mid \boldsymbol{x}) = 0.75$, compute $y^*(\boldsymbol{x})$.
8. Which term in the bias–variance decomposition measures sensitivity to different training sets?
   (a) Noise
   (b) Bias
   (c) Variance
   (d) Expected risk
9. True or False. For decision trees, increasing depth typically decreases bias and increases variance.
10. True or False. The quantity $f_{\mathcal{S}}(\boldsymbol{x})$ is a random variable, but the Bayes predictor $y^*(\boldsymbol{x})$ is not.

Level 3

11. Consider two learning algorithms trained on the same task, using squared loss.

    Algorithm A always produces the same predictor $f(\boldsymbol{x})$ regardless of the training set $\mathcal{S}$.

    Algorithm B produces predictors $f_{\mathcal{S}}(\boldsymbol{x})$ that vary substantially with the sampled training set, but whose expected prediction satisfies
    $$\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})] = y^*(\boldsymbol{x}).$$

    Which of the following statements is correct?

    (a) Algorithm B has zero variance but non-zero bias.
    (b) Algorithm A has zero bias but non-zero variance.
    (c) Algorithm B has both zero bias and zero variance.
    (d) Algorithm A has zero variance but may have non-zero bias.

12. For a fixed input $\boldsymbol{x}$, suppose
    $$\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})] = 3, \qquad y^*(\boldsymbol{x}) = 1.$$

    Compute the contribution of the bias term at x under squared loss.

13. For squared loss, which statement best explains *why* variance increases for very flexible models?

    (a) The noise term depends on the function class
    (b) The learned predictor fits the training set more thoroughly, because it has more ways to fit
    (c) The expected model $\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]$ becomes noisier as model complexity increases
    (d) The Bayes risk increases with model complexity

14. True or False. The bias–variance decomposition holds for the 0/1 classification loss.

15. Suppose a learning algorithm satisfies
    $$f_{S_n}(\boldsymbol{x}) = \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})] \quad \text{for all training sets } \mathcal{S}.$$

    Which statement must be true under squared loss?

    (a) The bias term is zero
    (b) The variance term is zero
    (c) The noise term is zero
    (d) The expected risk equals the Bayes risk

Proofs
You will not be not expected to reproduce any proof in the exam.
However, answering certain questions may be easier if you understand the proof.

Proof of Theorem 3.1 (bias/variance decomposition for squared loss)

We will prove the following decomposition of the expected risk:

$$\underbrace{\mathbb{E}_{\mathcal{S}}\Big[R(f_{\mathcal{S}})\Big]}_{\text{expected risk}} = \underbrace{\mathbb{E}_{\boldsymbol{xy}}\Big[\,(y-y^*(\boldsymbol{x}))^2\,\Big]}_{\text{noise}} + \underbrace{\mathbb{E}_{\boldsymbol{xy}}\Big[(y^*(\boldsymbol{x})-\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})])^2\Big]}_{\text{bias}} + \underbrace{\mathbb{E}_{\boldsymbol{xy}}\Big[\mathbb{E}_{\mathcal{S}}\left[(f_{\mathcal{S}}(\boldsymbol{x})-\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})])^2\right]\Big]}_{\text{variance}}, \tag{38}$$

The left hand side is the expected (squared) risk of a model,

$$\mathbb{E}_{\mathcal{S}}\Big[R(f_{\mathcal{S}})\Big] = \mathbb{E}_{\mathcal{S}}\Big[\;\mathbb{E}_{\boldsymbol{xy}}\Big[\;\Big(y-f_{\mathcal{S}}(\boldsymbol{x})\Big)^2\;\Big]\;\Big]. \tag{39}$$

Notice the first term on the right, labeled 'noise', is actually the *Bayes risk*, which we saw in Definition 7.

$$R(y^*) := \mathbb{E}_{\boldsymbol{xy}}\Big[(y-\mathbb{E}_{y|\boldsymbol{x}}[y])^2\Big] \tag{40}$$

The difference between these two is called the expected **excess risk**,

$$\mathcal{E}(f) := \mathbb{E}_{\mathcal{S}}\Big[R(f_{\mathcal{S}})\Big] - R(y^*) \tag{41}$$

Now subtract/add the risk of the *expected model*, $R(\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})])$,

$$\mathcal{E}(f) \;=\; \underbrace{\mathbb{E}_{\mathcal{S}}\Big[R(f_{\mathcal{S}})\Big] - R(\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})])}_{\text{variance}} + \underbrace{R(\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]) - R(y^*)}_{\text{bias}}, \tag{42}$$

and we have the core decomposition. We will now prove each component. Take the first pair of terms:

$$\begin{aligned}
\mathbb{E}_{\mathcal{S}}\Big[R(f_{\mathcal{S}})\Big] - R(\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]) &= \mathbb{E}_{\mathcal{S}}\Big[\;\mathbb{E}_{\boldsymbol{xy}}\Big[\;\Big(y-f_{\mathcal{S}}(\boldsymbol{x})\Big)^2\;\Big]\;\Big] - \mathbb{E}_{\boldsymbol{xy}}\Big[(y-\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})])^2\Big] && (43)\\
&= \mathbb{E}_{\mathcal{S}}\Big[\;\mathbb{E}_{\boldsymbol{xy}}\Big[\;\Big(y-f_{\mathcal{S}}(\boldsymbol{x})\Big)^2 \;-\; (y-\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})])^2\Big]\Big] && (44)\\
&= \mathbb{E}_{\mathcal{S}}\Big[\;\mathbb{E}_{\boldsymbol{xy}}\Big[\;y^2 + f_{\mathcal{S}}(\boldsymbol{x})^2 - 2yf_{\mathcal{S}}(\boldsymbol{x}) \;-\; y^2 - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]^2 + 2y\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]\Big]\Big] && (45)\\
&= \mathbb{E}_{\mathcal{S}}\Big[\;\mathbb{E}_{\boldsymbol{xy}}\Big[\;f_{\mathcal{S}}(\boldsymbol{x})^2 - 2yf_{\mathcal{S}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]^2 + 2y\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]\Big]\Big] && (46)\\
&= \mathbb{E}_{\boldsymbol{xy}}\Big[\;\mathbb{E}_{\mathcal{S}}\Big[f_{\mathcal{S}}(\boldsymbol{x})^2\Big] - 2y\mathbb{E}_{\mathcal{S}}\Big[f_{\mathcal{S}}(\boldsymbol{x})\Big] - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]^2 + 2y\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]\Big] && (47)\\
&= \mathbb{E}_{\boldsymbol{xy}}\Big[\;\mathbb{E}_{\mathcal{S}}\Big[f_{\mathcal{S}}(\boldsymbol{x})^2\Big] - 2y\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})] - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]^2 + 2y\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]\Big] && (48)\\
&= \mathbb{E}_{\boldsymbol{xy}}\Big[\;\mathbb{E}_{\mathcal{S}}\Big[f_{\mathcal{S}}(\boldsymbol{x})^2\Big] - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]^2\;\Big] && (49)\\
&= \mathbb{E}_{\boldsymbol{xy}}\Big[\;\mathbb{E}_{\mathcal{S}}\Big[\Big(f_{\mathcal{S}}(\boldsymbol{x}) - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]\Big)^2\;\Big]\;\Big] && (50)\\
&= \text{variance}(f) && (51)
\end{aligned}$$

where the final steps use the identity $\mathbb{E}[Z^2] - (\mathbb{E}[Z])^2 = \mathbb{E}[(Z-\mathbb{E}[Z])^2]$, i.e. the variance of any random variable Z is equal to the *mean of the square* of Z minus the *square of the mean* of Z.

And the second pair,

$$
\begin{aligned}
R(\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]) - R(y^*) &= \mathbb{E}_{\boldsymbol{xy}}\Big[\ \Big(y - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]\Big)^2\ \Big] - \mathbb{E}_{\boldsymbol{xy}}\Big[\ (y - y^*(\boldsymbol{x}))^2\ \Big] && (52)\\
&= \mathbb{E}_{\boldsymbol{xy}}\Big[\ \Big(y - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]\Big)^2 - \Big(y - y^*(\boldsymbol{x})\Big)^2\Big] && (53)\\
&= \mathbb{E}_{\boldsymbol{xy}}\Big[\ y^2 + \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]^2 - 2y\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})] - y^2 - (y^*(\boldsymbol{x}))^2 + 2yy^*(\boldsymbol{x})\Big] && (54)\\
&= \mathbb{E}_{\boldsymbol{x}}\Big[\ \mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}\Big[\ \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]^2 - 2y\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})] - (y^*(\boldsymbol{x}))^2 + 2yy^*(\boldsymbol{x})\Big]\Big] && (55)\\
&= \mathbb{E}_{\boldsymbol{x}}\Big[\ \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]^2 - 2y^*(\boldsymbol{x})\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})] - (y^*(\boldsymbol{x}))^2 + 2(y^*(\boldsymbol{x}))^2\Big] && (56)\\
&= \mathbb{E}_{\boldsymbol{x}}\Big[\ \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]^2 - 2y^*(\boldsymbol{x})\mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})] + (y^*(\boldsymbol{x}))^2\Big] && (57)\\
&= \mathbb{E}_{\boldsymbol{x}}\Big[\ \Big(y^*(\boldsymbol{x}) - \mathbb{E}_{\mathcal{S}}[f_{\mathcal{S}}(\boldsymbol{x})]\Big)^2\Big] && (58)\\
&= \mathrm{bias}(f) && (59)
\end{aligned}
$$

where we have used that $\mathbb{E}_{\boldsymbol{xy}}[...] = \mathbb{E}_{\boldsymbol{x}}[\mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}[...]]$, and we recall $y^* = \mathbb{E}_{\boldsymbol{y}|\boldsymbol{x}}[y]$.

Now by simple rearrangement, we have proved the theorem. □

4 Combining Multiple Models

Outline: Sometimes it's really hard to train a model to the level of performance you need. **Too hard**. Perhaps we could just accept that any *single* model will have limitations, and instead learn *multiple* models, each of which has a different perspective on the problem. For any new testing data, we *combine* their predictions, like a committee. This is our topic for the week—both a simple practical algorithm, and the underlying theory. We will see how all of this is fundamentally tied to the bias/variance decomposition that we met last week.

4.1 A simple ensemble learning algorithm

In the field of ML, learning a "committee" of models is commonly referred to as 'ensemble learning'. We will begin with a simple and very well-known "ensemble learning" algorithm, then cover the underlying theory. The overall principle of *ensemble learning* is that the algorithm takes the data, and produces not just one, but multiple models.

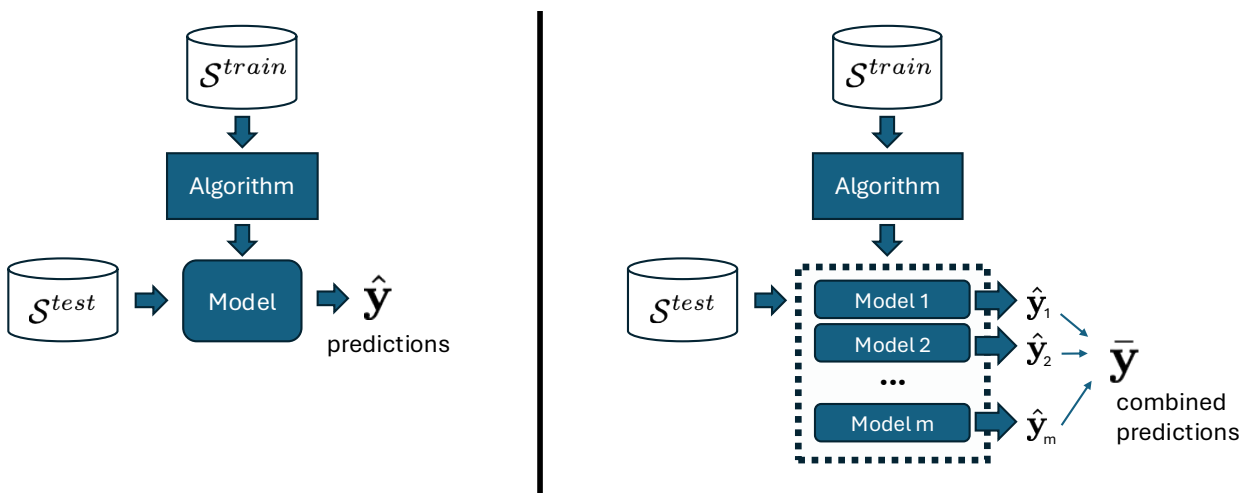


Figure 11: Learning a single model (LEFT), versus learning an ensemble of models (RIGHT).

The motivation for using an ensemble is that it might "average out" the mistakes of individuals, allowing some models to compensate for others. If we want this, then it's obvious that we don't want all identical predictions; and, it's equally obvious we don't want them to be different just for the sake of it, ignoring the overall system performance. **We need something in between the these two extremes.**

So, how can we encourage a little bit of difference between our models? Last week we covered how our models can be *sensitive* to their training data, resulting in model *variance*. Maybe we can 'operationalise' this theory? Yes, we can. This is the 'Bagging' algorithm (Breiman, 1996), which is a contraction of **B**ootstrap **agg**regat**ing**. So what's a 'bootstrap'?

Definition 10 (Bootstrap sample)**.** *Given a set of n examples, $\mathcal{S} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$, a bootstrap is a new set $\mathcal{S}'$, also of size n, where each $(\boldsymbol{x}_i, y_i)$ is sampled uniformly* with replacement *from the original $\mathcal{S}$.*

In practice, if I had $n = 1, \ldots, 100$, I need to generate 100 integers uniformly from $1 - 100$, allowing repeats. Sampling with replacement makes the bootstrap sample behave like a fresh draw from the unknown data-generating distribution, approximated by the empirical distribution that we have in the data. With this, we can now define the "Bagging" algorithm.

Algorithm 1 Bagging / Bootstrap Aggregating, Breiman (1996)

Require: Training set $\mathcal{S} = \{(x_i, y_i)\}_{i=1}^n$, a learning algorithm $\mathcal{A}$, number of bootstrap samples m

1: **for** $j = 1$ to m **do**
2: Draw a bootstrap sample $\mathcal{S}^j$ from $\mathcal{S}$ (i.e. sampling with replacement)
3: Train a model:

$$f_j \leftarrow \mathcal{A}(\mathcal{S}^j)$$

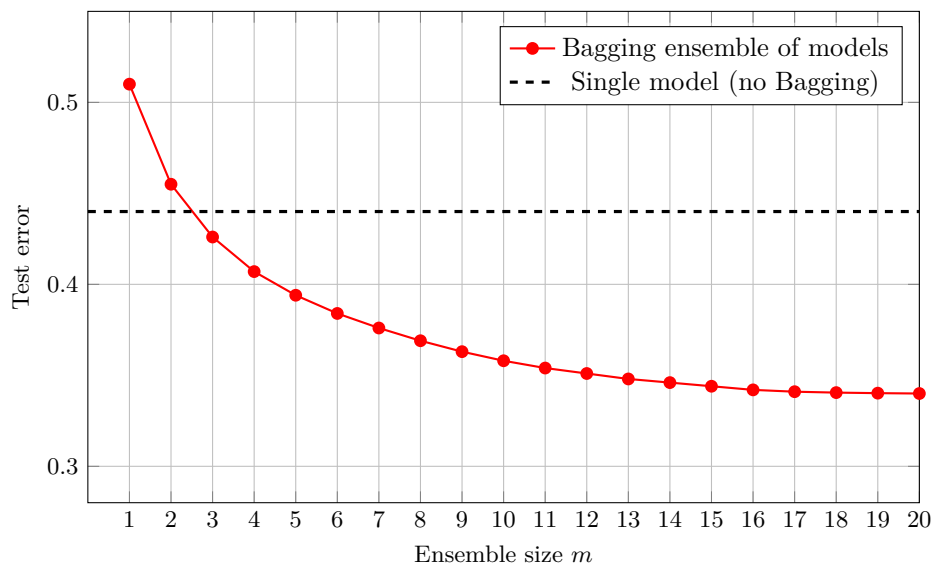
4: **end for**
5: Define the aggregated predictor:

$$f_{\text{bag}}(\boldsymbol{x}) = \frac{1}{m} \sum_{j=1}^{m} f_j(\boldsymbol{x}).$$

6: **return** f_{bag}

In summary, we learn a committee of models, where each one is trained from a *new* i.i.d. sample $\mathcal{S}'$ generated from the original $\mathcal{S}$. In practice this means that the models will be *slightly different* from one another, but still learn the underlying distribution of the data.

Below are the results of a simple experimental test, building an increasingly large ensemble of decision trees. We compare the Bagged ensemble to a single model, which does not use the Bagging algorithm at all.



You can see that our performance is just getting better and better, with larger ensembles. We're of course not getting this performance increase for free—every additional ensemble member is more computational effort expended—and, it's slowing down, getting to an asymptote where there are diminishing returns. But, given the simplicity of this technique, there's a relatively large payoff in terms of performance.

What do you notice about the performance of the single model versus the ensemble? For small m, in this case, $m = \{1, 2\}$, the ensemble is actually *worse* than the single model. Can you figure out why?

4.2 Ensembles can guarantee a reduction in loss

In the previous section we took a little detour from the usual course mission—we examined a practical algorithm. We will now switch back to the theory, and try to understand *why such an algorithm can succeed at all.* An important result here is the *Ambiguity Decomposition* (Krogh et al., 1995).

Theorem 4.1 (**Ambiguity decomposition**)**.** *For a set of m trained models $\{f_i\}_{i=1}^m$, define the ensemble combiner as the arithmetic mean of the individual models, $\bar{f}(\boldsymbol{x}) = \frac{1}{m}\sum_{i=1}^m f_i(\boldsymbol{x})$. For a new test point $(\boldsymbol{x}, y)$, the loss of the ensemble breaks into two terms: the average loss, and the "ambiguity" of the ensemble.*

$$\underbrace{\left(\bar{f}(\boldsymbol{x}) - y\right)^2}_{ensemble\ loss} = \underbrace{\frac{1}{m}\sum_{i=1}^m \left(f_i(\boldsymbol{x}) - y\right)^2}_{average\ loss} - \underbrace{\frac{1}{m}\sum_{i=1}^m \left(f_i(\boldsymbol{x}) - \bar{f}(\boldsymbol{x})\right)^2}_{ambiguity}. \tag{60}$$

Notice the negative sign on the ambiguity term. The ambiguity term quantifies how much the individual models *disagree* on the given test point $\boldsymbol{x}$. Since this ambiguity is ≥ 0, the result implies that the ensemble loss is *guaranteed* to be less than or equal to the average loss of the m models. This is a powerful observation.

> For squared loss, combining models as an arithmetic mean, $\bar{f}(\boldsymbol{x}) = \frac{1}{m}\sum_{i=1}^m f_i(\boldsymbol{x})$, guarantees the ensemble loss will be less than or equal to the average loss of the m models.

Note that this is a statement about the average loss of the models, i.e. does not mean that the ensemble will be better than any *single model within the ensemble.* The idea of the ensemble guaranteeing a loss reduction holds for more than just squared loss. For binary cross-entropy, the result is as follows.

Theorem 4.2 (Ambiguity decomposition for binary cross-entropy)**.** *Given a label $y \in \{0, 1\}$, a set of models $\{f_i\}_{i=1}^m$, and $\ell_{ce}(y, f(\boldsymbol{x})) = -\Big(y \ln f(\boldsymbol{x}) + (1-y)\ln(1 - f(\boldsymbol{x}))\Big)$ then the following holds.*

$$\underbrace{\ell_{ce}\Big(y,\ \mathring{f}(\boldsymbol{x})\Big)}_{ensemble\ loss} = \underbrace{\frac{1}{m}\sum_{i=1}^m \ell_{ce}\Big(y,\ f_i(\boldsymbol{x})\Big)}_{average\ loss} - \underbrace{\frac{1}{m}\sum_{i=1}^m D_{KL}\Big(\mathring{f}(\boldsymbol{x}),\ f_i(\boldsymbol{x})\Big)}_{ambiguity}. \tag{61}$$

where D_{KL} is the KL-divergence, and $\mathring{f}(\boldsymbol{x}) := Z^{-1}\prod_{i=1}^m f_i(\boldsymbol{x})^{1/m}$ is the normalized geometric mean, i.e. Z normalizes to ensure $\mathring{f}(\boldsymbol{x})$ is a valid probability distribution.

Notice the ambiguity here is now the KL divergence between the distributions, that we met in week 2. As I said then, we won't dwell further on it, but it really does pop up all over Machine Learning. The important thing here is that its non-negative, so we again have a guarantee of reducing the ensemble loss.

> For cross-entropy loss, combining models as a geometric mean, $\mathring{f}(\boldsymbol{x}) := Z^{-1}\prod_{i=1}^m f_i(\boldsymbol{x})^{1/m}$ guarantees the ensemble loss will be less than or equal to the average loss of the m models.

For a direct application of this theory: with a regression problem, we use Bagging and take the arithmetic mean, to guarantee a performance boost. For classification, we take the normalized geometric mean of the probability estimates, and get a similar guarantee of reduction in the cross-entropy loss.

These Ambiguity decompositions are neat and nice to have. But, notice that they are statements about *pre-trained* models. We have not talked *at all* about actually learning the set of models. Does over-fitting still happen here? To address this, we will return to the bias-variance decomposition from last week, a a useful conceptual tool to understand model fitting, including ensembles as a kind of model in their own right. It is important to be clear about the perspective we adopt from this point onward. An ensemble is not merely a collection of trained models combined at prediction time. Rather, the choice to train multiple models and aggregate their predictions defines a new type of 'super' model in its own right.

4.3 Revisiting bias/variance in the ensemble setting

The bias-variance decomposition (section 3) told us that there are several components hidden within the expected risk of a model: the noise, the bias, and the variance. Some standard results from classical statistics can shed some light on the **variance** term. We will treat models as random variables, where the randomness arises from the training set they are provided with.

Reminder: linear combinations of random variables. Let $X_1, \ldots, X_m$ be random variables, and let $a_1, \ldots, a_m \in \mathbb{R}$ be constants. We consider a linear combination $\bar{X} = \sum_{j=1}^{m} a_j X_j$.

Variance of a linear combination. If the random variables $X_1, \ldots, X_m$ are uncorrelated, then

$$\mathrm{Var}\left(\sum_{j=1}^{m} a_j X_j\right) = \sum_{j=1}^{m} a_j^2 \,\mathrm{Var}(X_j).$$

In particular, for the simple arithmetic mean, $\bar{X} = \frac{1}{m}\sum_{j=1}^{m} X_j$, we have

$$\mathrm{Var}(\bar{X}) = \frac{1}{m^2}\sum_{j=1}^{m} \mathrm{Var}(X_j).$$

We now link this to our ensemble learning scenario.

Fix an input $\boldsymbol{x}$. For an ensemble of m models, we treat each prediction $f_j(\boldsymbol{x})$ as a random variable, where the randomness arises from the training set $\mathcal{S}$. The ensemble is, as before, the arithmetic mean, $\bar{f}(\boldsymbol{x}) = \frac{1}{m}\sum_{j=1}^{m} f_j(\boldsymbol{x})$. We now simply apply the result above.

Variance of the ensemble. If the models (random variables) $f_1(\boldsymbol{x}), \ldots, f_m(\boldsymbol{x})$ are uncorrelated, then

$$\mathrm{Var}\left(\bar{f}(\boldsymbol{x})\right) = \frac{1}{m^2}\sum_{j=1}^{m} \mathrm{Var}\left(f_j(\boldsymbol{x})\right).$$

In particular, if we assume all the individual models have the same variance, taking an ensemble as a simple arithmetic mean reduces this by a factor proportional to $1/m$.

For an arithmetic mean ensemble, assessed by squared loss, if the ensemble members are uncorrelated, the variance of the ensemble is $\frac{1}{m}$ times the average variance of the individuals.

In practice, they're always correlated, and variance reduction depends on how correlated they are.

This is a very nice result to have, and it is intuitive that the 'disagreement' (quantified as correlation) determines the amount of reduction in variance. But, this says nothing about bias. How is that affected?

To understand things more generally, we will consider an extension of the bias-variance decomposition.

Bias, Variance, and Diversity

Recall that with a single model, we get a decomposition of the expected risk into 3 components—noise, bias, and variance. It turns out that when we have an ensemble, we just end up with an extra component that accounts for the expected level of disagreement between ensemble members.

Theorem 4.3 (**Bias-Variance-Diversity decomposition for squared loss**, Wood et al. (2023))**.** *For a set of models* $\{f_i\}_{i=1}^m$*, combined as an arithmetic mean* $\bar{f}(\boldsymbol{x}) = \frac{1}{m}\sum_{i=1}^m f_i(\boldsymbol{x})$*. Denote the following terms:*
The noise/Bayes risk: $\textbf{noise} = \mathbb{E}_{\boldsymbol{xy}}\Big[\,(y - y^*(\boldsymbol{x}))^2\,\Big]$*;*
the bias of the ith model: $\textbf{bias(i)} = \mathbb{E}_{\boldsymbol{x}}\Big[(y^* - \mathbb{E}_{\mathcal{S}}[f_i(\boldsymbol{x})])^2\Big]$*;*
the variance of the ith model: $\textbf{var(i)} = \mathbb{E}_{\boldsymbol{x}}\Big[\mathbb{E}_{\mathcal{S}}\left[(f_i(\boldsymbol{x}) - \mathbb{E}_{\mathcal{S}}[f_i(\boldsymbol{x})])^2\right]\Big]$*;*
the diversity of the set: $\textbf{diversity} = \mathbb{E}_{\boldsymbol{x}}\Big[\mathbb{E}_{\mathcal{S}}\left[\frac{1}{m}\sum_{i=1}^m (f_i(\boldsymbol{x}) - \bar{f}(\boldsymbol{x}))^2\right]\Big]$*.*

Then the expected risk of the ensemble admits the following decomposition:

$$\mathbb{E}_{\mathcal{S}}\Bigg[\mathbb{E}_{\boldsymbol{xy}}\Big[(\bar{f}(\boldsymbol{x}) - y)^2\Big]\Bigg] \;=\; \text{noise} \;+\; \frac{1}{m}\sum_{i=1}^m \text{bias(i)} \;+\; \frac{1}{m}\sum_{i=1}^m \text{var(i)} \;-\; \text{diversity} \tag{62}$$

This breaks expected loss of the ensemble into **four** terms. The first is the Bayes risk (noise), which is exactly as we saw in the original single model case. After this, we now have the *average* bias of the individual models, plus the *average* variance, **minus** the "diversity" of the ensemble. The critical thing here are the signs—whilst we wish to *reduce* both bias and variance, we want to *increase* diversity. It's still a trade-off, but instead of the *two-way* trade-off with single models, we have a *three-way* trade-off, with the diversity as the extra 'dimension' to manage. Note that we usually ignore the noise term when discussing trade-offs, as it is a constant that cannot be affected by any learning algorithm.

With single models, we have a two-way **bias/variance** trade-off.
With ensembles, we have a three-way **bias/variance/diversity** trade-off.

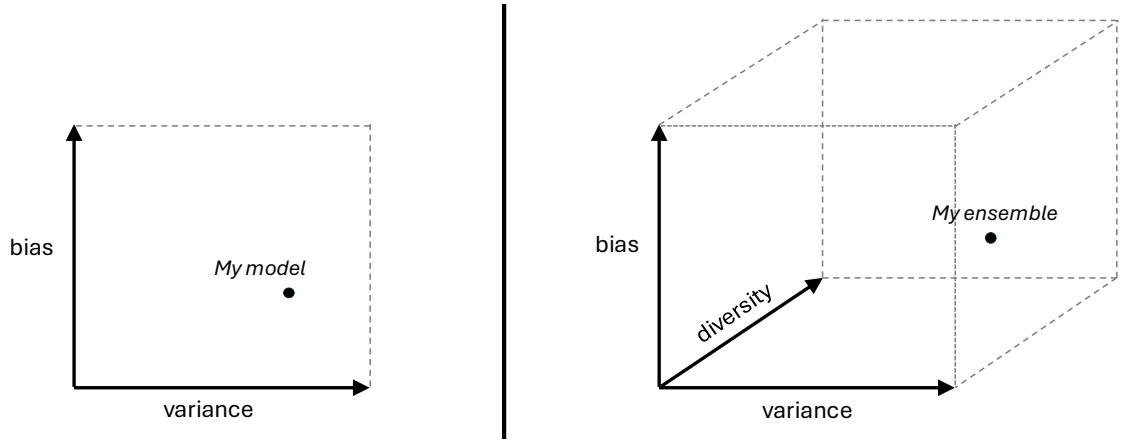


Figure 12: The bias-variance trade-off for a single model (LEFT) versus the bias/variance/diversity trade-off for an ensemble (RIGHT). Note that this diagram assumes all the models are identical in structure, apart from the training data provided. In this case, the bias is the same for all models (as is variance), and the only factor that determines performance is the ensemble diversity.

And, this does apply beyond just squared loss. In subsection 3.5, we briefly mentioned the idea of bias-variance for cross-entropy. Accordingly, there is a bias-variance-diversity decomposition, for the cross-entropy (Wood et al., 2023), and many other losses. We won't go into the maths, but just consider the intuition—how do these three terms balance against each other in practice?

The Bias/Variance/Diversity Trade-off

Let's revisit our Bagging ensemble from earlier today. We learnt an ensemble of regression trees, varying the number of members in the ensemble. This time however, we will break the expected risk into the three components: bias, variance, and diversity, to see what is happening to them. In this way, they act as a kind of 'diagnostic probe', telling us where the performance gain is coming from.

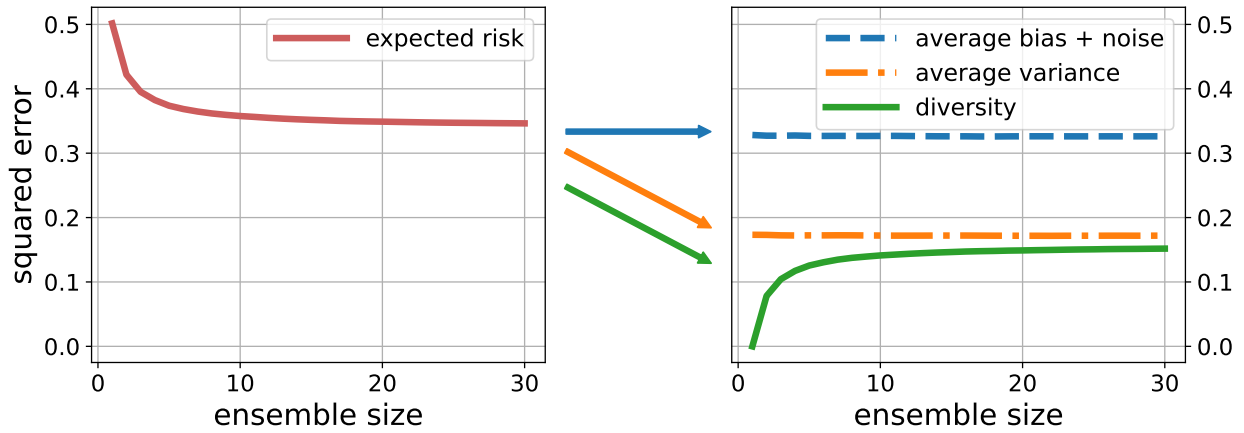


Figure 13: Ensembles of regression trees (each depth 8) of increasing size.

Whilst expected risk decreases, the average bias and average variance are constant. This is as we might anticipate, since the individual capacities are constant, it is only the ensemble size, m, that we change. In contrast, diversity increases with m—subtracting from the expected risk—and the improvement is determined entirely by diversity. This is not so if we vary something other than m. Below we fix $m = 10$, and instead vary the tree depth, i.e. making the *individual* models more complex.

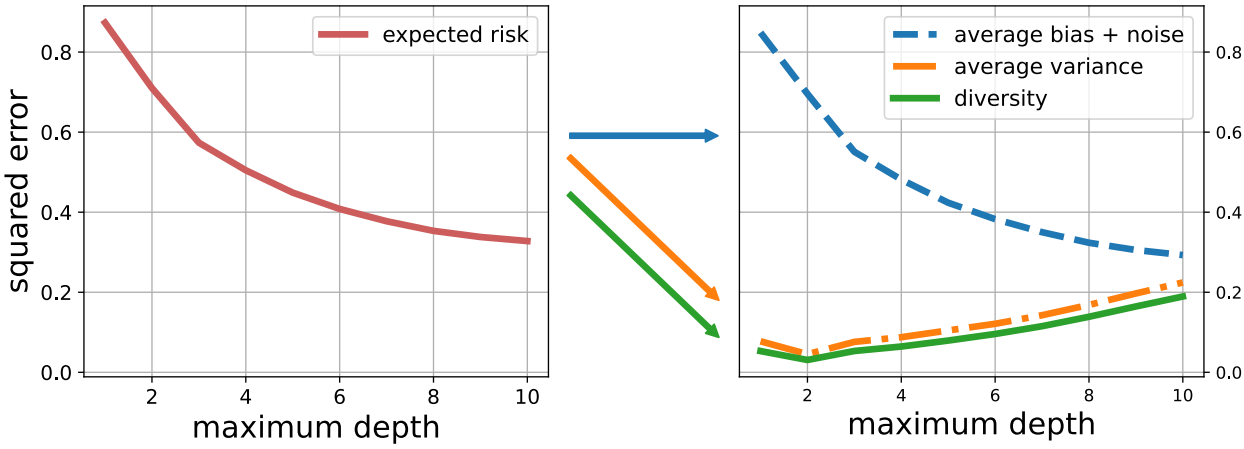


Figure 14: Ensembles of $m = 10$ regression trees, varying their individual complexity.

Now, all three components change. Overall, we have a bias-variance-diversity trade-off to manage—defined by the complexity and number of the individual, and how much they disagree with each other.

4.4 Theory for Majority Voting ensembles

In the previous sections we saw how combining multiple predictions can reduce loss. These analyses were phrased in terms of *real-valued* predictions, where it is natural to average the model predictions. However, many problems of interest do not operate with real-valued predictions, instead requiring a discrete class label, e.g. predicting today's weather from the label set $\{rain, snow, wind\}$. A common way to combine predictions in this setting is *majority voting*. Each model produces a class label prediction, and the ensemble returns the label that gets the most votes. This rule is simple, intuitive, and widely used in practice.

In this section we study voting from a probabilistic perspective. Rather than decomposing risk into components, we ask a different question: if we have many classifiers that are each slightly better than random guessing, when does the majority vote outperform an individual classifier?

Imagine a single model has probability ϵ of making a mistake, then among a group of m voters, the probability of *exactly* k making errors is given by an application of the binomial theorem:

$$P(\text{exactly } k \text{ errors from } m) = \binom{m}{k}\epsilon^k(1-\epsilon)^{(m-k)}. \tag{63}$$

We can then sum this probability for k greater than half-plus-one of the voters, in order to get the probability of error in a majority vote.

$$P(majority\ vote\ error) = \sum_{k \geq \lceil \frac{m+1}{2} \rceil} \binom{m}{k}\epsilon^k(1-\epsilon)^{(m-k)}. \tag{64}$$

This is illustrated in the figures below.

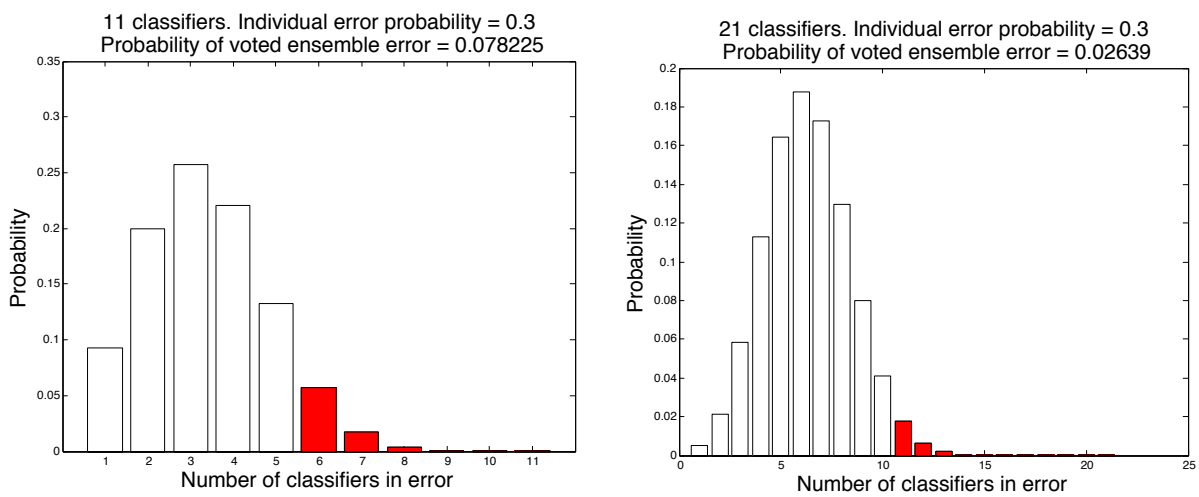


Figure 15: Probability of error for voting ensembles of different sizes (left: 11 classifiers, or right: 21.)

The shaded red area is the probability that the voting ensemble makes an error. Notice in the left figure (with 11 voters) the area is quite big, with a 7.8% chance of ensemble error. In the right figure, (with 21 voters) it is smaller, at about 2.6% chance of error. Yet in both, the chances of an error for an *individual* voter is the same, $\epsilon = 0.3$. The voters are the same, but the ensemble increases in performance because there are more independent voters.

We can illustrate this idea further, and plot the voting error as the number of voters increases. Figure 16 shows this, for different values of ϵ. Note that the error approaches zero as the number of voters increases.

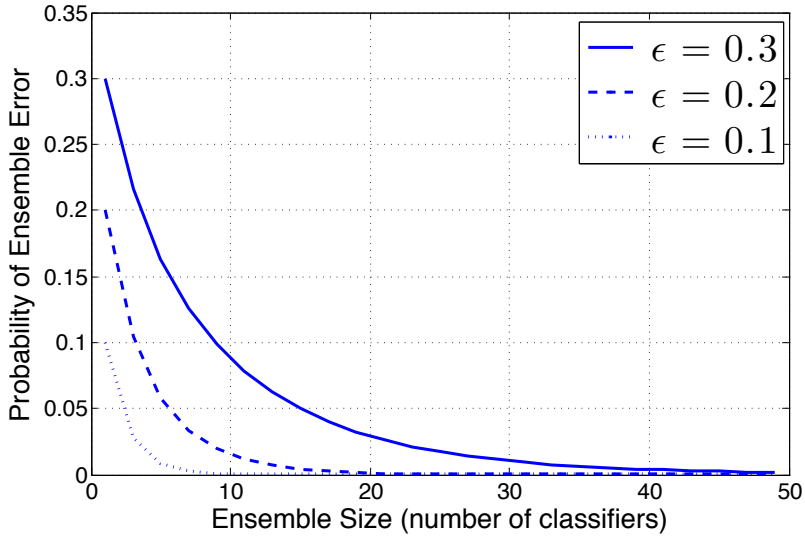


Figure 16: Theoretical majority voting error as we increase the ensemble size (number of voters), *assuming errors are statistically independent.* If each model has only 20% error rate ($\epsilon = 0.2$), the ensemble will have $< 1\%$ error if we combine 15 of them, or $< 0.1\%$ if we combine 20 of them.

Seems like magic, right? But, remember the critical assumption... that the voters make *statistically independent* errors. There do exist techniques to put an *upper bound* on the number of probability of ensemble error, given the degree of independence, but that goes beyond the scope of this course.

4.5 Summary

The *ensemble learning* methodology is *very* popular in ML. It gets you that last bit of possible performance, with relatively little effort—it is the "secret sauce" behind most competition wins, and applications in industry are extremely common, even though the flashier techniques tend to get the spotlight.

Combining multiple models is not just a heuristic trick, but a principled way of improving performance. By averaging or voting across predictors, ensembles can reduce loss in a way that is guaranteed under very general conditions.

If you recall the start of this course, we named the model choice, the algorithm/loss, and the dataset, as the "essential elements" of any ML system. Next week we formalise this broad statement with theory, and look at how each of these affects performance.

Combining Multiple Models — Practice Exam Questions

Each week we provide you a few practice questions, representative of what may be in the final written exam. The exam will be in Canvas, so all questions match this style. Answers provided separately.

If you can answer all of **Level 1**... you're on track, but doing the bare minimum to pass.
If you can answer all of **Level 2**... you're doing well, more than enough to pass.
If you can answer all of **Level 3**... well ahead, into first class territory. I expect 20% of the class here.

Level 1

1. My ensemble prediction is defined using $\bar{f}(\boldsymbol{x}) = \frac{1}{m}\sum_{j=1}^{m} f_j(\boldsymbol{x})$. Which operation is being applied to the individual model outputs?

   (a) Thresholding
   (b) Majority mean voting
   (c) Arithmetic mean
   (d) Normalized geometric mean.

2. Match each term to the most appropriate description.

Term	**Description**
1. Ensemble	A. Sensitivity to the particular training set
2. Variance	B. Combines real-valued predictions
3. Majority voting	C. A collection of predictors used together
4. Averaging	D. Combines discrete predictions

3. Suppose all models in an ensemble always return exactly the same prediction for every input $\boldsymbol{x}$. Which statement best describes the effect of ensembling in this case?

   (a) Bias is reduced but variance is unchanged
   (b) Variance is reduced but bias is unchanged
   (c) Neither bias nor variance is reduced
   (d) Both bias and variance are reduced

4. Consider the Bagging algorithm with ensemble size m. Which of the following is critical to induce diversity between the individual models?

   (a) Using the same learning algorithm for each model
   (b) Sampling the testing set with replacement
   (c) Averaging the final predictions
   (d) Sampling the training set with replacement.

5. Which of the following statements best describes the goal of ensemble learning?

   (a) To reduce bias by increasing model complexity
   (b) To reduce variance by combining multiple predictors
   (c) To minimise popluation risk directly
   (d) To approximate the Bayes model exactly

Level 2

6. Which term in the bias–variance decomposition is reduced by averaging independent predictors?
   (a) Noise
   (b) Bias
   (c) Variance
   (d) Bayes risk

7. Fill in the blank. If each predictor $f_j(\boldsymbol{x})$ has variance σ^2 and the predictors are independent, then

$$\mathrm{Var}\left(\frac{1}{m}\sum_{j=1}^{m} f_j(\boldsymbol{x})\right) = \underline{\qquad\qquad\qquad}.$$

8. Which assumption is essential for the variance reduction result in averaging ensembles?
   (a) The predictors must be unbiased
   (b) The predictors must have identical variance
   (c) The predictors must be weakly correlated
   (d) The loss function must be squared loss

9. Let $f_1(\boldsymbol{x}), \ldots, f_m(\boldsymbol{x})$ be predictors, and define the ensemble

$$\bar{f}(\boldsymbol{x}) = \frac{1}{m}\sum_{j=1}^{m} f_j(\boldsymbol{x}).$$

   Suppose the loss is squared loss. Which statement is true?
   (a) $R(\bar{f}) < \min_j R(f_j)$
   (b) $R(\bar{f}) < \frac{1}{m}\sum_{j=1}^{m} R(f_j)$
   (c) $R(\bar{f}) \leq \frac{1}{m}\sum_{j=1}^{m} R(f_j)$
   (d) $R(\bar{f}) \geq \max_j R(f_j)$

10. Suppose all predictors in an ensemble satisfy

$$f_j(\boldsymbol{x}) = f_1(\boldsymbol{x}) \quad \text{for all } j, \text{for all } \boldsymbol{x}.$$

   Which statement is correct?
   (a) Averaging reduces variance but not bias
   (b) Averaging reduces bias but not variance
   (c) Averaging reduces neither bias nor variance
   (d) Averaging reduces both bias and variance

Level 3

11. Each classifier in a binary classification ensemble predicts the correct label independently with probability $p > \frac{1}{2}$. Let m be odd. Which expression gives the probability that the majority vote classifier is correct?

    (a) p^m
    (b) $\sum_{k=0}^{(m-1)/2} \binom{m}{k} p^k (1-p)^{m-k}$
    (c) $\sum_{k=(m+1)/2}^{m} \binom{m}{k} p^k (1-p)^{m-k}$
    (d) $1-(1-p)^m$

12. Which assumption is most critical for the theoretical guarantees of majority voting?

    (a) Low bias of individual classifiers
    (b) Statistical independence of classifier errors
    (c) Use of squared loss
    (d) Large training set size

13. Suppose each classifier in a voting ensemble has error probability $\epsilon = 0.5$. What is the error probability of the majority vote classifier?

    (a) Less than 0.5
    (b) Equal to 0.5
    (c) Greater than 0.5
    (d) It depends on the number of classifiers

14. Fill in the blank. Majority voting can improve classification performance even when individual classifiers are weak, provided that their errors are sufficiently _____.

15. Which statement best contrasts averaging ensembles and voting ensembles?

    (a) Both rely on bias reduction
    (b) Voting reduces variance, averaging reduces bias
    (c) Both rely on minimising empirical risk
    (d) Averaging reduces variance, voting exploits disagreement

Proofs
You will not be not expected to reproduce any proof in the exam.
However, answering certain questions may be easier if you understand the proof.

Proof of Theorem 4.3, the Bias/Variance/Diversity decomposition. Take the expected loss of the ensemble model $\bar{f}$ at a single point $(\boldsymbol{x}, y)$, and apply the Ambiguity decomposition, Theorem 4.1.

$$\mathbb{E}_{\mathcal{S}}\left[(\bar{f}(\boldsymbol{x}) - y)^2\right] = \mathbb{E}_{\mathcal{S}}\left[\frac{1}{m}\sum_{i=1}^{m}(f_i(\boldsymbol{x}) - y)^2 - \frac{1}{m}\sum_{i=1}^{m}(f_i(\boldsymbol{x}) - \bar{f}(\boldsymbol{x}))^2\right] \tag{65}$$

$$= \frac{1}{m}\sum_{i=1}^{m}\mathbb{E}_{\mathcal{S}}\left[(f_i(\boldsymbol{x}) - y)^2\right] - \mathbb{E}_{\mathcal{S}}\left[\frac{1}{m}\sum_{i=1}^{m}(f_i(\boldsymbol{x}) - \bar{f}(\boldsymbol{x}))^2\right] \tag{66}$$

Take the first term on the right hand side, and apply the bias-variance decomposition, Theorem 3.1. In this case, the noise is zero, since we have a single test point x. Doing this...

$$\mathbb{E}_{\mathcal{S}}\left[(\bar{f}(\boldsymbol{x}) - y)^2\right] = \frac{1}{m}\sum_{i=1}^{m}\mathbb{E}_{\mathcal{S}}\left[(f_i(\boldsymbol{x}) - y)^2\right] - \mathbb{E}_{\mathcal{S}}\left[\frac{1}{m}\sum_{i=1}^{m}(f_i(\boldsymbol{x}) - \bar{f}(\boldsymbol{x}))^2\right] \tag{67}$$

$$= \frac{1}{m}\sum_{i=1}^{m}\left[(\mathbb{E}_{\mathcal{S}}[f_i(\boldsymbol{x})] - y)^2 + \mathbb{E}_{\mathcal{S}}\left[(f_i(\boldsymbol{x}) - \mathbb{E}_{\mathcal{S}}[f_i(\boldsymbol{x})])^2\right]\right] - \mathbb{E}_{\mathcal{S}}\left[\frac{1}{m}\sum_{i=1}^{m}(f_i(\boldsymbol{x}) - \bar{f}(\boldsymbol{x}))^2\right]$$

$$= \underbrace{\frac{1}{m}\sum_{i=1}^{m}(\mathbb{E}_{\mathcal{S}}[f_i(\boldsymbol{x})] - y)^2}_{average\ bias} + \underbrace{\frac{1}{m}\sum_{i=1}^{m}\mathbb{E}_{\mathcal{S}}\left[(f_i(\boldsymbol{x}) - \mathbb{E}_{\mathcal{S}}[f_i(\boldsymbol{x})])^2\right]}_{average\ variance} - \underbrace{\mathbb{E}_{\mathcal{S}}\left[\frac{1}{m}\sum_{i=1}^{m}(f_i(\boldsymbol{x}) - \bar{f}(\boldsymbol{x}))^2\right]}_{diversity}$$

Taking the expectation over $p(\boldsymbol{x}, y)$ yields the final result, introducing the noise term.

5 Understanding the Sources of Risk

Outline: Can you explain why an **optimal** learning algorithm, given **infinite** computational budget, can still have a non-zero risk ... and where each part of that risk comes from? By the end of this session, you will be able to. Today is more conceptual than mathematical, but will hopefully give you a new way to think about Machine Learning systems: both when they succeed, and when they fail.

5.1 What causes risk?

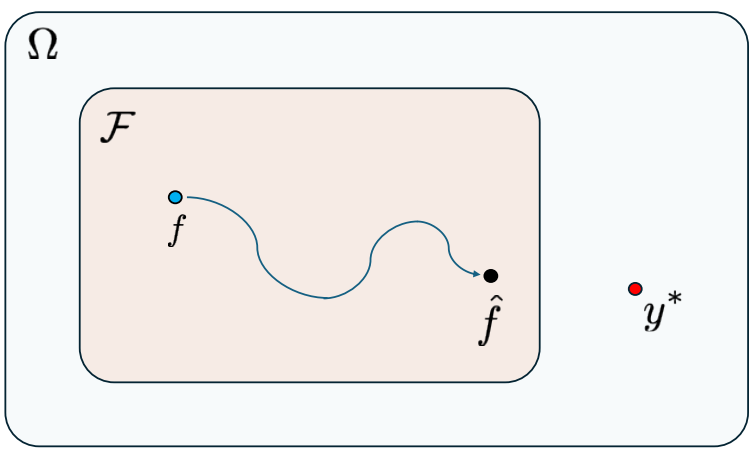
We've established that *population risk* is our ultimate judge of performance—even though we cannot measure it directly, and only estimate it via *empirical risk.* We also know there are a few critical things that determine our risk: the sample of **data**, the choice of **model**, and the choice of **learning algorithm**.

$$\left.\begin{array}{l}\text{A small or unrepresentative data sample}\\ \text{A model family not suited to the problem}\\ \text{A poor learning algorithm / limited compute}\end{array}\right\}\ \text{...any of these will probably lead to higher risk}$$

Can we isolate *how much* of the risk is due to each of these factors? How much is due to the bad sample of data? How much to the algorithm? We will pursue something like this:

$$\text{overall risk} = \big(\text{algorithm limitations}\big) + \big(\text{data limitations}\big) + \big(\text{model limitations}\big)$$

To understand this we have to learn a few new terms, starting with the idea of a *function class.* Do you remember this diagram from week 1?



Please note that this image is more *conceptual* and *illustrative* than a precise characterization.
For example, the precise straight line distance between $\hat{f}$ and y^* in the image doesn't mean anything.

The outer box, marked Ω, is the space of *all possible predictors*, of any form. Mathematically, this is the set of *all measurable functions* $f : \mathcal{X} \to \mathcal{Y}$, but you don't need to know what that phrasing means. Any predictor that you can write some code for, is a *measurable* function.

The inner box, marked $\mathcal{F}$, is the restricted family of functions, that we have access to once we choose a model family. This is technically called our **function class**. For example, if we choose the model family to be neural nets of a fixed size/architecture, then $\mathcal{F}$ is the space of functions available via setting the weights. By choosing the model family, we choose the size and location of $\mathcal{F}$ within Ω.

The point marked $\hat{f}$ is the model obtained via our learning algorithm, on a finite dataset. For completeness, we'll give it a formal definition, below.

Definition 11 (Trained model, $\hat{f}$)**.** *Given a function class $\mathcal{F}$, a sample of data $\mathcal{S}$, and a learning algorithm $\mathcal{A} : \mathcal{F} \times \mathcal{S} \to \mathcal{F}$, the trained model is defined:*

$$\hat{f} \leftarrow \mathcal{A}(f; \mathcal{S}) \tag{68}$$

Our trained model $\hat{f}$ is the result of using a limited learning algorithm, with a limited dataset, with a limited function class. Basically, this is real life. This is what an ML engineer has to cope with.

Now, let's make an assumption. Let's say, somehow, that we have the perfect learning algorithm with infinite compute budget. If we had that, we could find a model that minimizes the (empirical) risk on our limited training set. We call this type of model an *empirical risk minimizer*.

Definition 12 (Empirical risk minimizer, f_{erm})**.** *Given a function class $\mathcal{F}$, and a risk measure $\hat{R}(f; \mathcal{S})$, an empirical risk minimizer (ERM) model is defined:*

$$f_{erm} \coloneqq \arg\min_{f \in \mathcal{F}} \hat{R}(f; \mathcal{S}) \tag{69}$$

This is the model that gets the minimum possible risk on the training data. Note that, it's entirely possible that there will be **multiple** models that achieve minimal training error, i.e. a model f_{erm} is not unique.

An ERM is a model that achieves minimum possible empirical risk, but it is not necessarily unique. There can be many models that get minimal empirical risk, which look equivalent with finite data.

What if we now assume we have **an infinite amount of data**? If we had this, we could find the model that minimizes *population risk.* This is called the 'best-in-class' or 'best-in-family' model:

Definition 13 (Best-in-class model, f^*)**.** *Given a function class $\mathcal{F}$, and a population risk measure $R(f)$, the best-in-class model is defined as that $\in \mathcal{F}$ which minimizes population risk,*

$$f^* \coloneqq \arg\min_{f \in \mathcal{F}} R(f) \tag{70}$$

The best-in-class (or best-in-family) is the best model within our chosen function class. This is the one we would find, if we had infinite data and a perfect learning algorithm.

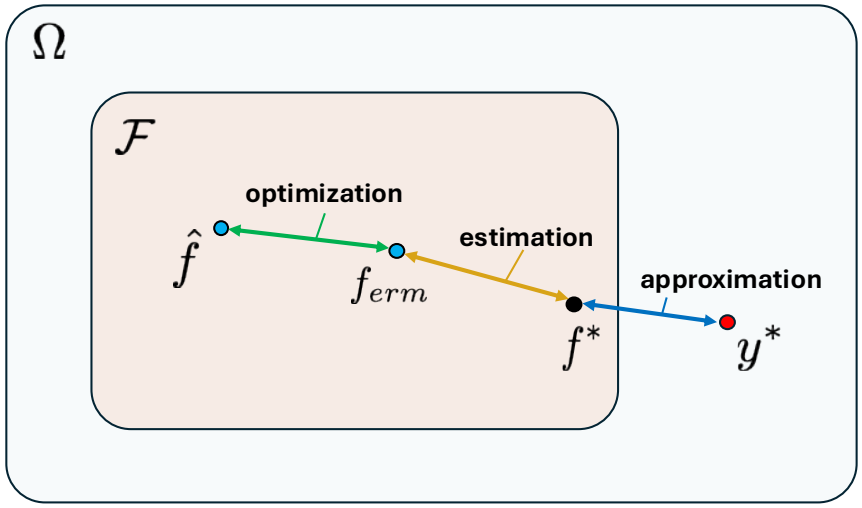
Finally, let's give ourselves **infinite model capacity**. We will say we no longer have a limited function class, but can minimize over all Ω. This would give us y^***, the Bayes Model**, the hypothetical predictor (from all possible predictors Ω) that gets the lowest population risk on this problem.

$$y^* \coloneqq \arg\min_{f \in \Omega} R(f) \tag{71}$$

This gives us **four** important models to consider within our overall space Ω.

- $\hat{f}$ ... A trained model, returned by $\mathcal{A}(f; \mathcal{S})$
- f_{erm} ... An empirical risk minimizer, i.e. assume a perfect learning algorithm, but using limited data
- f^* ... The best-in-family model, i.e. perfect learning algorithm, using *infinite* data
- y^* ... The Bayes model, i.e. perfect learning algorithm, infinite data, and infinite model capacity.

As we go down the list, we're making more assumptions about available resources. These models can be visualised in our function class diagram.



The difference of risks between each pair have special names.

First, let's take $\hat{f}$ versus y^*. This is the extra (excess) risk we have with $\hat{f}$, on top of that with y^*

$$\text{ExcessRisk}(\hat{f}) \;=\; R(\hat{f}) - R(y^*). \tag{72}$$

Note that by definition, the excess risk is always ≥ 0 since y^* minimizes population risk over Ω.

With a simple trick, this excess risk can be written as a sum of terms, each of which is a gap between pairs of the models above. These are called the *optimisation* error, estimation error, and approximation error:

$$\underbrace{R(\hat{f}) - R(y^*)}_{\text{excess risk}} \;=\; \underbrace{R(\hat{f}) - R(f_{erm})}_{\text{optimisation error}} \;+\; \underbrace{R(f_{erm}) - R(f^*)}_{\text{estimation error}} \;+\; \underbrace{R(f^*) - R(y^*)}_{\text{approximation error}}. \tag{73}$$

Whilst this seems trivial—just adding/subtracting terms in the middle—the components actually have meaningful interpretations within the excess risk.

Optimization error is there because we could not find f_{erm} with our learning algorithm. This is a *computational limitation*: even with the right function class and enough data, a practical learning algorithm may fail to reach the empirical risk minimiser, and thus have no chance of finding a useful model.

Estimation error is there because we could not find f^*, since we had a limited sample $\mathcal{S}$. This is a *statistical limitation*: we use the finite sample $\mathcal{S}$ to guide our learning, but if it's not a *good* sample from $p(\boldsymbol{x}, y)$, then it will guide us in the wrong direction. Thus, even if there's a great model $f^* \in \mathcal{F}$, we may not be able to find it. More importantly, we may have multiple models (ERMs) that all look *equivalent* with the finite dataset, and we have no way to distinguish them—this means the estimation error becomes *another* random variable for us to think about.

Approximation error is there because we restrict our function class, and could not get to y^*. This reflects a *structural limitation*: even with infinite data and perfect optimisation, we are confined to a restricted function class $\mathcal{F}$. If the Bayes model y^* lies outside this class, no amount of computation or data can remove this gap.

Seen in this light, underfitting and overfitting are surface phenomena rather than fundamental explanations. There are in fact multiple underlying factors that all contribute. Underfitting is typically dominated by approximation error: the function class is too restrictive. Overfitting is typically dominated by estimation error: the class is too rich relative to the available data. But these are general rules of thumb, not laws. In real systems, all three terms coexist and interact.

In the next section, we will explore what can happen when we change the size of the function class.

5.2 Let's expand our function class

Up to this point, we have analysed risk by fixing a function class $\mathcal{F}$ and asking how well we can perform within that class. Now we're going to consider *expanding* our function class.

The *approximation error* measures the additional risk of the *best* model in our class, compared to the Bayes model. Now consider enlarging this class in a particular way. Suppose we have a nested sequence

$$\mathcal{F}_1 \subset \mathcal{F}_2 \subset \mathcal{F}_3 \subset \cdots \subset \Omega.$$

As the class grows, the minimum achievable population risk *cannot increase*:

$$\min_{f \in \mathcal{F}_{k+1}} R(f) \leq \min_{f \in \mathcal{F}_k} R(f).$$

This is simply because $\mathcal{F}_{k+1}$ contains $\mathcal{F}_k$. This implies that the best model in $\mathcal{F}_k$, is also in $\mathcal{F}_{k+1}$. All this means the approximation error *never increases as we expand the class.*

At the same time, the estimation problem becomes more delicate. When we choose f_{erm} from a larger class using the finite set $\mathcal{S}$, we are selecting from a richer collection of functions. The additional flexibility that reduces approximation error also increases the number of ways in which the training sample can be fit closely, including ways that capture noise rather than structure. See the illustration below....

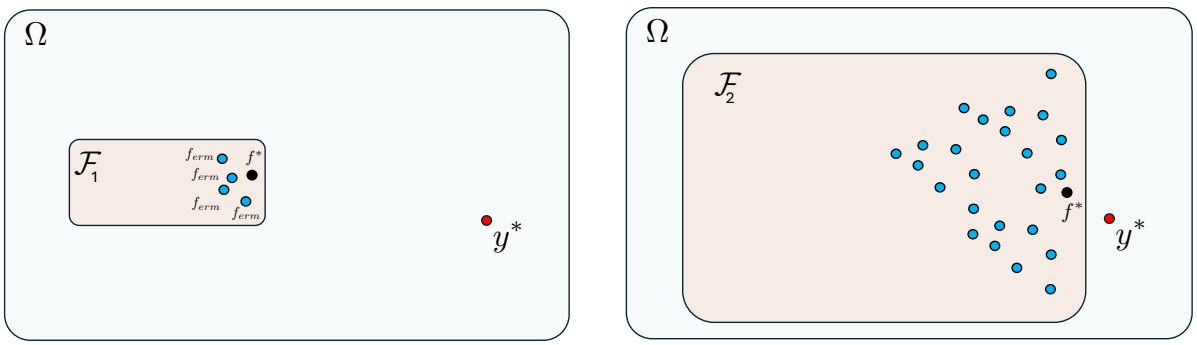
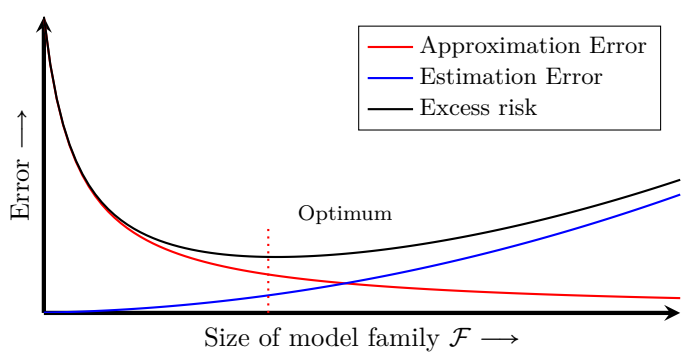


Figure 17: LEFT: $\mathcal{F}_1$ is a *small* function class. Approximation error is high (f^* is a long way from y^*) but estimation error is low, as there are few ERMs, so it's relatively easy to find something close to f^*. **RIGHT:** $\mathcal{F}_2$ is a much larger function class. Approximation error is lower, as there are more rich/complex functions, getting closer to y^*. However estimation error is higher—with complex models there are many ways to fit the data. This creates many ERMs (blue circles) which capture noise rather than structure, and it becomes harder to distinguish them, as they all have low (or zero) empirical risk on the finite $\mathcal{S}$. The esimation error is determined by which one of these we randomly end up with.

As the size of the function class increases, the approximation error falls, but beyond a certain point the estimation error dominates and overall test risk begins to increase. The optimization error usually plays a nominal role as modern optimizers are so powerful, but it's still there. The resulting curve is a familiar U-shaped relationship between model complexity and generalization performance....



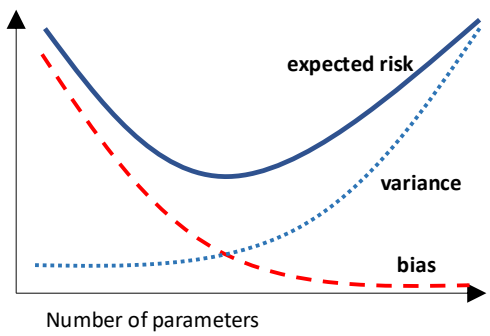
Looks familiar? This is very reminiscent of the bias/variance trade-off we saw last week. But, the approximation/estimation/optimization decomposition provides a more fine-grained way to understand overfitting—attributing components to specific sources, instead of just saying one is systematic (bias) and one random (variance). The two decompositions can be related. The basic idea is:

$$\text{excess risk} \quad = \quad \underbrace{(\text{approximation}) + (\text{estimation})}_{\text{bias}} + \underbrace{(\text{optimization})}_{\text{variance}}$$

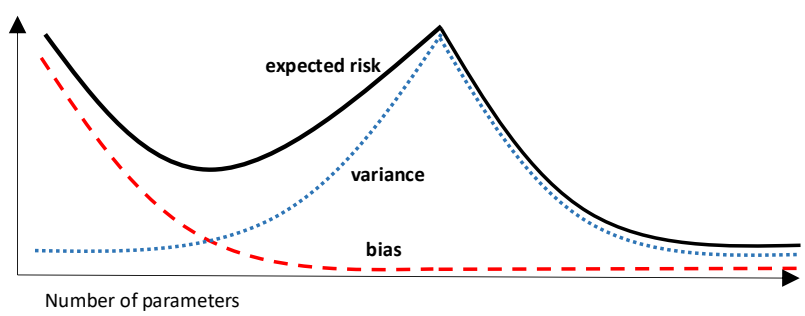
i.e. the bias is the approximation error plus *part* of estimation error, and the variance is the remaining part, plus the optimization error. Full details are beyond the scope of this course[4], but this relation can be explored a bit more.

5.3 The bias/variance trade-off cannot explain ALL overfitting: The Double-Descent Phenomenon

Figure 10 was plotted using decision tree models. For many years, in Universities across the world, lecture notes (including my own) showed an idealized illustration of this, like below.



What are we meant to conclude from this? More complex models overfit. This was the dominant view in Machine Learning for *decades.* Make your models complex, but not too complex, to avoid over-fitting. However, with the advent of deep neural networks, Belkin et al. (2018) found that if they kept making them deeper, they sometimes got this...



Deep Neural Networks can exhibit a "double descent" behaviour in their risk. The model starts to overfit, but eventually improves even better than it was before. Somehow the variance peaks, then decreases again. This really surprised the ML research community[5], and it's still considered a largely open research question.

At first sight, it appears to contradict the classical bias–variance story. According to that framework, enlarging the function class should only increase variance, and so the model should therefore generalize poorly.

[4] If you want to know more see Brown and Ali (2024).

[5] Why didn't someone make a deeper network earlier? Well they kind of did. But the compute power needed was orders of magnitude beyond what we had, and only really became possible in the mid-2010s, with the advent of GPU compute farms. Once this bottleneck was removed, the issue could be explored, e.g. `https://openai.com/index/deep-double-descent/`

However, the first thing to note is the x-axis above, which is the *number of parameters*—we mentioned in the last couple of weeks how the parameter count does not necessarily determine the capacity of the model. So what does determine the capacity in such a scenario?

One popular theory is the idea of *implicit regularization.* The peak in the graph tends to be at a particular point on the x-axis, where the number of parameters p is more than the number of training examples n. This regime, $p \gg n$, is called an **over-parameterized model**. The dominant algorithm for training neural nets is *stochastic gradient descent* (SGD). Recent results have shown that in *over-parameterized* scenarios, SGD has a preference (i.e. it is *"implicitly regularized"*) toward simpler solutions, in particular those with small absolute values of the weights, called a "small norm". In our approximation/estimation/optimization decomposition, this means some ERMs are preferred over others, which causes the estimation error to go down as we move into the over-parameterized regime.

So, while the function class may be huge, the *effective* class explored by the learning dynamics can be far smaller, and more suited to the problem at hand.

5.4 Summary

Understanding generalization requires tools that go beyond counting parameters or reasoning purely in terms of bias/variance. The classical bias–variance trade-off is not incorrect, but incomplete. It describes how risk behaves as we enlarge a function class under the assumption that empirical risk minimization freely selects among minimizers. In modern overparameterized regimes, the interaction between function class and optimization procedure becomes central.

In the final weeks of this course you will hear about more refined notions of "capacity" that allow us to reason about generalization even in very large function classes.

Understanding the Sources of Risk — Practice Exam Questions

Each week we provide you a few practice questions, representative of what may be in the final written exam. The exam will be in Canvas, so all questions match this style. Answers provided separately.

If you can answer all of **Level 1**... you're on track, but doing the bare minimum to pass.
If you can answer all of **Level 2**... you're doing well, more than enough to pass.
If you can answer all of **Level 3**... well ahead, into first class territory. I expect 20% of the class here.

Level 1

1. **Fill in the blank.** The Bayes model is defined as

$$y^* \in \arg\min_{f \in \Omega} \underline{\hspace{3cm}}.$$

2. **True or False.** The empirical risk minimiser depends on the particular dataset that was observed.

3. **Multiple choice.** Which of the following objects is generally *random*?
   (a) The Bayes model
   (b) The best-in-class model
   (c) The empirical risk minimiser
   (d) The function class

4. **Matching.** Match each predictor to its description.

Bayes model	(A) Minimises empirical risk on a dataset
Best-in-class model	(B) Output of a learning algorithm
ERM	(C) Minimises population risk over $\mathcal{F}$
Learned model	(D) Minimises population risk over Ω

5. **True or False.** Approximation error can be reduced by increasing the size of the training dataset.

Level 2

6. **Fill in the blank.** Approximation error arises because the function class $\mathcal{F}$ does not contain the _____.

7. **Multiple choice.** Which error component is affected by changing the optimisation algorithm, while keeping the dataset and function class fixed?
   (a) Approximation error
   (b) Estimation error
   (c) Optimisation error
   (d) Bayes error

8. **Numerical calculation.** You are given the following population risks:

$$R(\hat{f}) = 0.48, \quad R(f_{\text{erm}}) = 0.40, \quad R(f^*) = 0.32, \quad R(y^*) = 0.30.$$

   Compute:
   (a) Approximation error
   (b) Estimation error
   (c) Optimisation error

9. **True or False.** If optimisation error is zero, then the learned model equals the ERM.

10. **Multiple choice.** In a large-data regime with a very simple function class, which error component is most likely to dominate?

    (a) Approximation error
    (b) Estimation error
    (c) Optimisation error
    (d) Generalisation gap

Level 3

11. **Multiple choice.** Which of the following equalities must hold if optimisation error is zero?

    (a) $R(\hat{f}) = R(y^*)$
    (b) $R(\hat{f}) = R(f^*)$
    (c) $R(\hat{f}) = R(f_{\text{erm}})$
    (d) $\hat{R}(\hat{f}, \mathcal{S}) = R(f^*)$

12. **Numerical reasoning.** Suppose the following quantities are observed:

$$R(y^*) = 0.20, \quad R(f^*) = 0.20, \quad R(f_{\text{erm}}) = 0.35.$$

    Which error component is strictly positive?

    (a) Approximation error only
    (b) Estimation error only
    (c) Optimisation error only
    (d) Both estimation and optimisation error

13. **True or False.** The double descent phenomenon implies that increasing model capacity always increases approximation error.

14. **Scenario analysis (single best answer).** A model fits the training data exactly, yet achieves low test error in a highly over-parameterised regime. Which explanation is most consistent with current theory?

    (a) The Bayes risk is zero
    (b) Implicit regularisation induced by the optimisation algorithm
    (c) Elimination of estimation error
    (d) Vanishing approximation error due to finite data

15. **True or False.** Double descent directly contradicts the bias-variance decomposition.

6 How to be Probably Approximately Correct

** Under revision for February 2026 **

6.1 ...

6.2 ...

6.3 ...

7 Some more mathematics ... that you need to know

Outline: In Lectures 8 – 11 we will learn proofs of two kinds (a) how certain ML algorithms work in non-intuitive ways when applied to situations where the number of trainable parameters is larger than the number of training data and (b) how to measure the dependency of the gap between test and train error on the number of trainable parameters.

These kinds of analysis will require being able to quantify various properties of the loss functions, matrices and sequences of vectors. This chapter will introduce all these basic mathematics that will be required in the last 4 lectures of this module.

Concept Check 7.1. Before you start on this chapter, it might be useful to try doing the "Set 1" of practice MCQs given at the end of it. *If you are finding yourself stuck with the first three MCQs of this "Set 1" then you might want to attempt them again after seeing the concepts in linear algebra, as given in Sections 3.1, 3.3-3.6 of this book* — as was mentioned in the pre-requisites given in the handbook.

Alternatively, you can also attempt this "Set 1" after reading this chapter where we will cover these ideas in a slightly restricted setting, than how its taught in the above reference.

We consider 2 different notations for specifying a function, that we shall use interchangeably - as may be suitable in a given context.

Definition 14 (Some Notation About Functions). *Let $\mathbb{R}^p$ and $\mathbb{R}^q$ be $p-$ and $q-$dimensional Euclidean spaces. Then a function F between them would be specified by either of the following 2 definitions,*

$$\begin{aligned} F : \mathbb{R}^p &\to \mathbb{R}^q \\ \boldsymbol{x} &\mapsto F(\boldsymbol{x}) \end{aligned} \tag{74}$$

or in an "in-line" form,

$$\mathbb{R}^p \ni \boldsymbol{x} \mapsto F(\boldsymbol{x}) \in \mathbb{R}^q$$

If one wants to specify that the function F depends on certain parameters (say a vector $\boldsymbol{z}$) then one might replace F with $F_{\boldsymbol{z}}$.

As a demonstration of the above notation, let's consider the squared loss function (say ℓ) on linear predictors at a labelled data $(\boldsymbol{x}, y)$ where y is 1–dimensional and $\boldsymbol{x}$ is p–dimensional. Then we have,

$$\begin{aligned} \ell_{(\boldsymbol{x},y)} : \mathbb{R}^p &\to \mathbb{R} \\ \boldsymbol{w} &\mapsto \ell_{(\boldsymbol{x},y)}(\boldsymbol{w}) = \frac{1}{2} \cdot (y - \langle \boldsymbol{w}, \boldsymbol{x} \rangle)^2 \end{aligned} \tag{75}$$

or in an "in-line" form,

$$\mathbb{R}^p \ni \boldsymbol{w} \mapsto \ell_{(\boldsymbol{x},y)}(\boldsymbol{w}) = \frac{1}{2} \cdot (y - \langle \boldsymbol{w}, \boldsymbol{x} \rangle)^2 \in \mathbb{R}$$

When the data dependency of the loss is not needed to be emphasized one may also simplify $\ell_{(\boldsymbol{x},y)}(\boldsymbol{w})$ to just $\ell(\boldsymbol{w})$. Note that in above, the inner product $\boldsymbol{w}^\top \boldsymbol{x}$ between vectors $\boldsymbol{w}$ and $\boldsymbol{x}$ is denoted as $\langle \boldsymbol{w}, \boldsymbol{x} \rangle$ - a notation that we urge the reader to get familiar with.

Three key properties of differentiable functions that we will focus on are whether they are "Lipschitz" or not, "convex" or not and whether they are "smooth" or not. Which of these properties are true for a given loss function makes a dramatic difference to how machine learning algorithms behave for them.

7.1 Introduction to Lipschitzness, Convexity and Smoothness

Definition 15 (Lipschitzness). *A function $F : \mathbb{R}^p \to \mathbb{R}$ is said to be L–Lipschitz if $\forall\ \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^p$, we have,*

$$|F(\boldsymbol{x}) - F(\boldsymbol{y})| \leq L\|\boldsymbol{x} - \boldsymbol{y}\|$$

Consider, $F(x) = x$. Its clear that this is 1–Lispchitz.

But on the contrary, consider the function $F(x) = x^2$. Now we can compare its values between say a point x and 0 and ask if it is true that there exists a $L > 0$ s.t for all x we will have, $|F(x) - F(0)| = x^2 \leq L|x|$. But clearly for no $L > 0$ can the inequality $x^2 \leq L|x|$ hold if $|x| > L$. Thus $F(x) = x^2$ is not a Lipschitz function.

Concept Check 7.2. At this point the reader is strongly urged to read up on the Huber loss and convince themselves that the squared loss is not a Lipschitz function but the Huber loss is.

M.L. for almost all known scenarios is an optimization problem i.e a question of wanting to minimize or maximize some "objective" function. Finding the global minimum or maximum of a function is extremely challenging for a vast number of classes of functions and yet this is easily approximately doable for very many useful M.L. cases. This dichotomy is at the heart of research in this field.

In this regard, the idea of "convexity" has proven to be remarkably fundamental, to the point that the tractability of an optimization problem is nowadays assessed, more often than not, by whether or not the problem benefits from some sort of underlying convexity. And decades ago this was already pointed out in the famous monograph Rockafellar (1993) where it was said,

"...the great watershed in optimization isn't between linearity and nonlinearity, but convexity and nonconvexity."

A convex function F can be informally understood as one whose graph between any two points x_1 and x_2 always lie below the straight line joining the points $(x_1, F(x_1))$ and $(x_2, F(x_2))$. *Note, that this notion of convexity does not need the function to be differentiable anywhere.* This general idea of convexity is informally defined via the diagram below,

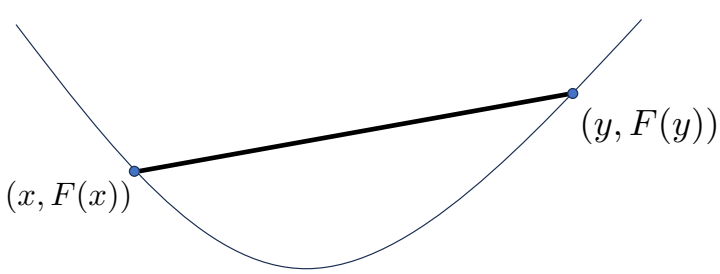


Figure 18: Illustration of Definition 16 for a convex function F

Definition 16 (Convexity (Version 1)). *A function $F : \mathbb{R}^p \to \mathbb{R}$ will be said to be a convex function if $\forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ and $\forall \theta \in [0, 1]$ we have,*

$$F(\theta\boldsymbol{x} + (1 - \theta)\boldsymbol{y}) \leq \theta F(\boldsymbol{x}) + (1 - \theta)F(\boldsymbol{y})$$

This above view of convexity will be extremely important in the last lecture of this course.

Additionally, we would call F to be "strictly convex" if the inequality above holds in the strict sense – i.e it never becomes equality for any choice of $\boldsymbol{x}, \boldsymbol{y}$ and $\theta \in (0,1)$. Note that at $\theta = 0, 1$ i.e at the end points of the cord, the inequality becomes equality by definition. For intuition consider the function, $F(x) = x^4$ - note that this is an example of a strictly convex function while any $\mathbb{R} \to \mathbb{R}$ convex function is not strictly convex if it becomes a constant on any interval in the domain, like $f(x) = \max\{0, x\}$, the "Rectified Linear Unit (ReLU)" - which is the most common activation function in neural nets.

But if the function can be assumed to be differentiable (as is the case for all optimization examples that we will consider) then the following notion of convexity also becomes useful.

Definition 17 (**Convexity (Version 2)**). *An at least once differentiable function $F : \mathbb{R}^p \to \mathbb{R}$ is convex if $\forall\ \boldsymbol{x}, \boldsymbol{y}$ we have,*

$$F(\boldsymbol{x}) + \nabla F(\boldsymbol{x})^{\top}(\boldsymbol{y} - \boldsymbol{x}) \leq F(\boldsymbol{y})$$

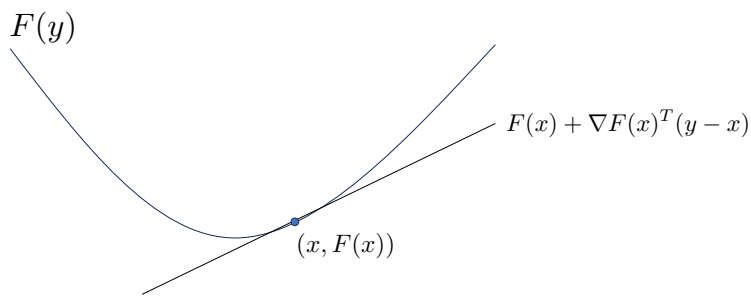


Figure 19: We can visualize Definition 17 as saying that a differentiable convex function can be thought of as having the property that for any point x in the domain, the tangent to the function at that point is below the graph of the function.

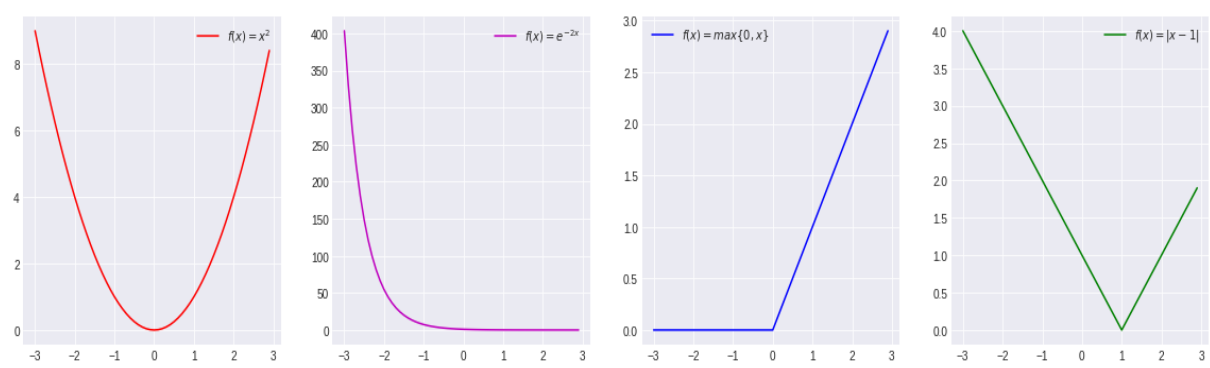


Figure 20: In here we can see represenative samples of what convex functions can look like. From left-to-right notice that we have examples of convex functions which (a) have a unique global minima (b) have no minima at all (c) are not differentiable everywhere and have an uncountable number of global minima and (d) are not differentiable everywhere but have a unique global minima.

Strongly convex functions are a particular subset of convex functions which are of great significance.

Definition 18 (α–**Strong Convexity (Version** 1**)**). *For some $\alpha > 0$, a $F : \mathbb{R}^p \to \mathbb{R}$ is said to be α–strongly convex if we have that $F(\boldsymbol{x}) - \frac{\alpha}{2}\|\boldsymbol{x}\|^2$ is a convex function.*

Essentially, strong-convexity captures the intuition of having a convex function be lowerbounded by a paraboloid. From the definition above it is clear that, for any $\alpha > 0$, adding $\frac{\alpha}{2}x^2$ to any convex function would yield a $\alpha-$strongly convex function. Thus we get a simple mechanism to generate a large number of examples of strongly convex functions.

If we assume differentiability then we can also define strong convexity as follows,

Definition 19 ($\alpha-$**Strong Convexity (Version** 2**)).** *For some $\alpha > 0$, an atleast once differentiable function $F : \mathbb{R}^p \to \mathbb{R}$ is said to be $\alpha-$strongly convex if $\forall\ \boldsymbol{x}, \boldsymbol{y}$ we have,*

$$F(\boldsymbol{y}) \geq F(\boldsymbol{x}) + \nabla F(\boldsymbol{x})^\top(\boldsymbol{y} - \boldsymbol{x}) + \frac{\alpha}{2} \cdot \|\boldsymbol{y} - \boldsymbol{x}\|^2$$

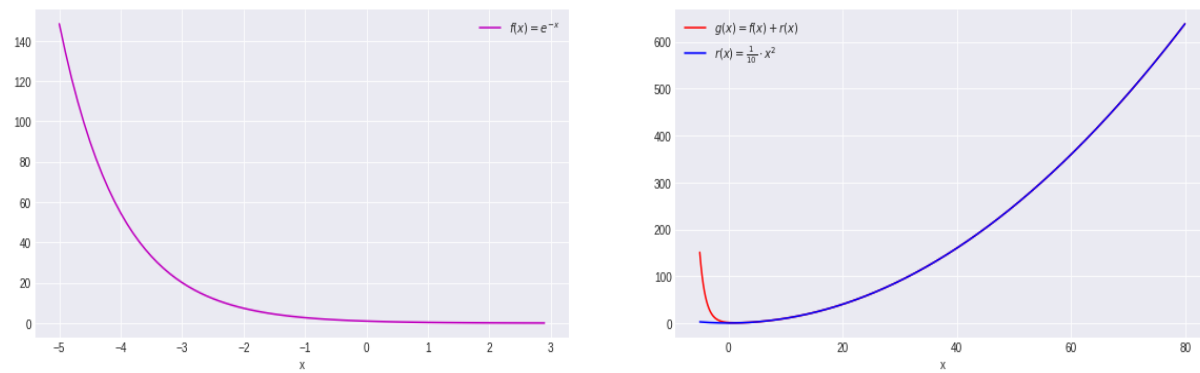


Figure 21: In above we can see how adding an arbitrarily small amount of a strongly convex function r to a convex function f with no minima leads to a strongly convex function g with a unique global minima

Concept Check 7.3. For training data of the form $(\boldsymbol{x}, y)$ and for a real valued predictor h consider the loss being given as $\ell((\boldsymbol{x}, y), f) = e^{-yh(\boldsymbol{x})}$. If $y = \pm 1$ i.e in a binary classification setting, this exponential loss is a very standard loss to try to minimize (over choices of h) for trying to find a good classifier. In particular, it provides a neat sandbox for proving various algorithmic properties which can be messy to establish for other losses. Consider the special case when $h(\boldsymbol{x}) = \langle \boldsymbol{w}, \boldsymbol{x} \rangle$. Then for any $\alpha \geq 0$, a "regularized" version of the exponential loss for the aformentioned data would be,

$$\ell(\boldsymbol{w}) = e^{-y\langle \boldsymbol{w}, \boldsymbol{x} \rangle} + \frac{\alpha}{2}\|\boldsymbol{w}\|^2$$

Show that the above is strongly convex (in $\boldsymbol{w}$) for $\alpha > 0$ but only convex if $\alpha = 0$.

Concept Check 7.4. Prove the following :

Claim 1. *If F is at least twice differentiable then F is $\alpha-$strongly convex iff $\lambda_{\min}([\partial_i \partial_j F]_{i,j=1,\ldots,p}) \geq \alpha$*

The $p \times p$ matrix $[\partial_i \partial_j F]_{i,j=1,\ldots,p}$ is called the "Hessian" of such an at least twice differentiable F.

Definition 20 (**Lipschitz-Smoothness).** *For some $\beta > 0$, an at least once differentiable function $F : \mathbb{R}^p \to \mathbb{R}$ is said to be $\beta-$smooth or $\beta-$Lipschitz smooth or $\beta-$Gradient Lipschitz if its gradients are $\beta-$Lipschitz i.e $\forall\ \boldsymbol{x}, \boldsymbol{y}$, we have,*

$$\|\nabla F(\boldsymbol{x}) - \nabla F(\boldsymbol{y})\| \leq \beta\|\boldsymbol{x} - \boldsymbol{y}\|$$

Eg. $F(x) = x^2$ is smooth with $\beta = 2$.

Note that this notion of "smoothness" as given above albeit standard across machine learning and optimization books, differs from what is called a smooth function in calculus or analysis textbooks! In there, smoothness refers to a function being infinitely differentiable and we can see that there are easy examples of infinitely differentiable functions which are not smooth by the above definition, like $F(x) = x^3$. *One needs to be careful about this conflict of terminologies when reading across different sources!*

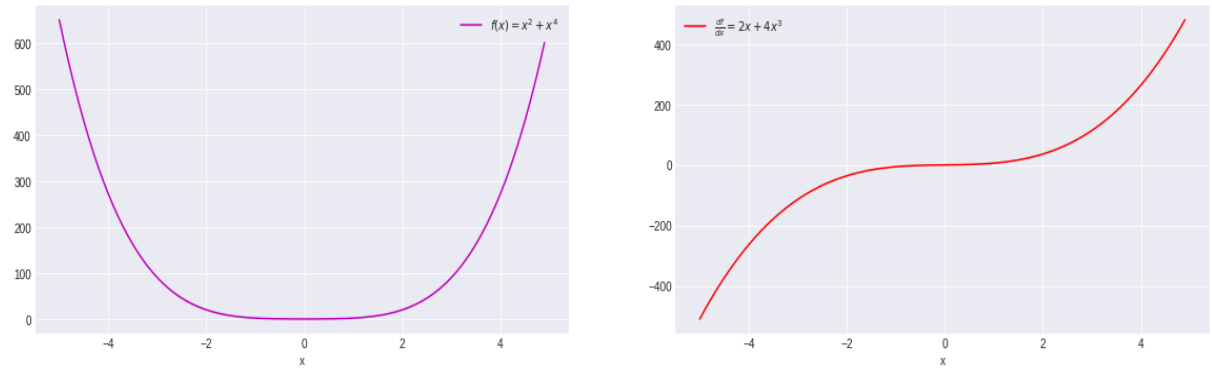


Figure 22: In above we see an example of a quartic function on the left and a plot of its derivative on the right. Its easy to convince onself from these diagrams as to why this quartic function is not Lipschitz smooth.

Concept Check 7.5. Prove the following statements and do the computational exercises embedded inside some of them. Note : consider the following to be functions of the vector $\boldsymbol{x}$ or the scalar x, as the case maybe, and all other letters represent constants/parameters the function depends on.

Some Instructive Examples of Lipschitz Functions

- $\log\left(1+e^{-\beta x}\right), e^{-\beta x}$
  are smooth, Lipschitz & convex functions which are not strongly convex
- $\|\boldsymbol{x}\|_1$
  is non-smooth, Lipschitz & convex but not strongly convex
  – with a unique global minima at $\mathbf{0}$.
- $\frac{1}{1+e^{-\beta x}}$:
  is a smooth, Lipschitz & non-convex function
  – with no critical points
- (a) $-e^{-\lambda\|\boldsymbol{x}\|^2} - p \cdot e^{-\|\boldsymbol{x}-\boldsymbol{a}\|^2}$
  is a smooth, bounded gradient/Lipschitz & non-convex function
  – with a unique global minima and a non-trivial local minima. (find their locations!)
  (b) $\frac{1}{1+e^{-|x|}}$:
  is a non-smooth, Lipschitz & non-convex function
  – with a unique global minima. (find its location!)

Concept Check 7.6. Prove the following statements and do the computational exercises embedded inside some of them. Note: consider the following to be functions of the vector $\boldsymbol{x}$ or the scalar x, as the case maybe, and all other letters represent constants/parameters the function depends on.

Some Instructive Examples of non-Lipschitz Functions

- (a) $x^4 + x^3$
  is a non-smooth, non-Lispchitz & non-convex function,
  with a unique global minima at $x = -\frac{3}{4}$ and an inflection point at $x = 0$.
  (b) $(1-|x|)^2$
  is a non-smooth, non-Lipschitz & non-convex function,
  with 2 global minima at $x = \pm 1$.
- $x^2 + |x|$
  is a non-smooth, non-Lipschitz & strongly convex function,
  with a unique global minima. (find its location!)
- $\|\boldsymbol{x}\|^2$
  is a smooth, non-Lipschitz & strongly convex function.

From the above two sets of exercises, one might have already started guessing that there is an interesting relationship between convexity and the presence or multiplicity of global minima. One fundamental result about this is in the exercise stated below.

Concept Check 7.7. Prove that, for a convex function every local minima of it is global and if it is also strongly convex then it has a unique global minimum.

Recall the function x^4 that we saw earlier as an example of a strictly convex function. Note that it has a unique global minima (at $x = 0$) while it is not strongly convex. But, consider the function e^{-x}, this is another strictly convex function but it does not have any minima at all. And this dichotomy between these two examples is representative – the reader is encouraged to convince themselves that a strictly convex function can either have no global minima or have only one global minima but not multiple.

Next, consider the function that is at the heart of modern deep-learning, $F(x) = \max\{0, x\}$ – which is a convex function that is not differentiable everywhere (and hence it falls within the ambit of Definition 16 but not 17) - and it has an uncountably infinite number of global minima (i.e all $x \leq 0$). This is neither strictly convex nor strongly convex.

Moving ahead from the fact established in the above exercises, we realize that way too many real-world loss functions have local minima which are not global. In the following we shall see an example of such a loss function - and in this example we shall use a strongly convex regularizer to emphasize that its presence alone cannot ameliorate this complexity of having non-trivial local minima.

Example 7.1 (**Example Of A Neural Loss**). *Consider the following training dataset consisting of* 4 *data,* $(x_1, y_1) = (0.5, -100), (x_2, y_2) = (-1, 300), (x_3, y_3) = (1, 1), (x_4, y_4) = (-0.5, -400)$. *We recall that one of the simplest neural nets is one consisting of a single sigmoid gate of weight* w *which would be mapping,*

$$\mathbb{R} \ni x \mapsto \frac{1}{1 + e^{-w \cdot x}} \in [0, 1)$$

Then a very natural instance of a regularized squared loss function, which we call ℓ *below, on the above gate for the above training data would be,*

$$\mathbb{R} \ni w \mapsto F(w) \coloneqq \frac{1}{4} \sum_{i=1}^{4} \ell_{x_i, y_i}(w) + \lambda w^2 \tag{76}$$

$$\coloneqq \underbrace{\frac{1}{4} \sum_{i=1}^{4} \frac{1}{2} \cdot \left(y_i - \frac{1}{1 + e^{-w x_i}} \right)^2}_{\text{mean-squared-loss for the sigmoid gate}} + \underbrace{\lambda w^2}_{\text{“regularizer”}} \in [0, \infty) \tag{77}$$

Notice how we think of losses as being univariate functions of the weights, while the gates are thought of as univariate functions of the input data. [6]

Now we can plot this F *above, for say* $\lambda = 0.13$, *as a function of* w *to obtain the following plot — which is clearly a non-Lipschitz function (owing to the regularizer) and neither is it convex (since it has a local maxima) and notice how it has two local minima, only one of which is a global minima.*

[6] Up to the λ–term note that the function F in equation 76 is an instance of the quantity "$\hat{R}(f, \mathcal{S}_n)$".

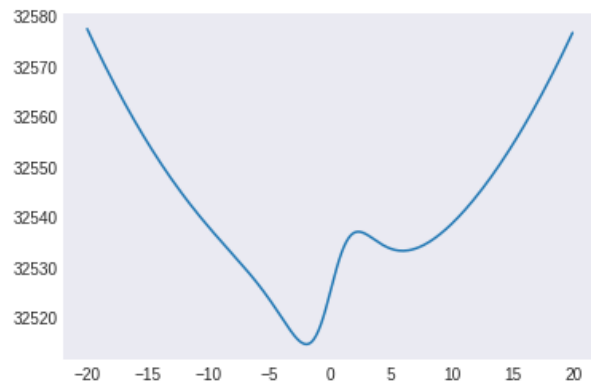


Figure 23: A plot of the empirical loss function, $F(w)$ of a very simple neural net vs its weight w

7.2 Introduction to Convergence

At this point, the students who have done the course COMP24112 might want to review what they have already seen in Chapter 6A therein, about how gradient descent works on ℓ_2−regularized linear regression. A key feature of that example - which made things very easy - is that the loss function used there was convex. But in a very large number of powerful ML applications in the real-world this property does not hold. Towards enlarging the horizon of possibilities that we want to capture, in what follows we shall again see an example of gradient descent on a regularized squared loss function but now on neural nets - a situation where the loss is not a convex function of the training parameters.

To motivate the idea of convergence let us return to the simple example of a neural loss function that we had seen in equation 76. To match usual conventions in mathematical literature on convergence, we shall rename the weight variable therein as x. Thus we have,

$$\mathbb{R} \ni x \mapsto F(x) \coloneqq \frac{1}{4}\sum_{i=1}^{4} \ell_{x_i,y_i}(x) + 0.13 \cdot x^2 \coloneqq \frac{1}{4}\sum_{i=1}^{4} \frac{1}{2} \cdot \left(y_i - \frac{1}{1+e^{-x \cdot x_i}}\right)^2 + 0.13 \cdot x^2 \in [0, \infty) \tag{78}$$

Recall, that on the above loss function , an implementation of the "Gradient Descent" algorithm at a constant "step-length" of $\eta > 0$ and starting from x_0 would be an iterative execution of the following method,

$$x_{t+1} = x_t - \eta \cdot \frac{\mathrm{d}F}{\mathrm{d}x}\Big|_{x_t}$$

Said in words, at "time t" or at the tth−step we change the current value of the weight additively by an amount which is $-\eta$ times the current value of the gradient - that is we take a step η times the value of the current gradient but in the opposite direction of the current gradient. Let's see visually as to what the progress of such an algorithm looks like when starting from $x_0 = 7$ and the above being run for 10^3 steps (Figure 24), 10^4 steps (Figure 25) and 10^5 steps (Figure 26).

It is clear from the Figures 24 - 26 that the algorithm makes significant progress between where it was after 10^3 steps and the next 9000 steps. But for the next 90000 steps after $t = 10^4$, almost nothing seems to happen - the algorithm seems to have stalled. This is a tell-tale sign of "convergence" - that this sequence of numbers x_t seems to be getting arbitrarily close to a certain number as $t \to \infty$.

Also, since the function f here is an actual example of a neural loss function, along the way these experiments also happen to point out that without a careful choice of starting point and step-lengths, G.D. can get "stuck"

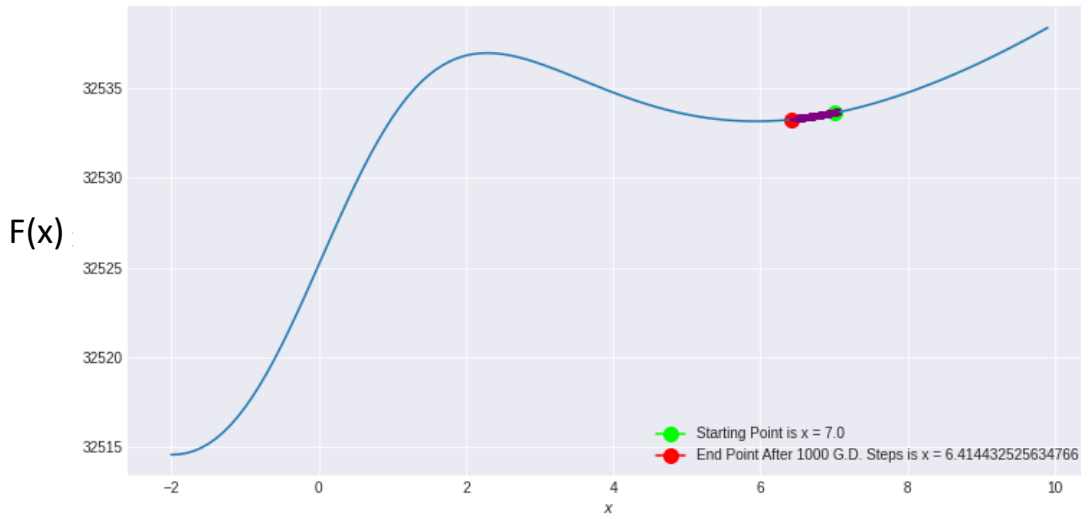


Figure 24: G.D. on F starting from $x_0 = 7$ and using a constant step-length of $\eta = 10^{-3}$ and running for 10^3 steps reaches $x \sim 6.414$

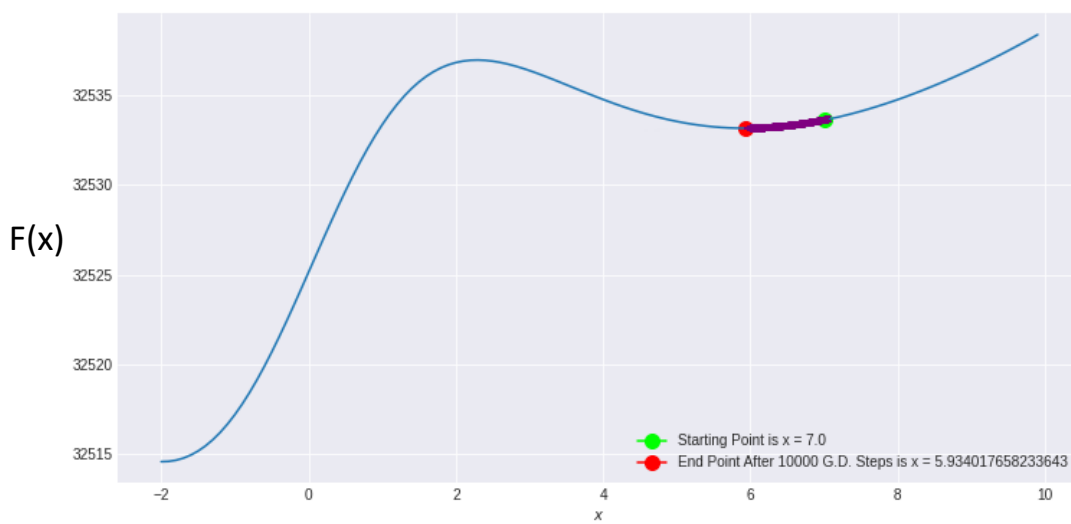


Figure 25: G.D. on F starting from $x_0 = 7$ and using a constant step-length of $\eta = 10^{-3}$ and running for 10^4 steps reaches $x \sim 5.934$

at a non-trivial local minima - and not reach the global minima of the function. From Figure 23 we know that this f indeed has a unique global minima, which the G.D. settings in the above diagrams seem to fail to reach.

Concept Check 7.8. Play around with the above function and G.D. on it to figure out values of x_0 and η for which G.D. would converge to its global minima.

Thus we realize that to understand how the gradient descent algorithm works, it's absolutely essential to formally define and understand the notion of convergence. And that's what we shall do below, in general for sequences in arbitrary dimensions - as is the ubiquitous case for G.D. running on multi-dimensional/parameter models that are useful in practical M.L.

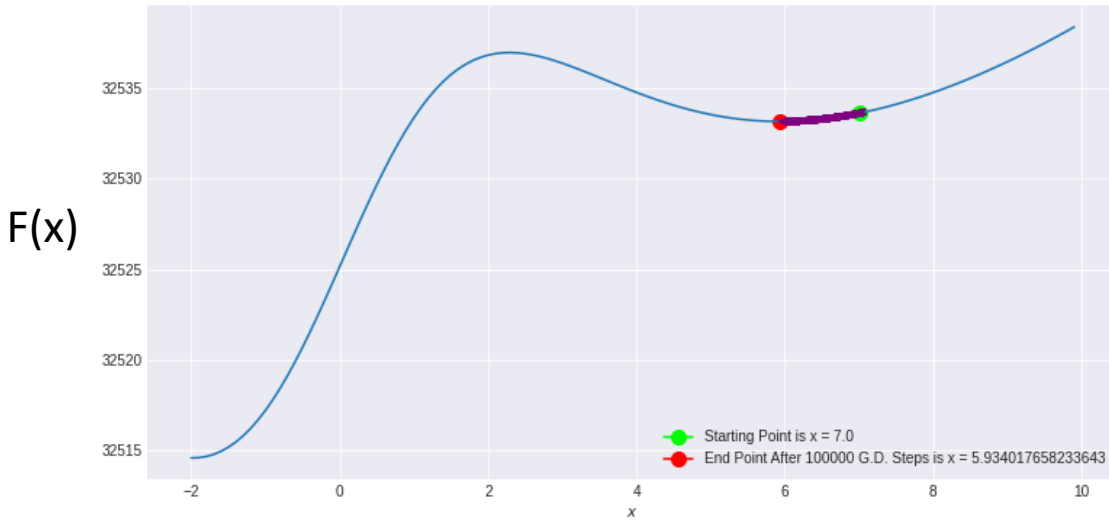


Figure 26: G.D. on F starting from $x_0 = 7$ and using a constant step-length of $\eta = 10^{-3}$ and running for 10^5 steps reaches $x \sim 5.934$

Definition 21 (Defining Convergence (Version 1)). *A sequence of points $\boldsymbol{x}_i, \forall i = 1, 2, \ldots$ in $\mathbb{R}^n$, will be said to converge to a point $\boldsymbol{x}_* \in \mathbb{R}^n$ if $\lim_{n\to\infty} \|\boldsymbol{x}_n - \boldsymbol{x}_*\|_2 = 0$. If this condition is satisfied then $\boldsymbol{x}_*$ will be called the "limit point" or the "accumulation point" of the sequence.*

Definition 22 (Defining Convergence (Version 2)). *A sequence of points $\boldsymbol{x}_i, \forall i = 1, 2, \ldots$ in $\mathbb{R}^n$, will be said to converge to a point $\boldsymbol{x}_* \in \mathbb{R}^n$, if $\forall \epsilon > 0$, all but finitely many members of the sequence satisfy the condition that $\|\boldsymbol{x}_n - \boldsymbol{x}_*\|_2 \leq \epsilon$. If this condition is satisfied then $\boldsymbol{x}_*$ will be called the "limit point" or the "accumulation point" of the sequence.*

The later of the above views is more fundamental because it can be generalized to more situations than just the Euclidean space, $\mathbb{R}^n$. For the purposes of this course, the first definition shall be sufficient.

Concept Check 7.9. Why are the above two definitions of convergence equivalent?

We can build further intuition by seeing two simple examples,

- The sequence $\frac{1}{2}, \frac{1}{2^2}, \frac{1}{2^3}, \ldots$ is a decreasing sequence that converges to 0
- The sequence $\left(-1 - \frac{1}{2}\right), \left(-1 - \frac{1}{3}\right), \left(-1 - \frac{1}{4}\right), \ldots$ is an increasing sequence that converges to the point -1

7.3 Introduction to Supremums and Infimums

Consider the set of numbers $\{0, -1, 1.2\}$. Its quite easy to say that -1 is the lowest number in the set and 1.2 is the largest. But note that this simple idea gets a lot more subtle when we have to deal with infinite sets. Consider the set of numbers which are greater than equal to 0 but less than 1 i.e the interval $[0, 1)$. (Such an interval is said to be "closed" on the left and "open" on the right. In this course we shall not get into the details of this terminology.) Now notice two salient points, that (a) every number in this interval is less than 1 while 1 is not in the interval and (b) that if one picks any number $x \in [0, 1)$ and wants to declare x as the maximum of this interval then one can always come up a very small number $\epsilon > 0$ s.t $x + \epsilon \in [0, 1)$

and thus $x+\epsilon$ becomes a more credible candidate for being called the maximum - and starting from $x+\epsilon$ we can again repeat this argument and this can be continued ad infinitum!

Thus we are led to realize that the language of "maximums" and "minimums" is simply not sufficient to quantify how large an infinite set is. Thus arises the need to define the ideas of "supremums" and "infimums". Towards that we need to first define the idea of an "upper bound" and a "lower bound".

Definition 23 (**Upper Bound & Lower Bound**). *Let I be a subset of $\mathbb{R}$. Suppose there exists a number M s.t $x \leq M$, $\forall x \in I$. Then M is an upper bound on I. Suppose there exists a number m s.t $x \geq m$, $\forall x \in I$. Then m is a lower bound on I.*

To understand the above definition consider two situations (a) For the case $I = [0,1)$ considered earlier realize that it has uncountably many upper and lower bounds. For a start realize that the integers $1, 2, 3, \ldots$ are all upper bounds on I and further, every real number greater than 1 is an upper bound on I - and there is an uncountably infinite number of them. Similarly, $-1, -2, -3, \ldots$ are all lower bounds on I and that further, every real number less than 0 is a lower bound on I - and there is an uncountably infinite number of them.

(b) Consider $I = [0,1) \cup [1.5, 2]$. Now a number like 1.3 is an upper bound on a part of the I and is a lower bound on another part of I. Thus 1.3 is neither an upper bound nor a lower bound on this I. But realize that every real number greater than 2 is an upper bound on this I and every real number less than 0 is a lower bound on this I.

The realization of the multiplicity of the idea of upper and lower bounds leads us to the definition of supremums and infimums.

Definition 24 (**Supremum**). *Let I be a subset of $\mathbb{R}$. Suppose there exists an upper bound M on I s.t $M \leq M'$, $\forall$ upper bounds M' on I. Then M is the "least upper bound" on I and it's called the "supremum" of I and is denoted as $\sup I$. And when $\sup I \in I$ we call it the "maximum" of I, $\max I$.*

Definition 25 (**Infimum**). *Let I be a subset of $\mathbb{R}$. Suppose there exists a lower bound m on I s.t $m \geq m'$, $\forall$ lower bounds m' on I. Then m is the "greatest lower bound" on I and it's called the "infimum" of I and is denoted as $\inf I$. And when $\inf I \in I$ we call it the "minimum" of I, $\min I$.*

For intuition, we can look back at the previous examples to realize that (a) when $I = [0,1)$, we have $\sup I = 1$ and $\inf I = 0 = \min I$. And when $I = [0,1) \cup [1.5, 2]$ we have, $\inf I = 0 = \min I$ and $\sup I = 2 = \max I$.

It is easy to see that one can as well use the above definitions when the interval under consideration arises as the evaluation of some real valued function on a space of functions. Like, consider the "functional" F which maps a polynomial (say f) to its derivative at 1 i.e $F(f) = \frac{\mathrm{d}f}{\mathrm{d}x}|_{x=1}$. Now consider a space of quadratic functions given as, $\mathcal{F} = \{x^2 + m \cdot x \mid m \in [-1, 1]\}$. Then we have $F(x^2 + m \cdot x) = 2 + m$ – and its largest value over all functions in the set $\mathcal{F}$ is 3. This we shall write as, $\sup_{f \in \mathcal{F}} \left(\frac{\mathrm{d}f}{\mathrm{d}x}|_{x=1}\right) = 3$

Concept Check 7.10. Recompute the value of $\sup_{f \in \mathcal{F}} \left(\frac{\mathrm{d}f}{\mathrm{d}x}|_{x=1}\right)$ if $m \in [-1, 1)$

7.4 Introduction to Spectral Norms of Matrices

Definition 26. *For any matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ we define its $(2,2)$-norm as,*

$$\|\boldsymbol{A}\|_{2,2} \coloneqq \sup_{\|\boldsymbol{x}\|_2 = 1} \|\boldsymbol{A}\boldsymbol{x}\|_2 = \sup_{\boldsymbol{x} \neq 0} \frac{\|\boldsymbol{A}\boldsymbol{x}\|_2}{\|\boldsymbol{x}\|_2}$$

When its clear from context, the above will also be called just the $2-$norm of $\boldsymbol{A}$ and denoted as $\|\boldsymbol{A}\|_2$ or spectral norm of $\boldsymbol{A}$.

Its clear from the above definition that the spectral norm of a matrix captures the largest the matrix can stretch in length any unit vector in its domain. As a special case of the above consider the situation when $n = 1$ i.e when $\boldsymbol{A}$ is a column vector of m dimensions.

Then we have,

$$\|\boldsymbol{A}\|_2 = \sup_{|x|=1} \|\boldsymbol{A}x\|_2 = \sup_{|x|=1} |x| \|\boldsymbol{A}\|_2 = \|\boldsymbol{A}\|_2 = \sqrt{\sum_{i=1}^{m} A_i^2}$$

Thus we see how the idea of $(2,2)-$norm contains as a special case the more elementary idea of an Euclidean norm of a vector.

Lets see a somewhat tricky example to get a feel for why the idea of a spectral norm of a matrix is more subtle than it might seem at first glance. Consider the following family of matrices parameterized by a real number θ,

$$\boldsymbol{A}(\theta) \coloneqq \begin{bmatrix} 0 & \theta \\ 0 & 0 \end{bmatrix} \tag{79}$$

One can solve the equation $\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$ to obtain the eigenvalues (λ) and eigenvectors ($\boldsymbol{v}$) for this matrix and realize that its only eigenvalue is 0. But, for an arbitrary vector on the unit circle in $\mathbb{R}^2$ - parameterized as say $\begin{bmatrix} \sin(\alpha) \\ \cos(\alpha) \end{bmatrix}$ we have,

$$\left\| \boldsymbol{A}(\theta) \begin{bmatrix} \sin(\alpha) \\ \cos(\alpha) \end{bmatrix} \right\| = \left\| \begin{bmatrix} \theta \cos(\alpha) \\ 0 \end{bmatrix} \right\| = |\theta| \cdot |\cos(\alpha)|$$

Thus we have,

$$\|\boldsymbol{A}(\theta)\| = \sup_{\alpha} |\theta| \cdot |\cos(\alpha)| = |\theta|$$

Thus by choosing θ to be arbitrarily large in magnitude we can make the spectral norm of the matrix $\boldsymbol{A}(\theta)$ as large as we want, while the largest eigenvalue magnitude (also called the "spectral radius" of a matrix) remains at 0 at all θ. In general one can show that the spectral radius of a square matrix is upperbounded by its spectral norm but as we see in this example, the gap between them can be arbitrarily large. One can generalize this argument above to any upper triangular matrix in any dimensions with 0s along its diagonal and construct similar examples in larger dimensions.

Optional material (non-assessed)
The reader is strongly encouraged to read up about the ideas of a "singular value" of a matrix and understand how it is equal to the idea of the 2−norm given above.

7.5 Introduction to Rank of Matrices

Inside $\mathbb{R}^2$ denote two special vectors, $\boldsymbol{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\boldsymbol{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, Note that any vector $\boldsymbol{w}$ in $\mathbb{R}^2$ can be written as $\boldsymbol{w} = w_1\boldsymbol{e}_1 + w_2\boldsymbol{e}_2$ for some choice of numbers w_1 and w_2. Hence the two vectors $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ are said to "span" the "vector space" $\mathbb{R}^2$. We can formally define the idea of "span" as follows,

Definition 27 (Span). *Given a finite set of vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$, each in $\mathbb{R}^n$, we denote,*

$$\mathrm{Span}(\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}) \coloneqq \left\{ \sum_{i=1}^{k} w_i \boldsymbol{v}_i \mid w_i \in \mathbb{R}, \forall i = 1, \ldots, k \right\}$$

And we will call the set of vectors $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ as being the "spanning set" for the (vector) space, $\text{Span}(\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\})$.

Infact, a little more thinking gets us to realize that every vector in $\mathbb{R}^2$ can be written in a unique way as a weighted linear combination of $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$. This makes the set $\{\boldsymbol{e}_1, \boldsymbol{e}_2\}$ not only a spanning set for $\mathbb{R}^2$ but also a "basis" for the space $\mathbb{R}^2$. That $\mathbb{R}^2$ can be spanned by a basis of size two is what defines the fact that $\mathbb{R}^2$ is a "2–dimensional" vector space.

Concept Check 7.11. Using the fact that $\boldsymbol{e}_1^\top \boldsymbol{e}_2 = 0$, prove the claim that every vector in $\mathbb{R}^2$ can be written in a unique way as a weighted linear combination of the vectors $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$.

It is easy to see how to extend the above discussion to $\mathbb{R}^n$.

And, we give the following definitions in that generality,

Definition 28 (**Basis & Dimension**). *Given a finite set of mutually orthogonal vectors $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_p$, each in $\mathbb{R}^n$, we denote,*

$$p = \text{dimension}\left(\text{Span}(\{\boldsymbol{z}_1, \ldots, \boldsymbol{z}_p\})\right)$$

$$\{\boldsymbol{z}_1, \ldots, \boldsymbol{z}_p\} \text{ is an instance of a basis of } \text{Span}(\{\boldsymbol{z}_1, \ldots, \boldsymbol{z}_p\})$$

It is important to note that (a) in above $\text{Span}(\{\boldsymbol{z}_1, \ldots, \boldsymbol{z}_p\})$ *has other basis too than the one specified above and (b) the ideas of a "basis" and "dimension" do not need the notion of orthogonality to be defined - as was assumed in above. Towards understanding those more general ideas, that the reader is strongly encouraged to read up on the formal definition of a vector space (link) - an idea which will be good to know to understand some of the details in Chapter 9.*

Concept Check 7.12. Let $\boldsymbol{z} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$. Now convince yourself that the set of vectors $\{\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{z}\}$ is also a spanning set for $\mathbb{R}^2$. Now choose an arbitrary vector, say $\begin{bmatrix} 1 \\ 2 \end{bmatrix}$ and show that it can be realized as multiple distinct linear combinations of the vectors $\{\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{z}\}$. Thus realize that this set of three vectors although a spanning set for $\mathbb{R}^2$ is not a basis.

Now, consider the following 3 matrices,

$$\boldsymbol{A} := \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, \boldsymbol{B} := \begin{bmatrix} -1 & 0 \\ 0 & 0 \end{bmatrix}, \boldsymbol{C} := \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix},$$

All the 3 matrices map from $\mathbb{R}^2 \to \mathbb{R}^2$ i.e given any arbitrary vector $\boldsymbol{v} := \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} \in \mathbb{R}^2$ we gave $\boldsymbol{A}[\boldsymbol{v}], \boldsymbol{B}[\boldsymbol{v}]$ & $\boldsymbol{C}[\boldsymbol{v}]$ all in $\mathbb{R}^2$. We realize that, $\forall \boldsymbol{v} \in \mathbb{R}^2$, we have,

$$\boldsymbol{A}[\boldsymbol{v}] = 2v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2, \;\; \boldsymbol{B}[\boldsymbol{v}] = -v_1\boldsymbol{e}_1$$

$$\boldsymbol{C}[\boldsymbol{v}] = 0$$

Relating back to the earlier discussion we can see that the above is tantamount to saying that,

$$\text{dimension}\left(\{\boldsymbol{A}[\boldsymbol{v}] \mid \boldsymbol{v} \in \mathbb{R}^2\}\right) = 2,\ \text{dimension}\left(\{\boldsymbol{B}[\boldsymbol{v}] \mid \boldsymbol{v} \in \mathbb{R}^2\}\right) = 1$$

$$\text{dimension}\left(\{\boldsymbol{C}[\boldsymbol{v}] \mid \boldsymbol{v} \in \mathbb{R}^2\}\right) = 0$$

This is what is encapsulated by the idea of "rank" of a matrix and the calculation done above can be said to have proven that the rank of the matrices $\boldsymbol{A}, \boldsymbol{B}$ &$\boldsymbol{C}$ are $2, 1$ & 0 respectively. *It is easy to see how to extend the above discussion to arbitrary rectangular matrices.* Thus we have motivated the following definitions,

Definition 29 (**Image and Rank of a Matrix**). *Given any $m \times n$ matrix $\boldsymbol{T}$ i.e $\boldsymbol{T} : \mathbb{R}^n \to \mathbb{R}^m$, we define its "image" as,*

$$\text{Image}(\boldsymbol{T}) \coloneqq \{\boldsymbol{T}[\boldsymbol{v}] \mid \boldsymbol{v} \in \mathbb{R}^n\}$$

Then we can define,

$$\text{rank}(\boldsymbol{T}) \coloneqq \text{dimension}(\text{Image}(\boldsymbol{T}))$$

Lastly, if $\text{rank}(\boldsymbol{T}) = \min\{n, m\}$*, then we call* $\boldsymbol{T}$ *to be of "full rank".*

Definition 30 (**Null Space of a Matrix**). *Given any $m \times n$ matrix $\boldsymbol{T}$ i.e $\boldsymbol{T} : \mathbb{R}^n \to \mathbb{R}^m$, we define its "null space" as,*

$$\text{Null Space}(\boldsymbol{T}) \coloneqq \{\boldsymbol{v} \in \mathbb{R}^n \mid \boldsymbol{T}[\boldsymbol{v}] = 0\}$$

Note that sometimes "null space" is also called a "kernel" and we would write the above set as $\ker(\boldsymbol{T})$.

The above two ideas shall be of critical importance in Chapter 9 - which gives one of the most basic demonstrations of a key theme in this course, that of understanding the role of overparameterization.

Concept Check 7.13. Consider the matrix $\boldsymbol{Z} = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 0 & 2 \end{bmatrix}$. What is the rank of $\boldsymbol{Z}$ and $\boldsymbol{Z}^\top$?

Practice MCQs : Chapter 7 (Set 1)

Each week we provide you a few practice questions, representative of what may be in the final written exam. The exam will be in Canvas, so all questions match this style. **Answers to the following MCQs will be provided separately on Canvas and will also be discussed in class**

1. Given a finite set of vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ each in the "vector space of n–dimensional vectors" $\mathbb{R}^n$ we denote, $\text{Span}(\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}) \coloneqq \left\{\sum_{i=1}^{k} w_i \boldsymbol{v}_i \mid w_i \in \mathbb{R}, \forall i = 1, \ldots, k\right\}$ i.e the "span" of a set of vectors is the set of all possible weighted linear combinations of them. Further, suppose we denote as (a, b) the position vector of the point in 2–dimensions with x– and y– coordinates being a and b respectively. Then, the space of all 2–dimensional vectors can be written as which of the following spans,

   a) not possible for the span of any set of vectors
   b) $\text{Span}(\{(1, 0)\})$
   c) $\text{Span}(\{(0, 3)\})$
   d) $\text{Span}(\{(1, 0), (0, 1)\})$

2. Recall the notion of inner-product of two vectors that was introduced in Section 1.5 of the notes. Two vectors are said to be "orthogonal" if their inner-product is 0. Then which of the following pairs of vectors is orthogonal?

   a) $(0, 1)$ and $(3, 0)$
   b) $(1, 0, 3)$ and $(2, 0, 6)$
   c) $(0, 4)$ and $(0, 8)$
   d) $(1, 0)$ and $(2, 3)$

3. Given a finite set of mutually orthogonal vectors $\mathbf{z}_1, \ldots, \mathbf{z}_p$, each in $\mathbb{R}^n$, we denote, $p = \text{dimension}\,(\text{Span}(\{\mathbf{z}_1, \ldots, \mathbf{z}_p\}))$. Further, $\{\boldsymbol{z}_1, \ldots, \boldsymbol{z}_p\}$ is then an instance of a "basis" of the $\text{Span}(\{\mathbf{z}_1, \ldots, \mathbf{z}_p\})$. Then which of the following is a correct statement?

   a) $(1, 0)$ is a basis of $\text{Span}(\{(0, 2)\})$
   b) $(0, 2)$ is a basis of $\text{Span}(\{(2, 0)\})$
   c) $2 = \text{dimension}\,(\text{Span}(\{(2, 0), (0, 5)\}))$

4. Consider the empirical loss given in equation 127 in the notes. Suppose there is a vector $\mathbf{w}_*$ s.t $\mathbf{y} = \mathbf{X}\mathbf{w}_*$. Then a global minima (the point where a function is at its lowest possible value) of this empirical loss is,

   a) $2\mathbf{w}_*$
   b) $-\mathbf{w}_*$
   c) 0
   d) $\mathbf{w}_*$

Practice MCQs : Chapter 7 (Set 2)

1. Which of the following is a convex function?

   a) $|x|$
   b) x^3
   c) $-x^2$
   d) $-\log x$

2. Consider the following loss functions for the data (x, y) evaluated for different predictors, all parameterized by a single weight w. Which of these is a non-convex loss?

   a) $\ell(w) = (y - \max\{0, wx\})^2$
   b) $\ell(w) = e^{-ywx}$
   c) $\ell(w) = \log(1 + e^{-ywx})$
   d) $\ell(w) = (y - wx)^2$

3. Which of the following functions is gradient Lipschitz?

   a) x^2
   b) x^3
   c) $\log \frac{1}{x},\ x \geq 0$
   d) e^x

4. Which of the following is a strictly convex function that does not attain its infimum?

   a) e^x
   b) x^3
   c) $\max\{0, x\}$
   d) x^2

8 A Formal Introduction to the Gradient Descent Algorithm

Outline: In recent years, very large models with billions of parameters have been shown to outperform almost every other type of model. Its strangely curious that the algorithm used to train such models is almost always some variant of the Gradient Descent (GD) algorithm. And over extensive experiments it is now well known that GD behaves very non-intuitively on large models. This week we will see the formal mathematical statement of the Gradient Descent algorithm. Next week we will use this framework to understand a specific case of GD interacting with large models. A basic kind of surprise with GD is that there exists numerous loss functions s.t on them the GD iterates keep getting close to their global minima. Understanding this, is a very deep mystery that is largely unsolved. As an immediate use case of the mathematical definition of GD, in this lecture we will see a simple example where this phenomenon can be proven.

In the last few years there has been a surge in literature on provable training of various kinds of neural nets in certain regimes of their widths or depths or for very specifically structured data. Motivated by detailed experimental studies it has often been surmised that even basic neutral training algorithms, like the so-called Stochastic Gradient Descent (SGD) running on neural losses – with proper initialization and learning rate – converges to a low-complexity solution that generalizes - when it exists. But quantifying these nebulous ideas has proved to be a daunting task.

This situation is further complicated by the fact, that in the real world (for actually training the large neural net models in use) the successful algorithms take their steps by a complicated transformation of the gradients they compute.

To really be able to appreciate the training technologies in use at the bleeding edge of modern data-science, we need to start from the vanilla form of this genre of algorithms i.e **"Gradient Descent" (GD)** whose pseudocode we give below,

Algorithm 2 G.D. on a Differentiable Function $F : \mathbb{R}^p \to \mathbb{R}$

Input: An initial point $\boldsymbol{w}_1$ and a specification $T > 0$ of the number of steps for the G.D.
Choose : A step-size sequence $\eta_t > 0,\ \forall t = 0, 1, 2, \ldots$.
for $t = 0, 1, \ldots, T-1$ **do**
    $\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta_t \cdot \nabla F(\boldsymbol{w}_t)$
end for
Output : $\boldsymbol{w}_T$

In this part of the course we shall take some key steps towards answering the following key question,

"If F has global minima then when does Algorithm 2 find one of them,
– and which one and how fast?"

A fully general answer to the above question is quite a core topic at the forefront of research. *To the best of our knowledge such questions are still not answered for most neural net classes when being trained via squared loss!*

8.1 Introduction to Proof of Convergence of G.D. & More Demonstrations

At this point, the students who have done the course COMP24112 might want to review what they had already seen in Chapter 6C therein, about how gradient descent works on quadratic functions. In what follows we shall start with taking a more precise look at the same.

We recall the idea of convergence of gradient descent that was introduced in the previous lecture. In the following theorem we shall see an example of making that precise for a simple objective function - that is strongly convex, differentiable and Lipschitz smooth.

Theorem 8.1. *Consider doing GD on the function $F(x) = x^2$, starting from any $x_0 \in \mathbb{R}$, $x_0 \neq 0$ and using any choice of constant step-length of $\frac{1-k}{2}$ for any $k \in (0,1)$. Then, (a) the iterates will asymptotically in time converge to the global minima at $x = 0$ and (b) taking $\frac{\log\left(\frac{|x_0|}{\varepsilon}\right)}{\log\frac{1}{k}}$ number of steps is sufficient for the iterates to be within a distance of ε of the global minima, for all $\varepsilon > 0$.*

One way to read the above conclusion is to realize that for GD to reach ε close to the global minima, it is sufficient for GD needs to take $\frac{1}{\log\frac{1}{k}} \cdot \left(\log|x_0| + \log\left(\frac{1}{\varepsilon}\right)\right)$ number of steps. One often ignores the constants involved here and focusses on the ε dependence of the required runtime and states this required number of steps to be scaling with ε as, $\mathcal{O}\left(\log\left(\frac{1}{\varepsilon}\right)\right)$. This is called "linear time convergence" - the reasons for this name if we were to explain would take us far off course.

You will not be not expected to reproduce any proof in the final MCQ exam.
However, answering certain questions would be easier,
if you understood the various techniques used in the proof.

Proof. Since $x = 0$ is the (unique) global minima of the objective function F (and $F(0) = 0$), we shall firstly try to find sufficient conditions so that $\lim_{t\to\infty} x_t = 0$ for x_t being given via Algorithm 2.

From the specification of the iterates of the algorithm we have,

$$\begin{aligned} x_{t+1} =& x_t - \eta_t F'(x_t) \\ =& x_t - \eta_t \cdot 2x_t \\ =& (1 - 2\eta_t)x_t \end{aligned} \tag{80}$$

From here we can "unroll" the recursion relation back in time to get,

$$\begin{aligned} x_{t+1} &= (1-2\eta_t)x_t = (1-2\eta_t)(1-2\eta_{t-1})x_{t-1} \\ &= (1-2\eta_t)(1-2\eta_{t-1})...(1-2\eta_{t-t})x_{t-t} \\ &= x_0 \prod_{i=0}^{t}(1-2\eta_i) \end{aligned} \tag{81}$$

Looking at the above equation we can make a number of observations,

- If it were to be true that $x_0 = 0$ then we can read off from the above that for all times t, we would have $x_t = 0$. Thus the algorithm would not move at all. This is why it was important to assume in the theorem that $x_0 \neq 0$.

  More generally if the algorithm were to start at a point x_0 s.t $F'(x_0) = 0$ i.e x_0 is a "critical point" of F, then the algorithm would not move at all. Thus critical points are not to be used as starting points of GD algorithms.

- We want $x_t \to 0$ (the global minima of f) as $t \to \infty$, and one can see from the above formula that a simple sufficient condition for that to happen is if for some constant $k \in (0,1)$, $(1-2\eta_i) = k$, $\forall i$ – and this can be ensured by choosing a constant step-length as,

$$\eta_t = \frac{1}{2} \cdot (1-k) \tag{82}$$

Making the above choice we can rewrite our previous expression for x_{t+1} as,

$$x_{t+1} = x_0 \prod_{i=0}^{t} (k) = x_0 \cdot k^{t+1} \tag{83}$$

Thus we have,

$$\lim_{t\to\infty} x_t = \lim_{t\to\infty} x_0 \cdot k^t = 0 \tag{84}$$

In the last equality above we have crucially invoked the fact that $k \in (0, 1)$. Thus, we have shown that for a range of choices of constant step-length, gradient descent converges on x^2 to its global minimum, regardless of x_0.

To obtain convergence time ("non-asymptotic") bounds, we check how long it takes to get within a $\varepsilon > 0$ interval of the global minima. That is,

$$|x_{t+1}| \leq \varepsilon \implies |x_0| \cdot k^t \leq \varepsilon \implies t \geq \frac{\log\left(\frac{|x_0|}{\varepsilon}\right)}{\log \frac{1}{k}} \tag{85}$$

Thus we have arrived at the fact that we set out to prove. □

8.1.1 Important Demonstrations of GD Convergence

Now we start to shift gears towards non-trivial examples. Recall the definition of convexity and Lipschitz smoothness seen earlier. One of the key insights in the theory of gradient descent is that one of the basic ways in which a function gets non-trivial for GD to work with is if one or both of these properties stop being true – *as is almost always the case in the real world!*. (Note that both the properties were true in the example above!). Thus we need to start to go beyond the confines of smooth convex functions.

In Figures 27 and 28 we see explicit experimental data of how the iterates of Algorithm 2 progress with time on two very carefully chosen edge cases. In the first one the objective function is a convex function which is not strongly convex and is neither Lipschitz nor Lipschitz smooth. In the second one the objective is a non-convex function which is neither Lipschitz nor Lipschitz smooth. Thus we see two quite varied examples where this deceptively simple looking gradient descent algorithm seems to be finding the global minima and also while using constant step-sizes i.e η_t in Algorithm 2 is chosen to be a constant (called η in the figure captions) and hence t–independent.

Demonstration of GD convergence on a differentiable convex function which is neither Lipschitz nor Lipschitz smooth. The reader is urged to recognize that in Figure 27. we see an instance of a GD algorithm running with fixed step size, starting from $x = 2.5$, and 'converging' to $x = 0$.

Demonstration of GD convergence on a differentiable function which is neither convex, nor Lipschitz nor Lipschitz smooth. The example in Figure 28 can be said to be more complex than the previous one since the target function here is not even convex. But we yet again see that there is an instance of a constant step-size GD algorithm which for all practical purposes seems to converge to the global minima of the target.

In the examples session we shall see some of the above examples in more details. *The reader is strongly urged to recognize that examples like the one in Figure 28 are not at all rare and in certain ways this can be seen to reflect some of the key features of how actual deep-learning happens.*

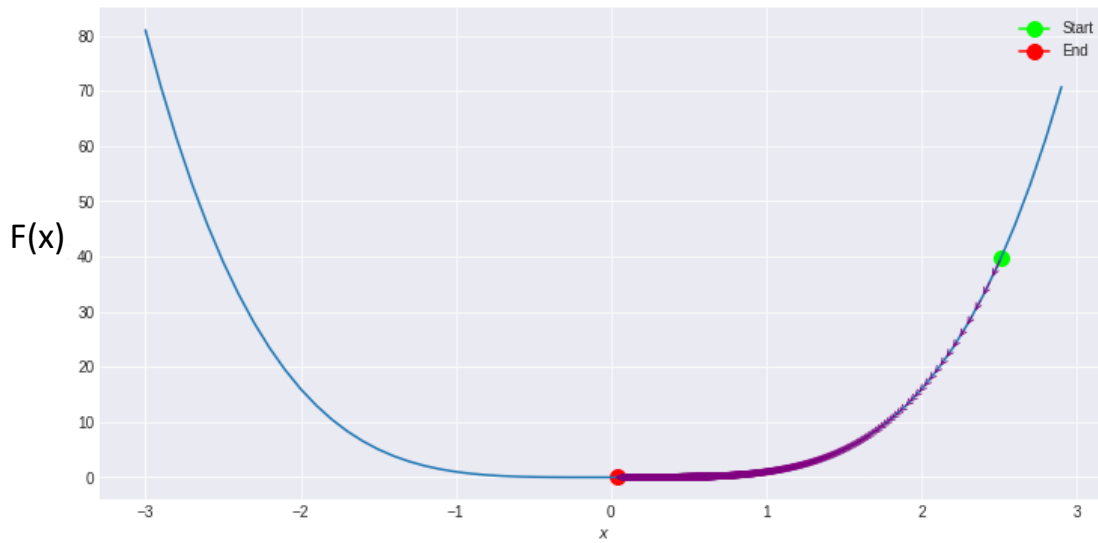


Figure 27: Progress to the unique global minima of Algorithm 2 on the function $F(x) = x^4$ for step-size $\eta = 10^{-3}$ and $T = 10^5$ and the algorithm starting near 2.5

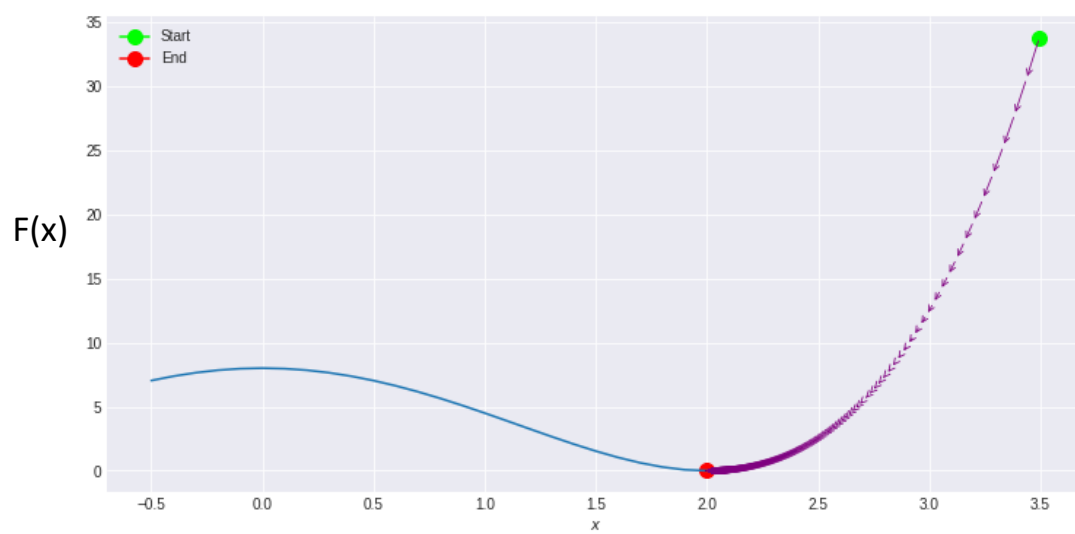


Figure 28: Progress to a global minima of Algorithm 2 on the function $F(x) = \frac{1}{2} \cdot (x^2 - 2^2)^2$ for $\eta = 10^{-3}$ and $T = 10^4$ and the algorithm starting near 3.5

Practice MCQs : Chapter 8

Each week we provide you a few practice questions, representative of what may be in the final written exam. The exam will be in Canvas, so all questions match this style. **Answers to the following MCQs will be provided separately on Canvas and will also be discussed in class**

1. Assume that labeled data is given and that each data $\mathbf{x}$ is labeled with a linear function of it ${\mathbf{w}_*}^\top\mathbf{x}$ for some vector $\mathbf{w}_*$. Given n such data consider the following loss function on a two gate neural net, $\ell(w) = \frac{1}{n}\sum_{i=1}^{n}({\mathbf{w}_*}^\top\mathbf{x}_i - \max\{0, \mathbf{w}^\top\mathbf{x}_i\} - \max\{0, -\mathbf{w}^\top\mathbf{x}_i\})^2$. Notice that in this setup both the gates have the same weight vector. Then which of the following is a correct statement about the global minima of this loss?

   a) only $-\mathbf{w}_*$ is a global minima of ℓ.
   b) only $\mathbf{w}_*$ is a global minima of ℓ.
   c) $\mathbf{w}_*$ and $-\mathbf{w}_*$ are both global minima of ℓ.
   d) none of the above

2. Which of the following is a Lipschitz function?

   a) $\frac{1}{1+e^{-x}}$
   b) e^x
   c) x^2
   d) x^3

3. Suppose GD on some function starting from x_0 has it's t^{th} step being given as $x_t = \frac{x_0}{3^t}$. Suppose that the global minima of the function is at 0. Then which of the following (after approximating to its nearest integer) is a possible estimate of the number of steps this instance of GD will take to get within $\epsilon > 0$ of the global minima?

   a) Undecidable
   b) ϵ
   c) $3x_0 \ln \epsilon$
   d) $\frac{\ln \frac{x_0}{\epsilon}}{\ln 3}$

4. Consider the function $F(x) = e^{-x^2}$. Suppose you do G.D. ($x_{t+1} = x_t - \eta\nabla F\,|_{x_t}$) on this objective starting from $x_0 = 0$ and using a step-length of $\eta > 0$. Then which of the following is x_3?

   a) 0
   b) η
   c) $\eta(1 + e^{-\eta})$
   d) Undecidable

9 Gradient Descent in *Overparameterized* Linear Regression

Outline: In this section we will see how to understand the progress of gradient descent in this deceptively simple setup of doing linear regression via this algorithm. We will realize that understanding this case already needs some non-trivial mathematical thinking. In this setup we will see an explicit example of a scenario of the loss function having multiple global minima and hence the question arises as to which one of them will the algorithm find. Eventually we will realize that in this case, because of the number of trainable parameters being larger than the number of training data, makes the algorithm converge to that global minima of the linear regression loss that is nearest to the origin. This is surprising because neither in the loss function nor in the algorithm was there anything that was explicitly promoting this choice. Thus we will see our first example of how overparameterization interacts with gradient based algorithms to induce an effect that is often called "implicit bias" or "algorithmic regularization".

Consider being given n data points in d dimensions, stacked as a $n \times d$ matrix called $\boldsymbol{X}$ - whose (i,j)–entry would be denoted as X_{ij}. Let the corresponding "labels" be given as a column vector $\boldsymbol{y} \in \mathbb{R}^{n\times 1}$ - whose i^{th}–entry would be denoted as y_i. Now we aim to understand the "linear regression" optimization question which can be stated as,

$$\min_{\boldsymbol{\beta}\in\mathbb{R}^d} \frac{1}{2}\left\| \underbrace{\boldsymbol{y}}_{n\times 1} - \underbrace{\boldsymbol{X}}_{n\times d}\underbrace{\boldsymbol{\beta}}_{d\times 1}\right\|_2^2$$

This motivates us to define the empirical loss function of linear regression, $\hat{R}$ as,

$$\hat{R}(\boldsymbol{\beta}) = \frac{1}{2}\cdot\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2 \tag{86}$$

Notice that, foreshadowing the upcoming analysis, in above we write $\hat{R}$ as being a function of the weight parameters of the linear predictor $\boldsymbol{\beta}$ rather than emphasizing its dependence on the training data. We assume two things about this setup - and what would make this benign looking question a lot more interesting than it might seem a priori,

- We assume that the model is "overparameterized" i.e we are in the regime when n (no. of training data) $< d$ (the number of training parameters).
- The data matrix $\boldsymbol{X}$ is full rank.

To appreciate why the above setup is special, let us consider a special case of the above at $n = 1, d = 2$ i.e having one training data in two dimensions, say $\boldsymbol{x} = (1,2)$ with label $y = 4$. Then the corresponding empirical risk as a function of the weight vector $\boldsymbol{\beta} = (\beta_1, \beta_2)$ is,

$$\hat{R}(\boldsymbol{\beta}) = \frac{1}{2}\cdot(4 - \beta_1 - 2\beta_2)^2 \tag{87}$$

Note that we can immediately read off from above a global minimum of the training error at $\boldsymbol{\beta}_* = (2,1)$. (Check that we have $\hat{R}(\beta_*) = 0$ and hence this weight qualifies to be a global minima of this risk.). Now choose a set of vectors parameterized by real numbers λ, $\boldsymbol{\theta}(\lambda) = \lambda(2,-1)$. Note that, (a) all these vectors $\boldsymbol{\theta}(\lambda)$ are orthogonal to the data $\boldsymbol{x}$ i.e $\langle\boldsymbol{\theta}(\lambda), \boldsymbol{x}\rangle = 0$ and (b) all vectors of the form $\boldsymbol{\beta}_* + \boldsymbol{\theta}(\lambda)$ are at the global minima of the linear regression risk considered above, i.e in equation 87, $\hat{R}(\boldsymbol{\beta}_* + \boldsymbol{\theta}(\lambda)) = 0$.

Thus we realize that this simple example of linear regression, that we cooked up in equation 87 has an uncountably infinite number of global minima, all of which lie on this line $\boldsymbol{\beta}_* + \boldsymbol{\theta}(\lambda) = (2 + 2\lambda, 1 - \lambda)$.

It might help intuition to plot the function $\hat{R}(\boldsymbol{\beta})$ and spot the line in the $\boldsymbol{\beta}$–plane along which it is at 0.

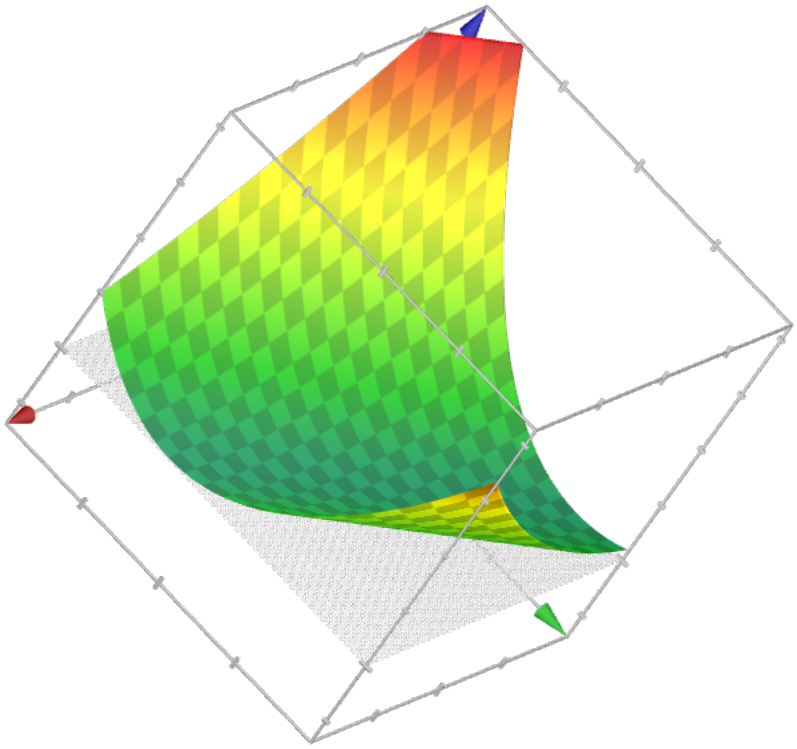


Figure 29: This is a plot of the full-rank overparameterized empirical risk function $\hat{R}(\boldsymbol{\beta})$ instantiated in equation 87. We only show the function restricted to the square $[-10, 10]^2$ in the $\boldsymbol{\beta}$–plane. The red marked axis jutting out on the left is for β_1 and the green marked axis on the right is for β_2.

Concept Check 9.1. The reader is encouraged to create more examples as given in equation 87 and create the corresponding plots as in Figure 29. An easy way to generate such plots is to type in "plot <the function>" into Google — remembering to rename β_1 as x and β_2 as y.

To develop a general understanding of the above phenomenon (and its subtle algorithmic consequences!) we need to get through a crucial mathematical fact that lies at the core of why our eventual analysis works out.

Claim 2. *If $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ is of rank n and $n < d$ then $\boldsymbol{X}\boldsymbol{X}^\top \in \mathbb{R}^{n \times n}$ is invertible.*

We will skip the proof of the above. The reader is encouraged to look at specific examples of $\boldsymbol{X}$ and get an intuition for this. For eg. check the above for an example with $n = 2, d = 3$, like, $\boldsymbol{X} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$

Given the above claim, we can conclude that at $\boldsymbol{\beta} = \boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{X}^\top)^{-1}\boldsymbol{y}$ our loss function $\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2$ is at its global minima i.e 0.

We denote,

$$\boldsymbol{X}^\dagger \coloneqq \boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{X}^\top)^{-1} \in \mathbb{R}^{d \times n}$$

and note that $\boldsymbol{X}\boldsymbol{X}^\dagger = \boldsymbol{I}_{n \times n}$. This leads to the terminology of calling $\boldsymbol{X}^\dagger$ as the "pseudo-inverse" of the data matrix $\boldsymbol{X}$. But is this $\boldsymbol{\beta}$ the only global minima of $\hat{R}$? No! And in the next claim we shall realize how there is a multiplicity of global minima.

Claim 3. *Let $\boldsymbol{x}_i^\top \in \mathbb{R}^{1\times d}$ be the rows of the data matrix $\boldsymbol{X}$. Given $\boldsymbol{y}$ and $\boldsymbol{X}$ as above, $\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2 = 0$ iff $\boldsymbol{\beta} = \boldsymbol{X}^\dagger \boldsymbol{y} + \boldsymbol{\xi}$ where $\boldsymbol{\xi}$ is s.t $\boldsymbol{\xi}^\top \boldsymbol{x}_i = 0$ for all i.*

You will not be not expected to reproduce any proof in the final MCQ exam.
However, answering certain questions would be easier,
if you understood the various techniques used in the proof.

Proof. Given $\boldsymbol{\beta}$ as in the above claim we can easily note that,

$$\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\| = \|\boldsymbol{y} - \boldsymbol{y} - \boldsymbol{X}\boldsymbol{\xi}\| = \|\boldsymbol{X}\boldsymbol{\xi}\| = 0$$

The last equality follows since it has been assumed that every row of $\boldsymbol{X}$ is orthogonal to $\boldsymbol{\xi}$.

Conversely, suppose being given a $\boldsymbol{\beta}$ s.t $\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\| = 0$. Then we can define $\boldsymbol{\eta} \coloneqq \boldsymbol{\beta} - \boldsymbol{X}^\dagger \boldsymbol{y}$ and repeat the same calculation as above to get that $\|\boldsymbol{X}\boldsymbol{\eta}\| = 0$. Since the zero vector is the only vector with a zero norm it follows that every coordinate of the vector $\boldsymbol{X}\boldsymbol{\eta}$ must be 0 and hence $\boldsymbol{\eta}$ is orthogonal to every row of the data matrix. This $\boldsymbol{\eta}$ satisfies all the conditions to be the $\boldsymbol{\xi}$ given in the claim statement. □

Recall that in the overparameterized regime we have, $n < d$. Hence the set of vectors orthogonal to n rows of the data matrix form a "vector space" inside $\mathbb{R}^d$ or more specifically a "vector subspace" of $\mathbb{R}^d$, the data space.

Concept Check 9.2. Suppose $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n$ are n vectors in $\mathbb{R}^d$. Let $\mathcal{S} \subset \mathbb{R}^d$ be the set of all vectors in $\mathbb{R}^d$ that are orthogonal to all the $\boldsymbol{v}_i$, $i = 1, \ldots, n$. Then prove that $\mathcal{S}$, when not an empty set, is a vector space (or a "vector subspace" of $\mathbb{R}^d$) i.e it satisfies the following properties,

- **(closure under addition)** For all $\boldsymbol{p}, \boldsymbol{q} \in \mathcal{S}$, the sum $\boldsymbol{p} + \boldsymbol{q} \in \mathcal{S}$.
- **(existence of additive identity)** There exists a vector $0 \in \mathcal{S}$ such that $\boldsymbol{p} + 0 = \boldsymbol{p}$ for all $\boldsymbol{p} \in \mathcal{S}$.
- **(existence of additive inverses)** For every $\boldsymbol{p} \in \mathcal{S}$ there exists $\boldsymbol{q}(= -\boldsymbol{p}) \in \mathcal{S}$ such that $\boldsymbol{p} + \boldsymbol{q} = 0$.
- **(closure under scalar multiplication)** $\forall\ \alpha \in \mathbb{R}$ and $\boldsymbol{p} \in \mathcal{S}$, the product $\alpha\boldsymbol{p} \in \mathcal{S}$.

Note that (a) in here we allow for the possibility of $n \geq d$, and that in turn allows for the possibility that we might have $\mathrm{Span}(\{\boldsymbol{v}_i \mid i = 1, \ldots, n\}) = \mathbb{R}^d$ and convince yourselves that this would lead to $\mathcal{S}$ being an empty set – which is the edge-case that had to be separately taken out in the above discussion. And (b) apart from this possibility with the span, no other property of the set of vectors $\boldsymbol{v}_i, i = 1, \ldots, n$ is relevant to the fact that $\mathcal{S}$ is a vector space.

Let's call this subspace, "$\mathrm{Orth}(\mathrm{Rows}(\boldsymbol{X}))$". Thus we have discovered that there are uncountably infinite global minima of our objective $\hat{R}$ which can be described as $\boldsymbol{X}^\dagger \boldsymbol{y} + \mathrm{Orth}(\mathrm{Rows}(\boldsymbol{X}))$ — which is an instance of an "affine space" i.e a translate of a vector space.

But the solution $\boldsymbol{X}^\dagger \boldsymbol{y}$ is still special and that is because it is the global minima of our loss with the least norm — i.e the solution of the linear regression problem that lies closest to the origin in the parameter space. And now we shall prove that fact.

Claim 4.

$$\boldsymbol{X}^\dagger \boldsymbol{y} = \underset{\boldsymbol{\beta} \mid \hat{R}(\boldsymbol{\beta}) = 0}{\mathrm{argmin}}\ \|\boldsymbol{\beta}\|_2$$

You will not be not expected to reproduce any proof in the final MCQ exam.
However, answering certain questions would be easier,
if you understood the various techniques used in the proof.

Proof. From the previous claim we know that if any $\boldsymbol{\beta}$ is s.t $\hat{R}(\boldsymbol{\beta}) = 0$ then we can write it as $\boldsymbol{\beta} = \boldsymbol{X}^{\dagger}\boldsymbol{y} + \boldsymbol{\xi}$ s.t $\boldsymbol{\xi}$ is orthogonal to each of the rows of $\boldsymbol{X}$. Now we can compute the norm of such a solution to get,

$$\begin{aligned}\|\boldsymbol{\beta}\|^2 &= \left\|\boldsymbol{X}^{\dagger}\boldsymbol{y}\right\|^2 + \|\boldsymbol{\xi}\|^2 + 2\left\langle \boldsymbol{X}^{\dagger}\boldsymbol{y}, \boldsymbol{\xi}\right\rangle \\ &= \left\|\boldsymbol{X}^{\dagger}\boldsymbol{y}\right\|^2 + \|\boldsymbol{\xi}\|^2 + 2\left\langle \boldsymbol{X}^{\top}(\boldsymbol{X}\boldsymbol{X}^{\top})^{-1}\boldsymbol{y}, \boldsymbol{\xi}\right\rangle \\ &= \left\|\boldsymbol{X}^{\dagger}\boldsymbol{y}\right\|^2 + \|\boldsymbol{\xi}\|^2 + 2\left\langle (\boldsymbol{X}\boldsymbol{X}^{\top})^{-1}\boldsymbol{y}, \boldsymbol{X}\boldsymbol{\xi}\right\rangle\end{aligned} \tag{88}$$

We can set the last term above to 0 since $\boldsymbol{X}\boldsymbol{\xi} = 0$ by our previous proof about $\boldsymbol{\xi}$ being orthogonal to the rows of the data matrix. Thus we get,

$$\|\boldsymbol{\beta}\|^2 = \left\|\boldsymbol{X}^{\dagger}\boldsymbol{y}\right\|^2 + \|\boldsymbol{\xi}\|^2 \geq \left\|\boldsymbol{X}^{\dagger}\boldsymbol{y}\right\|_2^2 \tag{89}$$

In the last step above its clear that the equality holds only when $\boldsymbol{\xi} = 0$ and thus we have proven that the 2−norm of all global minimizers of our linear regression loss $\hat{R}$ is bigger than that of $\boldsymbol{X}^{\dagger}\boldsymbol{y}$. □

Concept Check 9.3. Compute the value of this special global minimum $\boldsymbol{X}^{\dagger}\boldsymbol{y}$ for the example shown at the beginning of this lecture.

Now lets consider doing Gradient Descent (G.D.) on $\hat{R}(\boldsymbol{\beta}) = \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2$ starting from an initial weight β_0 and using a constant step-length η. Stated precisely we consider the following algorithm,

Algorithm 3 Linear Regression on the Loss $\hat{R}$ (Equation 86) By Constant Step-Size G.D.

1: **Input:** A starting point for the algorithm, $\boldsymbol{\beta}_0 \in \mathbb{R}^d$
2: **Input:** A step-size $\eta > 0$.
3: **for** $t = 1, \ldots$ **do**
4: $\quad \boldsymbol{\beta}_t = \boldsymbol{\beta}_{t-1} - \eta\nabla\hat{R}(\boldsymbol{\beta}_{t-1})$
5: **end for**

We note the following preliminary observation,

Lemma 9.1. *If the data matrix $\boldsymbol{X}$ is full rank, $\exists$ a continuum of choices of η s.t Algorithm 3 converges to the global minimizer of $\hat{R}$.*

The above follows from the fact that $\hat{R}$ has global minima (as shown in Claim 3), is convex and is Lipschitz smooth and hence one can invoke Theorem 3.4 in these notes, arXiv:2301.11235. Recall that in Theorem 8.1, we have seen an explicit special case of this more general result - but note that in the more general scenario of Theorem 3.4 in arXiv:2301.11235 one isn't able to get as fast a rate of convergence as was obtained in Theorem 8.1. *A more subtle fact is that one can prove that these speeds of convergence are essentially non-improvable.*

Optional material (non-assessed)

- Try reading the proof of Theorem 3.4 in arXiv:2301.11235.

Concept Check 9.4. Prove that our loss function $\hat{R}$ under consideration here satisfies the conditions for the above referenced Theorem 3.4 in these notes to be invoked on it.

But we have seen that our assumptions on the data matrix being full rank and overparameterized leads us to conclude that $\hat{R}$ has multiple global minima. Hence naturally a question arises as to which one of them is found by the above algorithm. For this very controlled setting we answer this question in the following theorem - our main result in this section,

Theorem 9.2. [**(Main Result)Full-Rank Overparameterized Linear Regression by G.D. Can Be Setup s.t It Finds The Minimum** 2**–Norm Interpolant**] *If the data matrix* $\boldsymbol{X}$ *is full rank and* $\boldsymbol{\beta}_0 = 0$ *then* $\exists$ *a continuum of choices of* η *s.t Algorithm 3 implementing constant step-length G.D. on the overparameterized linear regression loss* $\hat{R}$ *(Equation 86), converges to the global minimizer with the minimum* 2*–norm i.e* $\boldsymbol{X}^{\dagger}\boldsymbol{y}$.

You will not be not expected to reproduce any proof in the final MCQ exam.
However, answering certain questions would be easier,
if you understood the various techniques used in the proof.

Proof of Theorem 9.2. Recall that we have denoted $\mathbf{x}_1^{\top}, \ldots, \mathbf{x}_n^{\top}$ as the rows of the data matrix $\boldsymbol{X}$ and each $\mathbf{x}_i \in \mathbb{R}^{d\times 1}$. Then, via induction, we shall first try to prove that any of the iterates of G.D. as given in Algorithm 3 is a linear combination of the data.

To start the induction, note that trivially we have, $\boldsymbol{\beta}_0 = 0 \in \text{Span}(\mathbf{x}_1, \ldots, \mathbf{x}_n)$. Next, suppose that, $\boldsymbol{\beta}_{t-1} \in \text{Span}(\mathbf{x}_1, \ldots, \mathbf{x}_n)$. Now recall that we had,

$$\hat{R}(\boldsymbol{\beta}) = \frac{1}{2} \cdot \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2 = \frac{1}{2} \cdot \sum_{i=1}^{n} (y_i - (\boldsymbol{X}\boldsymbol{\beta})_i)^2 = \frac{1}{2} \cdot \sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{d} X_{ij}\boldsymbol{\beta}_j \right)^2$$

So, we get, $\frac{\partial \hat{R}}{\partial \beta_j} = -\sum_{i=1}^{n} (y_i - \sum_{k=1}^{d} X_{ik}\boldsymbol{\beta}_k) \cdot X_{ij}$. Stacking these into a column vector of d dimensions we have the derivative of the loss function w.r.t the weight vector being writable succinctly as,

$$\frac{\partial \hat{R}}{\partial \boldsymbol{\beta}} := \begin{bmatrix} \frac{\partial \hat{R}}{\partial \beta_1} \\ \vdots \\ \frac{\partial \hat{R}}{\partial \beta_d} \end{bmatrix} = -\boldsymbol{X}^{\top}(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})$$

Recall that the ith–column of $\boldsymbol{X}^{\top}$ is the vector $\mathbf{x}_i$. So the matrix multiplication that is done in the above expression of $\frac{\partial \hat{R}}{\partial \boldsymbol{\beta}}$ can be re-written as the following weighted sum of vectors,

$$\frac{\partial \hat{R}}{\partial \boldsymbol{\beta}} = \sum_{i=1}^{n} (-y_i + (\boldsymbol{X}\boldsymbol{\beta})_i) \cdot \mathbf{x}_i$$

Hence specific to $\boldsymbol{\beta}_{t-1}$, we can conclude from above that, $\frac{\partial \hat{R}}{\partial \boldsymbol{\beta}_{t-1}} \in \text{Span}(\mathbf{x}_1, \ldots, \mathbf{x}_n)$. Further, from the update rule being, $\boldsymbol{\beta}_t = \boldsymbol{\beta}_{t-1} - \eta \nabla \hat{R}(\boldsymbol{\beta}_{t-1})$ and recalling the induction hypothesis we can conclude what we set out to prove, that,

$$\boldsymbol{\beta}_t \in \text{Span}(\mathbf{x}_1, \ldots, \mathbf{x}_n), \ \forall t = 1, 2, \ldots$$

Thus we have demonstrated that the iterates of G.D. in our linear regression setup remain restricted to the linear span of the training data.

Note that for any $\hat{\boldsymbol{\beta}} \in \text{Span}(\mathbf{x}_1, \ldots, \mathbf{x}_n)$ it follows from definition that $\exists\ \boldsymbol{v} \in \mathbb{R}^n$ s.t $\hat{\boldsymbol{\beta}} = \boldsymbol{X}^\top \boldsymbol{v}$. From Claim 2 we know that our assumptions on the data imply that $\boldsymbol{X}\boldsymbol{X}^\top$ is an invertible matrix. Hence we can choose $\boldsymbol{v} = (\boldsymbol{X}\boldsymbol{X}^\top)^{-1}\boldsymbol{y}$ and for the corresponding $\boldsymbol{\beta}$ we have, $\hat{\boldsymbol{\beta}} = \boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{X}^\top)^{-1}\boldsymbol{y} = \boldsymbol{X}^\dagger \boldsymbol{y} \implies \hat{R}(\hat{\boldsymbol{\beta}}) = 0$. Note that this is the same weight vector that was shown to be the minimum norm interpolant in Claim 4.

Thus the above paragraph shows the existence of a $\hat{\boldsymbol{\beta}} \in \text{Span}(\mathbf{x}_1, \ldots, \mathbf{x}_n)$ s.t $\hat{R}(\hat{\boldsymbol{\beta}}) = 0$. Now towards showing its uniqueness, suppose we have a $\hat{\boldsymbol{\beta}} \in \text{Span}(\mathbf{x}_1, \ldots, \mathbf{x}_n)$ (i.e $\exists\ \boldsymbol{v} \in \mathbb{R}^n$ s.t $\hat{\boldsymbol{\beta}} = \boldsymbol{X}^\top \boldsymbol{v}$) s.t $\hat{R}(\hat{\boldsymbol{\beta}}) = 0$. Then it would follow that $\left\|\boldsymbol{y} - \boldsymbol{X}\hat{\boldsymbol{\beta}}\right\| = 0$ which is equivalent to having $\boldsymbol{y} = \boldsymbol{X}\hat{\boldsymbol{\beta}} = \boldsymbol{X}\boldsymbol{X}^\top \boldsymbol{v}$. It is easy to see that using the fact proven earlier that $\boldsymbol{X}\boldsymbol{X}^\top$ is an invertible matrix, it follows that this equation can be solved to get its unique solution, $\boldsymbol{v} = (\boldsymbol{X}\boldsymbol{X}^\top)^{-1}\boldsymbol{y}$.

Thus we have established that there exists a unique $\hat{\boldsymbol{\beta}}$ in the span of the data such that $\hat{R}(\hat{\boldsymbol{\beta}}) = 0$ and since 0 is the lower bound on $\mathcal{L}$ *it follows that our linear regression loss has a unique global minima in the span of its data*. Since G.D. is confined to be in this span and from Claim 9.1 we know that for a choice of step-lengths G.D. converges to its global minima, it follows that this convergence is to the minimum norm solution $\hat{\boldsymbol{\beta}}$.

□

Thus we have seen an explicit example of how having a large model size compared to the number of data leads to a multiplicity of global minima for the loss function and a surprising selectivity to creep into the G.D. algorithm as to which global minima it will converge to. This is only the tip of the iceberg of this phenomenon which gets called "implicit bias" or "algorithmic regularization". Such analysis gets extremely more involved for anything of practical interest. Almost everything remains unknown in this regard for neural nets - where this phenomenon is most pertinent and vividly visible.

10 A Measure of Capacity : Rademacher Complexity

Outline: In these last two lectures we return to the core question of machine learning of being able to theoretically bound the population risk in a given ML scenario. This lecture will involve going through a lot of setting-up with notation and formalism as we realize that naive ways of framing this target of bounding the true risk may make the question essentially unanswerable. Eventually, we will get to the somewhat sophisticated idea of "Rademacher Complexity" which we will see is a potent way to measure out-of-sample performance of a ML setup in a way that does not tie itself to any specific choice of a training algorithm. And this sets the stage for the last lecture where we will see explicit examples of how Rademacher complexity helps us understand in a precise quantitative way that the capacity of learning may not be tied to the size of the model.

Recall that in Section 1 you have seen the notions of empirical risk in Definition 4 and population risk in Definition 3. We had defined $f : \mathbb{R}^d \to \mathbb{R}^k$ as the predictor (the model at hand, say a neural net) and the loss function $\ell : \mathbb{R}^k \times \mathbb{R}^k \to [0, \infty)$ and the data $(\boldsymbol{x}, \boldsymbol{y}) \in \mathbb{R}^d \times \mathbb{R}^k$ sampled from a data distribution (say $\mathcal{D}$) on $\mathbb{R}^d \times \mathbb{R}^k$. And we had defined the "population risk" of f as,

$$R(f) \coloneqq \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})} \left[\ell(\boldsymbol{y}, f(\boldsymbol{x}))\right] \tag{90}$$

It is easy to see from the above, that the above expression captures the idea of the error in prediction the model f makes, on an average over all data that it can be expected to face. Note that the above quantity does not depend on the choice of training data, even if the f being used is instantiated as the output of a training algorithm which consumed some particular training data. Further, given a set of training data $\mathcal{S}_n = \{(\boldsymbol{x}_i, y_i), i = 1, \ldots, n\}$, each data being sampled from the distribution $\mathcal{D}$ specified above, we had denoted the empirical risk of f as,

$$\hat{R}(f, \mathcal{S}_n) \coloneqq \frac{1}{n} \sum_{i=1}^{n} \left[\ell(\boldsymbol{y}_i, f(\boldsymbol{x}_i))\right] \tag{91}$$

We note that the risk definitions depend on three key components, (a) the sampled data for the empirical risk and the data distribution for the population risk, (b) the loss function and (c) the predictor being used.

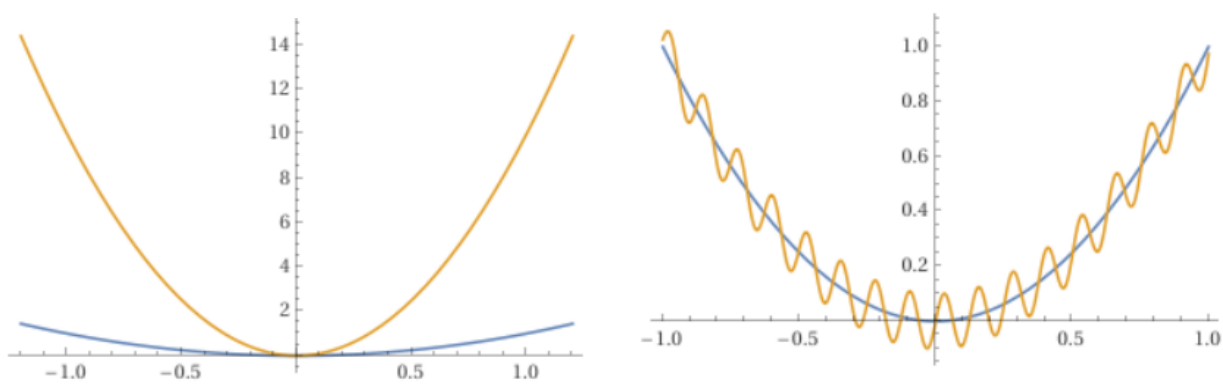


Figure 30: In the diagrams above we imagine the x-axis to be the possible parameter values of the parameterized predictor – say the weights of a neural net. For a fixed choice of training samples, the yellow curve is a cartoon for the values of the empirical risk for the predictors at the corresponding parameter values. The blue curve is a depiction of the values of the population risk for the same parameter values.

In Figure 30 we can see a cartoon depiction of two (of the many possible!) relationships that might exist between empirical and population risk for a class of predictors - for a fixed training dataset $\mathcal{S}_n$. These two

diagrams in the figure are representative of two extreme scenarios that can happen. In the left, we see the empirical risk (yellow curve) to always be greater (or equal) to the value of the population risk (blue curve) for all predictors and for weights further away from zero, the two risks are only getting unboundedly away from each other. In no meaningful sense, can these two risks be said to be close to each other in this scenario. In the right, we see a situation where the empirical risk is always very close to the population risk, though sometimes its larger and sometimes smaller than it.

Experience suggests that most often in the real world the situation is like the right schematic of Figure 30 — while proving such a phenomenon in any generality is exceptionally challenging, the techniques to be taught in these last two lectures are often a step towards obtaining such an understanding. *But note that in both these otherwise very different situations, the global minima of the empirical risk is very close to that of the population risk — i.e the population risk of f_{erm} and f_* from Definitions 12 and 13 are very close. It is fair to say that this luck is at play with the big models — and why in such modern ML setups approximately minimizing the empirical risk seems to make "learning" happen.*

This fortuitous situation described above is incredibly difficult to predict a priori. Motivated by the schematic in Figure 30 we summarize the salient challenges here in three parts. (A) Given a specific class of predictors, loss, data distribution and a sample from it, it is extremely difficult to figure out if the empirical and population risk curves are close for each predictor —- or if even the global minimas of the two risks, f_{erm} and f_* are close by predictors/have close by weights. (B) Its a further hard challenge to figure out if the proximity between the two risks holds or not with high-probability over sampling the training data — recall that the shape of the empirical risk curve (the yellow line in Figure 30) depends on the specifics of the training data chosen.

(C) Its natural to expect that with increasing number of training data being used, with high probability over sampling them, these two risk curves will eventually coincide in the large sample size limit. But such a large n convergence of $\hat{R}(f, \mathcal{S}_n)$ to $R(f)$ *for all* $f \in \mathcal{F}$ *simultaneously* cannot be taken for granted and establishing/proving this is quite a technical quest to undertake. Moreover, such a convergence of the risks needs non-trivial properties to be true about the loss and the data and the predictors.

Thus we realize that embarking to study the relationship between empirical and population risks quickly lands us to be surrounded with nearly-impossible challenges. Hence we see the need to reframe the questions we want ask so as to have any hope at all of developing an understanding of machine learning.

In this section, we will delineate an approach to study the relationship between empirical and population risks when the predictors are being chosen from a specific function class $\mathcal{F}$ – and *our focus will be on understanding an averaged sense in which the proximity between the two risks can be inferred.* Given any loss function, there is a very subtle interplay between n, $\mathcal{D}$ and $\mathcal{F}$ which we shall begin to slowly uncover here - in particular, we will see how the "size of the predictors" i.e number of parameters in the specification of a predictor (like the number of weights in a neural net) *may not* directly affect this relationship!

Note that in describing the challenges so far we have not considered any learning algorithm being deployed (to find f_*). Next we will outline why understanding the risk of the output of any learning algorithm is way too complicated - and why that path is not advisable in general. This gives a further concrete motivation to pursue the focus as emphasized above.

Challenges with Understanding the Risk of the Output of a Stochastic Algorithm We realize that given training data $\mathcal{S}_n$, our usual machine learning algorithms use these training data to help search through a set of f and find one that minimizes the empirical risk, equation 91. Usually in M.L., the set of f accessible to a algorithm is a parametric set of functions — like the set of neural nets at different weights on a fixed architecture. Thus, the algorithm effectively searches through the set of f by searching through the space of parameters of it - like the space of weights of a ne. *The reader is urged to carefully note that the space of all possible f that an algorithm is effectively searching through can be infinite dimensional space (and is commonly so) while the space of its parameters is not.*

Let us recall that we had defined $\mathcal{F}$ as the set of all predictor functions f. More specifically let us define,

$$\begin{aligned}\mathcal{F} \coloneqq & \text{ the set of all possible models(predictors) mapping } \mathbb{R}^d \to \mathbb{R}^k, \\ & \text{that a given training algorithm is searching over.}\end{aligned} \tag{92}$$

Given a fixed neural architecture, the set of all possible neural functions that can be obtained from every possible weight assignment to its edges, is an example of the set $\mathcal{F}$ that is associated to a neural training situation for a fixed architecture.

Further, we define ALG to be any chosen training algorithm which takes as input the 3–tuple $(\ell, \mathcal{S}_n, \mathcal{F})$ and searches through the above $\mathcal{F}$ for the "best" predictor - as defined in Definition 13. Thus at this point we have met all the main characters of the play i.e ALG, ℓ, $\mathcal{S}_n, \mathcal{D}$ and $\mathcal{F}$.

Ideally we would want to find conditions such that on-an-average the algorithmically obtained model has low errors on *all possible data*. That is what is captured by wanting that $R(\text{ALG}(\ell, \mathcal{S}_n, \mathcal{F}))$ - an instance of the quantity defined in equation 90 - be as low as possible - ideally it be precisely $= f_* = \inf_{f \in \mathcal{F}} R(f)$. Fact is that for almost all cases of practical interest, it is often impossible to ascertain if $\text{ALG}(\ell, \mathcal{S}_n, \mathcal{F})$ is even near f_{erm}.

Note that in computing $R(\text{ALG}(\ell, \mathcal{S}_n, \mathcal{F}))$, the expectation is not being taken over all the sources of randomness which in general can include sources of stochasticity in the training algorithm as well as the randomness in sampling $\mathcal{S}_n$ from the whole data distribution $\mathcal{D}$. Thus, we have no escape from having to live with the fact that "$\text{ALG}(\ell, \mathcal{S}_n, \mathcal{F})$" — the model output by the training algorithm — is not only random but that its distribution is way too complicated to be derivable. So we realize that there is an *almost insurmountable challenge* in being able to control the quantity that we are really after i.e $R(\text{ALG}(\ell, \mathcal{S}_n, \mathcal{F}))$.

One might want to define, a notion of "cumulative risk at the population/empirical risk infimum" as, $\mathbb{E}_{\mathcal{S}_n \sim \mathcal{D}}\left[R(f_{erm})\right]$ and $\mathbb{E}_{\mathcal{S}_n \sim \mathcal{D}}\left[\hat{R}(f_{erm}, \mathcal{S}_n)\right]$ respectively. It's easy to see how to generalize this to account for all sources of randomness in the algorithm by replacing f_{erm} by $\text{ALG}(\ell, \mathcal{S}_n, \mathcal{F})$ and taking a further expectation over the sources of randomness in the algorithm. In these lectures we will not explore this idea.

10.1 Expectation Over Data of the Worst Generalization Gap & Rademacher Complexity

The deeply fundamental idea that takes us beyond the challenges described above will be to relax our goals to wanting to control *only* the worst possible difference between population and empirical risk, in expectation over all possible training data $\mathcal{S}_n$.

Stated precisely, in expecatation over training sets $\mathcal{S}_n$ the worst (over all possible predictors) "generalization gap" between the population and empirical risk is,

$$\mathbb{E}_{\mathcal{S}_n}\left[\sup_{f \in \mathcal{F}}\left(\hat{R}(f, \mathcal{S}_n) - R(f)\right)\right] \tag{93}$$

It is to be noted that trying to control the above frees us from the details of any particular M.L. experiment that is trying to find $\text{arginf}_{f \in \mathcal{F}} R(f)$. In particular, by trying to understand the above quantity we shall become blind to the details of any algorithm and data being used in any experiment that is trying to find $\text{arginf}_{f \in \mathcal{F}} R(f)$. But this is the price we must pay to gain any insight at all into how the choice of $\mathcal{F}$ and $\mathcal{D}$ together influence our ability to access the population risk by computing the empirical risk — particularly when $\mathcal{F}$ is very complicated, like a set of deep neural nets!

We crucially note that *any condition we find on $n, \mathcal{F}$ and $\mathcal{D}$ which makes the above quantity small is a sufficient condition for the empirical and the population risk to be proportionately close for any predictor in the class $\mathcal{F}$. Thus, for the first time, we have the possibility to be in a situation when we can access properties of the true/population risk via looking at quantities that are visible in experiment, like the data and the trained model, the algorithm's behaviour etc.*

And such a condition which ensures proximity of empirical and population risk becomes particularly interesting if it does *not* depend on the number of parameters in the function of the set $\mathcal{F}$ – assuming these are parameterized functions like neural nets. Towards understanding such deatails we need the following notion which is the central object of our study, the "Rademacher Complexity",

Definition 31 (Empirical and Average Rademacher complexity)**.** *For a scalar/real valued function class $\mathcal{H}$, suppose being given a $n-$sized data-set $\mathcal{S}_n$ of points $\{\boldsymbol{z}_i \mid i = 1, \ldots, n\}$ in the domain of the elements in $\mathcal{H}$. Let $\boldsymbol{\epsilon}$ be a $n-$dimensional random vector s.t its coordinates are sampled as $\epsilon_i \sim \pm 1$ with equal probability and independently. Then the corresponding empirical Rademacher complexity is defined as,*

$$\hat{\mathcal{R}}_n(\mathcal{H}) \coloneqq \mathbb{E}_{\boldsymbol{\epsilon}}\left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \epsilon_i h(\boldsymbol{z}_i)\right] \tag{94}$$

If the elements of $\mathcal{S}_n$ above are sampled independently and identically from a common distribution $\mathcal{D}$ then we shall denote this using the notation $\mathcal{S}_n \sim \mathcal{D}^n$. Correspondingly the "average Rademacher complexity" is defined as,

$$\mathcal{R}_n(\mathcal{H}) \coloneqq \mathbb{E}_{\mathcal{S}_n}\left(\mathbb{E}_{\boldsymbol{\epsilon}}\left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \epsilon_i h(\boldsymbol{z}_i)\right]\right) \tag{95}$$

Remark. *Note that in above, for the first time we have formally defined the idea of a set of n randomly sampled data, $\mathcal{S}_n$. Recalling the data-label pairs $(\boldsymbol{x}, \boldsymbol{y})$ from the beginning of the section we note that in the supervised setting we would typically invoke the above definition for $\boldsymbol{z} = (\boldsymbol{x}, \boldsymbol{y})$ and $h(\boldsymbol{z}) = \ell(\boldsymbol{y}, f(\boldsymbol{x}))$. In such cases we would have the following identification between the random variables of interest to us,*

$$\mathcal{S}_n = \{\boldsymbol{z}_1, \ldots, \boldsymbol{z}_n\} = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_n, \boldsymbol{y}_n)\}$$

The merits of introducing this language of $\boldsymbol{z}$ as a notation for a single random sample of supervised data would become apparent when we present and prove the next couple of crucial theorems.

When is Rademacher Complexity Small? Foreshadowing the discussion to follow in the next subsection, we are motivated to ask for conditions when the average Rademacher complexity is small. It so happens that for very many cases — of which we will soon give an example in the next subsection — $\mathcal{R}_n(\mathcal{F})$ being small ensures proximity of empirical and population risks in the sense of equation 93 being small. *For the rest of this discussion we consider being in these fortuitous situations.*

When analogous to equation 95 we compute $\mathcal{R}_n(\mathcal{F})$ then for each $f \in \mathcal{F}$ the term "$\frac{1}{n}\sum_{i=1}^{n} \epsilon_i f(\boldsymbol{z}_i)$" would occur. It follows that if $f(z_i)$ and ϵ_i are of the same sign then that tends to increase the value of Rademacher complexity and more so if it is true for most or all i. Recalling that ϵ_i are equally likely to be 1 or -1, it can seen as a model of "noise". Hence intuitively one can say that Rademacher complexity of $\mathcal{F}$ tends to be low, if for most pairs of dataset $\mathcal{S}_n$ and $n-$length noise patterns $\boldsymbol{\epsilon}$, there is no corresponding function in $\mathcal{F}$ which evaluated on $\mathcal{S}_n$ leads to outputs of the same sign sequence as $\boldsymbol{\epsilon}$.

Rademacher complexity of $\mathcal{F}$ can then informally be seen to scale with the expected noise-fitting-ability of $\mathcal{F}$ over all data sets $\mathcal{S}_n$s that could be drawn according to the distribution $\mathcal{D}$. Thus Rademacher theory motivates the philosophy that a function class's ability to represent noisy labels does not bode well for its ability to learn.

10.1.1 A Foundational Property of Rademacher Complexity

And now we have all the setup to state what is possibly the most important property about Rademacher complexity,

Theorem 10.1 (**Rademacher Complexity & the Expectation of the Worst Generalization Gap).** *Suppose $\mathcal{H}$ is a set (also called a "class") of real valued functions on a common domain and let $\mathcal{S}_n$ denote $n-$sized samples being drawn from a distribution $\mathcal{D}$ on that domain. Then we have,*

$$\mathbb{E}_{\mathcal{S}_n}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}h(\boldsymbol{z}_i)-\mathbb{E}_{\boldsymbol{z}}\left[h(\boldsymbol{z})\right]\right)\right]\leq 2\mathcal{R}_n(\mathcal{H})$$

The proof of the above theorem needs certain techniques in probability theory that the reader might not have seen earlier. The full proof is explained in detail in Section 10.1.2. In the rest of this section, we shall try to develop an understanding of how such an inequality as above reveals a whole lot of deep things about how machine learning happens.

Considering $\mathcal{H}$ to be $\ell\circ\mathcal{F}$ for some class of predictors $\mathcal{F}$, we can see that the above theorems gives a bound on the expectation (over training data) of the worst generalization error among all possible predictors – an upper bound that is dependent on the loss, the predictor class, the distribution, and the number of samples. *Being able to incorporate all these dependencies in such a succinct inequality as above is a prime reason that makes this such an attractive approach to understanding machine learning.*

Note that we can replace $\mathcal{H}$ by the set of functions $-\mathcal{H}=\{-h\mid h\in\mathcal{H}\}$ i.e negations of the functions in the class $\mathcal{H}$, and for this class invoke the above theorem to get the following,

$$\mathbb{E}_{\mathcal{S}_n}\left[\sup_{-h\in-\mathcal{H}}\left(\mathbb{E}_{\boldsymbol{z}}\left[h(\boldsymbol{z})\right]-\frac{1}{n}\sum_{i=1}^{n}h(\boldsymbol{z}_i)\right)\right]\leq 2\mathcal{R}_n(-\mathcal{H})=2\mathcal{R}_n(\mathcal{H})$$

The last equality above is a property of Rademacher complexity that the reader is urged to prove for themselves. Further, we observe that the inner "$\sup_{-h\in-\mathcal{H}}$" can be replaced by "$\sup_{h\in\mathcal{H}}$" with no change in the value of the LHS. Thus we have,

$$\mathbb{E}_{\mathcal{S}_n}\left[\sup_{h\in\mathcal{H}}\left(\mathbb{E}_{\boldsymbol{z}}\left[h(\boldsymbol{z})\right]-\frac{1}{n}\sum_{i=1}^{n}h(\boldsymbol{z}_i)\right)\right]\leq 2\mathcal{R}_n(\mathcal{H}) \tag{96}$$

Although the above interpretations needed invoking the given theorem for $\mathcal{H}=\ell\circ\mathcal{F}$, mathematically it is typically easier to directly control the Rademacher complexity of the predictor class $\mathcal{F}$. Relating the Rademacher complexity of the class $\ell\circ\mathcal{F}$ and $\mathcal{F}$ is a somewhat subtle issue. In the examples class this week, a simple example would be demonstrated where $\mathcal{R}_n(\mathcal{F})$ and $\mathcal{R}_n(\mathcal{H})$ are equal upto a constant. That example would use the following setup - suppose one is doing binary classification and any arbitrary set of n training data be given as $\boldsymbol{z}_i=(\boldsymbol{x}_i,y_i)$ with $y_i=\pm1$ for $i=1,\ldots,n$. Then for any $f\in\mathcal{F}$ we consider the following loss on data $(\boldsymbol{x},y)$,

$$\ell_{0-1}((\boldsymbol{x},y),f)=\mathbf{1}_{y\neq f(\boldsymbol{x})}=\frac{1-yf(\boldsymbol{x})}{2}$$

Note that we shall restrict our analysis to the case when every $f\in\mathcal{F}$ is ±1 valued (could be neural nets composed with a ±1-valued threshold gate) – as this is the class where the set of possible output values of the predictor matches the set of possible values of the true labels and hence above loss function is sensible. Thus given the function class of $\mathcal{F}$, we have corresponding to it a loss class $\mathcal{H}$ whose elements could be imagined to be indexed by the elements of $\mathcal{F}$ i.e,

$$\mathcal{H}\ni h_f \text{ s.t } h_f(\boldsymbol{z})\coloneqq\ell_{0-1}((\boldsymbol{x},y),f) \tag{97}$$

Thus corresponding to a data distribution the averaged Rademacher complexity for this loss class would be,

$$\mathcal{R}_n(\mathcal{H}) = \mathbb{E}_{(\boldsymbol{x}_i, y_i), \forall i=1,\ldots,n} \left(\mathbb{E}_{\boldsymbol{\epsilon}} \left[\sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} \epsilon_i \cdot \ell_{0-1}((\boldsymbol{x}_i, y_i), f) \right] \right) \tag{98}$$

And the averaged Rademacher complexity for the predictor class would be,

$$\mathcal{R}_n(\mathcal{F}) = \mathbb{E}_{(\boldsymbol{x}_i, y_i), \forall i=1,\ldots,n} \left(\mathbb{E}_{\boldsymbol{\epsilon}} \left[\sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} \epsilon_i \cdot f(\boldsymbol{x}_i) \right] \right) \tag{99}$$

Note that in above the sampled labels y_i are not being used in the computation. In the examples class, the reader would work through a proof that shows that,

$$\mathcal{R}_n(\mathcal{H}) = \frac{1}{2} \cdot \mathcal{R}_n(\mathcal{F}) \tag{100}$$

We note that such simple equality relations between the Rademacher complexities of the loss class and the predictor class are not commonplace. But one of them bounding the other upto a constant is a more generic phenomenon - and explaining the reason for that is not within the scope of this course. *Note that this is the situation alluded to at the end of the last section, about cases when $\mathcal{R}_n(\mathcal{F})$ being small ensures proximity of empirical and population risks in the sense as considered in equation 93.*

We shall often try to find a function $\mathcal{C}(\mathcal{H}, \mathcal{D})$ s.t we can write,

$$\mathcal{R}_n(\mathcal{H}) \leq \frac{\mathcal{C}(\mathcal{H}, \mathcal{D})}{\sqrt{n}} \tag{101}$$

The reader is urged to note that to the best of our knowledge there is no general principle as to when such an inequality as above is possible. It seems extremely fortuitous that so many of the natural use-cases in machine learning practice have an inequality of the above form being true!

Consequentially, from Theorem 10.1 we would have,

$$\mathbb{E}_{(\boldsymbol{x}_i, \boldsymbol{y}_i), \forall i=1,\ldots,n} \left[\sup_{h \in \mathcal{H}} \left(\hat{R}(h, \mathcal{S}_n) - R(h) \right) \right] \leq \frac{2 \cdot \mathcal{C}(\mathcal{H}, \mathcal{D})}{\sqrt{n}} \tag{102}$$

It is clear, that in the above we have used an explicit representation of the data as appropriate in the supervised setting $\boldsymbol{z} = (\boldsymbol{x}, \boldsymbol{y})$ and invoked Theorem 10.1 on $h(\boldsymbol{z}) = \ell(\boldsymbol{y}, f(\boldsymbol{x}))$ and rewritten the LHS of Theorem 10.1 in the notation of the population and the empirical risks as introduced in the beginning.

An immediate consequence of such a bound is that we can read off from the above, that by training using $n \geq \left(\frac{2 \cdot \mathcal{C}(\mathcal{H}, \mathcal{D})}{\epsilon} \right)^2$ samples we can have the (data averaged) worst/maximum generalization error be at most ϵ, for any $\epsilon > 0$ arbitrarily small. One of the immediate roles of the idea of Rademacher complexity is to be able to provide such estimates of the number of samples needed to attain a target accuracy of learning - and such estimates are often called "sample complexity".

It is to be noted that as the function class $\mathcal{F}$ (and hence also the class $\ell \circ \mathcal{F}$) get more and more complicated, our sample complexity estimates obtained via the above machinery often start to wildly overestimate the number of required training data than what we see to be needed in practice to find predictors with generalization gap below ϵ. Resolving this theory vs experiment gap is a key direction at the cutting-edge of research!

When an inequality of the form as in equation 102 is obtainable *and $\mathcal{C}$ not depend on the number of parameters in the functions in $\mathcal{H}$* then we can see how using a large number of parameters in our models may not hurt anything seen from this lens. In such examples, we are essentially led to realize that the effective "capacity" of the setup (independent of any n) i.e $\mathcal{C}$ is controlled by properties of the $\mathcal{H}$ and $\mathcal{D}$ which can be set s.t they do not scale with the number of parameters in $\mathcal{H}$. And this possibility shall be our motivation towards going forward with the program to understand the average of the worst generalization gap.

10.1.2 Proof of Theorem 10.1

You will not be not expected to reproduce any proof in the final MCQ exam.
However, answering certain questions would be easier,
if you understood the various techniques used in the proof.

Proof. The proof uses a technique called "symmetrization" - which has far-reaching uses than just this. Given a data set $\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_n$, we imagine sampling another training data set (sometimes called the "ghost sample") $\boldsymbol{z}'_1, \boldsymbol{z}'_2, \ldots, \boldsymbol{z}'_n$ s.t as earlier, each is chosen independently of the other and from the same distribution $\mathcal{D}$. Then we can implement the following re-writing,

$$\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n} h(\boldsymbol{z}_i) - \mathbb{E}_{\boldsymbol{z}}\left[h(\boldsymbol{z})\right]\right) \tag{103}$$

$$= \sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n} h(\boldsymbol{z}_i) - \mathbb{E}_{\boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}\left[\frac{1}{n}\sum_{i=1}^{n} h(\boldsymbol{z}'_i)\right]\right) \tag{104}$$

Now the key idea is that since the first term in the sup above and the second term therein depend on two different sets of data one can just take the expectation "$\mathbb{E}_{\boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}$" also over the first term without changing its value - that's now essentially a dummy operation! Thus we get,

$$\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n} h(\boldsymbol{z}_i) - \mathbb{E}_{\boldsymbol{z}}\left[h(\boldsymbol{z})\right]\right) \tag{105}$$

$$= \sup_{h\in\mathcal{H}}\left(\mathbb{E}_{\boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}\left[\frac{1}{n}\sum_{i=1}^{n} h(\boldsymbol{z}_i) - \frac{1}{n}\sum_{i=1}^{n} h(\boldsymbol{z}'_i)\right]\right) \tag{106}$$

Now we notice a general property w.r.t any two variables x and y, the later being random, and a bivariate function g,

$$\sup_x\left(\mathbb{E}_y[g(x,y)]\right) \leq \sup_x\left(\mathbb{E}_y[\sup_{x'} g(x',y)]\right) = \mathbb{E}_y[\sup_x g(x,y)]$$

In the last equality above we realize that the outermost sup in the middle term has become redundant since nothing inside it anymore depends on x. Thus we can just remove it and rename the x' as x.

We use the above observation to get,

$$\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n} h(\boldsymbol{z}_i) - \mathbb{E}_{\boldsymbol{z}}\left[h(\boldsymbol{z})\right]\right) \tag{107}$$

$$\leq \mathbb{E}_{\boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n} h(\boldsymbol{z}_i) - \frac{1}{n}\sum_{i=1}^{n} h(\boldsymbol{z}'_i)\right)\right] \tag{108}$$

Now we take expectation over the data $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_n$ to get,

$$
\begin{aligned}
&\mathbb{E}_{\boldsymbol{z}_1,\boldsymbol{z}_2,\ldots,\boldsymbol{z}_n}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}h(\boldsymbol{z}_i)-\mathbb{E}_{\boldsymbol{z}}\left[h(\boldsymbol{z})\right]\right)\right]\\
\leq&\mathbb{E}_{\boldsymbol{z}_1,\boldsymbol{z}_2,\ldots,\boldsymbol{z}_n}\left[\mathbb{E}_{\boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}(h(\boldsymbol{z}_i)-h(\boldsymbol{z}'_i))\right)\right]\right]\\
\leq&\mathbb{E}_{\substack{\boldsymbol{z}_1,\boldsymbol{z}_2,\ldots,\boldsymbol{z}_n\\ \boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}(h(\boldsymbol{z}_i)-h(\boldsymbol{z}'_i))\right)\right]
\end{aligned}
$$

We shall make another critical observation at this point. Since $\boldsymbol{z}_i$ and $\boldsymbol{z}'_i$ are sampled independently from the same distribution $\mathcal{D}$, they have the same set of possible values with equal likelihood (or densities to be more specific). Thus "$q(h)_i \coloneqq h(\boldsymbol{z}_i)-h(\boldsymbol{z}'_i)$" is a symmetric random variable i.e $q(h)_i$ and $-q(h)_i$ are equally likely - in an intuitive sense. Now suppose σ_i for $i=1,\ldots,n$ are independent random variables s.t each is 1 or -1 with equal probability. Then we have,

$$
\mathbb{E}_{\substack{\boldsymbol{z}_1,\boldsymbol{z}_2,\ldots,\boldsymbol{z}_n\\ \boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}}\left[\mathbb{E}_{\sigma_1,\ldots,\sigma_n}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}\sigma_i\cdot q(h)_i\right)\right]\right]=\frac{1}{2^n}\sum_{\sigma_1=\pm1}\cdots\sum_{\sigma_n=\pm1}\mathbb{E}_{\substack{\boldsymbol{z}_1,\boldsymbol{z}_2,\ldots,\boldsymbol{z}_n\\ \boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}\sigma_i\cdot q(h)_i\right)\right]
$$

Given that we have realized that the density at $q(h)_i$ and $-q(h)_i$ are the same we realize that the value of each of the 2^n expectations in the R.H.S. above is the same - irrespective of the values assumed by σ_is. Thus each term in the above summand is identical irrespective of the values assumed by the σ_is. Thus the introduction of these auxiliary σ random variables is redundant and we can write,

$$
\begin{aligned}
&\mathbb{E}_{\boldsymbol{z}_1,\boldsymbol{z}_2,\ldots,\boldsymbol{z}_n}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}h(\boldsymbol{z}_i)-\mathbb{E}_{\boldsymbol{z}}\left[h(\boldsymbol{z})\right]\right)\right]\\
\leq&\mathbb{E}_{\substack{\boldsymbol{z}_1,\boldsymbol{z}_2,\ldots,\boldsymbol{z}_n\\ \boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}}\left[\mathbb{E}_{\sigma_1,\ldots,\sigma_n}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}\sigma_i\cdot(h(\boldsymbol{z}_i)-h(\boldsymbol{z}'_i))\right)\right]\right]\\
\leq&\mathbb{E}_{\substack{\boldsymbol{z}_1,\boldsymbol{z}_2,\ldots,\boldsymbol{z}_n\\ \boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}}\left[\mathbb{E}_{\sigma_1,\ldots,\sigma_n}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}\sigma_i\cdot h(\boldsymbol{z}_i)\right)+\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}-\sigma_i h(\boldsymbol{z}'_i))\right)\right]\right]\\
\leq&\mathbb{E}_{\substack{\boldsymbol{z}_1,\boldsymbol{z}_2,\ldots,\boldsymbol{z}_n\\ \boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}}\left[\mathbb{E}_{\sigma_1,\ldots,\sigma_n}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}\sigma_i\cdot h(\boldsymbol{z}_i)\right)\right]\right]+\mathbb{E}_{\substack{\boldsymbol{z}_1,\boldsymbol{z}_2,\ldots,\boldsymbol{z}_n\\ \boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}}\left[\mathbb{E}_{\sigma_1,\ldots,\sigma_n}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}-\sigma_i\cdot h(\boldsymbol{z}'_i)\right)\right]\right]\\
\leq&\mathbb{E}_{\boldsymbol{z}_1,\boldsymbol{z}_2,\ldots,\boldsymbol{z}_n}\left[\mathbb{E}_{\boldsymbol{\epsilon}}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}\epsilon_i\cdot h(\boldsymbol{z}_i)\right)\right]\right]+\mathbb{E}_{\boldsymbol{z}'_1,\boldsymbol{z}'_2,\ldots,\boldsymbol{z}'_n}\left[\mathbb{E}_{\boldsymbol{\epsilon}}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n}\epsilon_i\cdot h(\boldsymbol{z}'_i)\right)\right]\right]\\
\leq&2\cdot\mathcal{R}_n(\mathcal{H})
\end{aligned}
$$

In the second inequality above we have used the sub-additivity property of the supremum. In the third inequality above we have used linearity of expectations. In the last inequality above we have evaluated the redundant expectations (to 1) – and we have renamed the σ_is to ϵ_is to match to the original variable naming used in the definition of the Rademacher complexity.

And thus the proof is done. □

11 Elementary Examples of Bounding Rademacher Complexity

Outline: In this last lecture we will see explicit examples of how Rademacher complexity helps us understand in a precise quantitative way that the capacity of learning may not be tied to the size of the model. To achieve this demonstration we will start from the simple case of computing bounds on the Rademacher complexity of a finite set of real-valued functions. Then we will move to our most important case of an (uncountably infinite) set of linear predictors with bounded weight. Both these proofs will make the reader familiar with various kinds of important techniques with probability theory - which shall continue to be important when the reader sees more advanced machine learning beyond this course.

Theorem 11.1. (Massart's Finite Class Lemma) *Let $\mathcal{H}$ be a finite set of real-valued functions over a common domain and let $|\mathcal{H}|$ be the number of functions in this set. Let $\mathcal{S}_n \coloneqq \{\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_n\}$ be a set of n samples of data in the domain of the functions in $\mathcal{H}$. For all $h \in \mathcal{H}$, we define, $h(\mathcal{S}_n)$ as the vector of evaluations of h on the data i.e the $n-$dimensional vector, $(h(\boldsymbol{z}_1), h(\boldsymbol{z}_2), \ldots, h(\boldsymbol{z}_n))$. Suppose we define a constant M s.t $\frac{1}{\sqrt{n}}\|h(\mathcal{S}_n)\|_2 \leq M \;\; \forall h \in \mathcal{H}$. Then:*

$$\hat{\mathcal{R}}_n(\mathcal{H}) \coloneqq \mathbb{E}_{\boldsymbol{\epsilon}}\left[\sup_{h\in\mathcal{H}} \frac{1}{n}\sum_{i=1}^{n} \epsilon_i h(\boldsymbol{z}_i)\right] \leq \sqrt{\frac{2M^2 \ln|\mathcal{H}|}{n}}.$$

Note that corresponding to the function class above, if we further denote as $\mathcal{D}$ as the distribution on the domain from which each of the data $\boldsymbol{z}_i$s are being sampled independently then we can take expectation over data sampling on both sides of the above inequality and realize that we have proven $\mathcal{R}_n(\mathcal{H}) \leq \sqrt{\frac{2M^2 \ln|\mathcal{H}|}{n}}$. Then we can invoke Theorem 10.1 to get,

$$\mathbb{E}_{\mathcal{S}_n}\left[\sup_{h\in\mathcal{H}}\left(\frac{1}{n}\sum_{i=1}^{n} h(\boldsymbol{z}_i) - \mathbb{E}_{\boldsymbol{z}\sim\mathcal{D}}\left[h(\boldsymbol{z})\right]\right)\right] \leq 2\mathcal{R}_n(\mathcal{H}) \leq \sqrt{\frac{8\cdot M^2 \ln|\mathcal{H}|}{n}}$$

The reader is urged to note how we have rediscovered the essential insight of equation **??** as a special case of the method of Rademacher complexity. The above is a generalization bound which holds on average over sampling of data while the bound in equation **??** was proven to hold with high probability. One can use Rademacher techniques to get such high probability guarantees too but that we shall not get into this course.

We shall go over the proof of Theorem 11.1 in the examples session this week. Currently we shall try to get a feel for this bound above using an example about doing linear regression over quantized weights.

Suppose we restrict the weight vectors of the linear predictors to be searched over to be such that each coordinate can only be an integer representable by 4 bits, inclusive of the first bit which would be 0 or 1 to indicate whether the integer is positive or negative respectively. Recall that in this "2's complement" notation the smallest integer representable by 4 bits is "1000" which evaluates to $(-1)\cdot 2^3 = -8$ and the largest integer representable by 4 bits is "0111" which evaluates to $0\cdot 2^3 + 2^2 + 2^1 + 2^0 = 7$. Thus we can represent a total of 16 integers using 4 bits and hence the space of all possible $d-$dimensional weights representable using 4 bits per coordinate is 16^d. For each such "quantized" weight "$\boldsymbol{w} \in \mathbb{R}^d$" we have one linear predictor mapping $\boldsymbol{x} \mapsto \langle \boldsymbol{w}, \boldsymbol{x}\rangle$ and hence our search space consists of 16^d linear predictors.

Further, for each such quantized weight vector he have one loss function as follows, as a candidate h for Theorem 11.1 – different functions being now distinguished by their subscript,

$$\mathbb{R}^p \ni \boldsymbol{w} \mapsto h_{\boldsymbol{w}}(\boldsymbol{x}, y) = \frac{1}{2}\cdot(y - \langle \boldsymbol{w}, \boldsymbol{x}\rangle)^2 \in \mathbb{R}$$

Thus we can see that Theorem 11.1 can be invoked on the above set of hs with $\boldsymbol{z}_i = (\boldsymbol{x}_i, y_i)$, the i indexing over the n training data on which linear regression is to be done. And, the set $\mathcal{H}$ in this instance is s.t

$|\mathcal{H}| = 16^d$. Given that the weight vectors have bounded coordinate entries and because of the finiteness of the data it follows that a $M(d)$ as required exists and we can write for this case,

$$\hat{\mathcal{R}}_n(\mathcal{H}) \leq M(d)\sqrt{\frac{2d\ln(16)}{n}}$$

11.1 Rademacher Complexity for Linear Predictors

The next simplest class of predictors that is of concern in M.L. is that of linear functions. In this segment we shall get bounds on the Rademacher complexity of the set of linear functions whose weight vectors are bounded in 2−norm − which is a constraint that is often weakly implemented in the real world via regularization terms in the loss function. (And we have already seen a basic example of such a regularization in the set up of training a simple neural net − the term "λw^2" in the loss function in equation 76.)

Towards the above goal, first we need to know a simple but very useful piece of mathematics, "The Jensen's Lemma" - of which we shall now show a proof in the restricted setting that shall be of immediate use to us.

Lemma 11.2 (**Jensen's Lemma (Version** 1**)).** *Let $F : \mathbb{R} \to \mathbb{R}$ be a convex function and let X be a real valued random variable which takes finitely many values. Then we have,*

$$F(\mathbb{E}[X]) \leq \mathbb{E}[F(X)]$$

In Subsection 11.2 we shall give a full proof of the above statement. For an immediate intuition about what the Jensen's Lemma says, consider $F(x) = x^2$ − the F that we shall need in our upcoming use cases − and then we have the statement that, $(\mathbb{E}[X])^2 \leq \mathbb{E}[X^2]$.

Note that the specific form of the Jensen's Lemma that we shall need is stated below and its proof is essentially identical to that of the above.

Lemma 11.3 (**Jensen's Lemma (Version** 2**)).** *Let $f : \mathbb{R} \to \mathbb{R}$ be a convex function and let $\boldsymbol{X}$ be a n−dimensional vector valued random variable which takes finitely many values and let $T : \mathbb{R}^n \to \mathbb{R}$ be a given function. Then we have,*

$$F(\mathbb{E}[T(\boldsymbol{X})]) \leq \mathbb{E}[F(T(\boldsymbol{X}))]$$

Concept Check 11.1. Prove the above lemma!

Now we have all the tools necessary to prove an interesting upper bound on the Rademacher complexity of weight-bounded linear predictors. That is, we want to bound the Rademacher complexity of the following set of functions corresponding to some arbitrary input dimension d and a weight bound B,

$$\mathcal{F} = \{\mathbb{R}^d \ni \boldsymbol{x} \mapsto \langle \boldsymbol{w}, \boldsymbol{x} \rangle \in \mathbb{R} \mid \|\boldsymbol{w}\|_2 \leq B\}$$

Also, for getting clean results for the average Rademacher complexity, we need to assume some bounds on the data distribution. We shall assume that $\mathbb{E}[\|\boldsymbol{x}\|_2^2]$ is bounded.

Now we can state the main theorem that we want to establish,

Theorem 11.4 (**Rademacher Complexity of** 2**-Norm Bounded Linear Predictors).** *Given any $B > 0$, let the class of linear predictors we consider be $\mathcal{F} = \{\mathbb{R}^d \ni \boldsymbol{x} \mapsto \langle \boldsymbol{w}, \boldsymbol{x} \rangle \in \mathbb{R} \mid \|\boldsymbol{w}\|_2 \leq B\}$. Then for any set of n data in d−dimensions, say $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n\}$, we have,*

$$\hat{\mathcal{R}}_n(\mathcal{F}) \leq \frac{B}{n} \cdot \sqrt{\sum_{i=1}^{n} \|\boldsymbol{x}_i\|_2^2}$$

Further, if we assume that the data distribution be s.t $\exists\ C > 0$ *s.t,* $\mathbb{E}[\|\boldsymbol{x}\|_2^2] \leq C^2$*, then it holds that,*

$$\mathcal{R}_n(\mathcal{F}) \leq \frac{B \cdot C}{\sqrt{n}}$$

The reader is urged to notice the key feature of the above result - that we have a bound on the Rademacher complexity that does *not* scale with the dimension of the data d - which in this case also happens to be equal to the number of trainable parameters i.e dimension$(\boldsymbol{w})$.

The quantity B in above – the 2–norm bound on the weights of the linear functions – is the key quantity on which the Rademacher complexity of linear predictors is seen to depend. Thus, recalling Theorem 10.1 and from above it is easy to believe that when we are learning using linear functions with a fixed norm bound on their weight vectors, then irrespective of whether we are working in high or low dimensions, our ability to generalize is critically dependent on this norm bound and *not* on the number of trainable parameters.

Thus we are at the cusp of being able to see an explicit example of how the "size of the model" does not necessarily directly affect the learning properties of a function/model class. The remaining gap between this intuition and a proof is to figure out how to transfer any bound on the Rademacher complexity of the predictor class to a corresponding loss class with which the actual minimization/training would happen.

Recall that in the example session last week, we had gone through the steps of proving equation 100, which pertains to a specific predictor class and a corresponding loss class. Last lecture we had pointed out that it is outside the ambit of this module to explain how its not uncommon to be able to upper bound the Rademacher complexity of a loss class by a constant multiple of the Rademacher complexity of a predictor class. When such relationships hold then it can be imagined that results analogous to Theorem 11.4, but proven for the $\mathcal{F}$ under consideration, lead to an upper bound on the expected value of the worst case generalization error, as explained in Theorem 10.1. Thus a viable path is paved to help us conclude that for certain classes of predictors and corresponding losses their generalization error may not scale with the number of trainable parameters.

Also note, how the average Rademacher, as well as the empirical Rademacher complexity, both, vanish as the number of samples, n, goes to infinity - but they vanish at different rates. If this vanishing can be shown to continue to hold for the Rademacher complexity of a corresponding loss class too, that would be tantamount to proving that if we arbitrarily increase the number of training samples then the generalization gap closes. Further note that this $\mathcal{O}\left(\frac{1}{\sqrt{n}}\right)$ rate for the average Rademacher complexity is extremely generic across many function classes - *it is called a "slow rate" for reasons that lie much outside the scope of this course.* As the function classes get increasingly more complicated beyond just the linear example considered here, the techniques get increasingly more involved for proving such bounds as given above - but much of the key ideas we will see in the upcoming proof do continue to remain fruitful.

You will not be not expected to reproduce any proof in the final MCQ exam.
However, answering certain questions would be easier,
if you understood the various techniques used in the proof.

Proof of Theorem 11.4. *Note that in this proof all norms shall mean* 2*–norms and we shall not explicitly state that every time.* From the definition of empirical Rademacher complexity it follows that,

$$\begin{aligned}\hat{\mathcal{R}}_n(\mathcal{F}) &= \mathbb{E}_{\boldsymbol{\epsilon}}\left[\sup_{\|\boldsymbol{w}\|\leq B} \frac{1}{n}\sum_{i=1}^{n} \epsilon_i \langle \boldsymbol{w}, \boldsymbol{x}_i \rangle\right] \\ &= \frac{1}{n}\mathbb{E}_{\boldsymbol{\epsilon}}\left[\sup_{\|\boldsymbol{w}\|\leq B} \left\langle \boldsymbol{w}, \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i \right\rangle\right]\end{aligned}$$

Now we recall the Cauchy-Schwartz inequality which states that for any two finite dimensional vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ we have, $\langle \boldsymbol{a}, \boldsymbol{b} \rangle \leq \|\boldsymbol{a}\| \|\boldsymbol{b}\|$. We invoke this inequality in the argument of the sup above to get,

$$\hat{\mathcal{R}}_n(\mathcal{F}) \leq \frac{1}{n} \mathbb{E}_{\boldsymbol{\epsilon}} \left[\sup_{\|\boldsymbol{w}\| \leq B} \|\boldsymbol{w}\| \left\| \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i \right\| \right] \leq \frac{B}{n} \mathbb{E}_{\boldsymbol{\epsilon}} \left[\left\| \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i \right\| \right]$$

In the last inequality above we have invoked the assumption on the norms of the weights to compute the supremum.

Now recall the relationship between inner-products and 2−norms of vectors i.e $\|\boldsymbol{a}\| = \sqrt{\langle \boldsymbol{a}, \boldsymbol{a} \rangle}$.

Thus we can write the last step as,

$$\hat{\mathcal{R}}_n(\mathcal{F}) \leq \frac{B}{n} \mathbb{E}_{\boldsymbol{\epsilon}} \left[\sqrt{\left\langle \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i, \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i \right\rangle} \right]$$

Now we can invoke Lemma 11.3 for the convex function $F(x) = x^2$, $\boldsymbol{X} = \boldsymbol{\epsilon}$ and $T(\boldsymbol{X}) = \langle \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i, \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i \rangle$ to get,

$$\left(\mathbb{E}_{\boldsymbol{\epsilon}} \left[\sqrt{\left\langle \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i, \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i \right\rangle} \right] \right)^2 \leq \mathbb{E}_{\boldsymbol{\epsilon}} \left[\left\langle \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i, \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i \right\rangle \right]$$

Thus we have,

$$\hat{\mathcal{R}}_n(\mathcal{F}) \leq \frac{B}{n} \sqrt{\mathbb{E}_{\boldsymbol{\epsilon}} \left[\left\langle \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i, \sum_{i=1}^{n} \epsilon_i \boldsymbol{x}_i \right\rangle \right]}$$

Recalling that $\epsilon_i^2 = 1$, we can evaluate the inner-product inside the expectation above to get,

$$\hat{\mathcal{R}}_n(\mathcal{F}) \leq \frac{B}{n} \sqrt{\mathbb{E}_{\boldsymbol{\epsilon}} \left[\sum_{i=1}^{n} \|\boldsymbol{x}_i\|^2 + \sum_{\substack{i,j=1 \\ i \neq j}}^{n} \epsilon_i \epsilon_j \langle \boldsymbol{x}_i, \boldsymbol{x}_j \rangle \right]} \tag{109}$$

From definition of the random variables ϵ_is it follows that they are of mean 0 and we have already assumed that they are independently sampled. So each term in the double sum above can be evaluated as,

$$\mathbb{E}\left[\epsilon_i \epsilon_j \langle \boldsymbol{x}_i, \boldsymbol{x}_j \rangle \right] = \langle \boldsymbol{x}_i, \boldsymbol{x}_j \rangle \mathbb{E}[\epsilon_i] \mathbb{E}[\epsilon_j] = 0$$

Substituting the above into equation 109 we get the intended upper bound,

$$\hat{\mathcal{R}}_n(\mathcal{F}) \leq \frac{B}{n} \sqrt{\sum_{i=1}^{n} \|\boldsymbol{x}_i\|^2}$$

Further, we can again use the same ideas as earlier to say that,

$$\left(\mathbb{E}\left[\sqrt{\sum_{i=1}^{n}\|\boldsymbol{x}_i\|^2}\right]\right)^2 \leq \mathbb{E}\left[\sum_{i=1}^{n}\|\boldsymbol{x}_i\|^2\right] = \sum_{i=1}^{n}\mathbb{E}\left[\|\boldsymbol{x}_i\|^2\right]$$

In the last equality above we have used the linearity of expectation. Then we recall that each random variable $\boldsymbol{x}_i$ is sampled from the same distribution and hence the definition of C can be invoked on each of the expectations in the sum above to get,

$$\mathbb{E}\left[\sqrt{\sum_{i=1}^{n}\|\boldsymbol{x}_i\|^2}\right] \leq \sqrt{n \cdot C^2}$$

Hence we can taken an expectation over data for the upper bound we have obtained on $\hat{\mathcal{R}}_n(\mathcal{F})$ to get,

$$\mathcal{R}_n(\mathcal{F}) \leq \frac{B \cdot C}{\sqrt{n}}$$

□

11.2 Proof of Lemma 11.2

You will not be not expected to reproduce any proof in the final MCQ exam.
However, answering certain questions would be easier,
if you understood the various techniques used in the proof.

Proof. We will prove the lemma by induction on the number of values that X can take. Suppose X takes only two values x_1 and x_2 with probabilities $p(x_1), p(x_2) \in [0,1]$ s.t $p(x_1) + p(x_2) = 1$. Then explicitly we have,

$$F(\mathbb{E}[X]) - \mathbb{E}[F(X)] = f(p(x_1)x_1 + p(x_2)x_2) - p(x_1)F(x_1) + p(x_2)F(x_2)$$

Then it immediately follows from the definition of convex functions that the above is non-positive and hence the lemma gets proven for this case.

Now suppose that the random variable X takes n values $x_1, x_2, \ldots, x_n$ with probabilities, $p(x_1), p(x_2), \ldots, p(x_n) \in [0,1]$ s.t $\sum_{i=1}^{n} p(x_i) = 1$.

Without loss of generality, we can assume that $p(x_1) \neq 1$ and assume that we have already proven the theorem for random variables taking $n-1$ values - that being the induction hypothesis. Then, in this case we have,

$$\begin{aligned}\mathbb{E}[F(X)] &= p(x_1)F(x_1) + \sum_{i=2}^{n} p(x_i)F(x_i) \\ &= p(x_1)F(x_1) + (1 - p(x_1)) \cdot \sum_{i=2}^{n} \frac{p(x_i)}{1 - p(x_1)} F(x_i)\end{aligned}$$

Note that $\sum_{i=2}^{n} \frac{p(x_i)}{1-p(x_1)} = 1$ and hence the weights in the second term above are themselves specifying a probability distribution. Hence we can invoke the induction hypothesis on those $n-1$ variables to get,

$$
\begin{aligned}
\mathbb{E}[F(X)] &= p(x_1)F(x_1) + \sum_{i=2}^{n} p(x_i)F(x_i) \\
&= p(x_1)F(x_1) + (1 - p(x_1)) \cdot \sum_{i=2}^{n} \frac{p(x_i)}{1 - p(x_1)} F(x_i) \\
&\geq p(x_1)F(x_1) + (1 - p(x_1)) \cdot F\left(\sum_{i=2}^{n} \frac{p(x_i)}{1 - p(x_1)} \cdot x_i \right)
\end{aligned}
$$

Now we can again invoke the definition of convexity at the last step above to get,

$$
\begin{aligned}
\mathbb{E}[F(X)] &\geq F\left(p(x_1) \cdot x_1 + (1 - p(x_1)) \cdot \sum_{i=2}^{n} \frac{p(x_i)}{1 - p(x_1)} \cdot x_i \right) \\
&\geq F(\mathbb{E}[X])
\end{aligned}
$$

And hence induction follows and we have proven what we had set out to prove. □

11.3 Looking Ahead With Neural Nets

A fundamental mystery with neural nets can be stated by noting the following experimental fact about them. Firstly, it is possible to fix training data and a value of depth and choose increasingly wide nets s.t all of them have been trained to have training errors below a certain low threshold (like, imagine having all their squared loss errors being less than 10^{-3} – a situation that is not very uncommon). Thus, we start from having been given a sequence of nets of increasingly large number of parameters at a fixed depth, all of which have been trained as best as possible on a fixed training data. Then it turns out that for *many* such sequences of well trained nets that can be constructed in experiments, we observe 3 properties to hold in tandem,

- The (estimated) Lipschitz constant of the trained nets will neither be very small nor be decreasing – might even be moderately increasing as the number of parameters increase.
- Such good nets can be found with as many parameters as,
  data−dimension × number−of−training−data = $d \times n$, or even more.
- The test error would not be increasing and might even be improving as the number of parameters increase (while all of them have nearly identical and nearly zero training errors)

To the best of our knowledge, there is no mathematical formalism known till date which can explain the coexistence of these 3 *above-stated critical features in very many such experiments.* In this last section of the course we have made a modest attempt at trying to explain *some* small parts of this gargantuan mystery.

A Supervised Learning Basics

Outline: This section reviews the necessary background material on the basics of Machine Learning.

We assume you have a good understanding of everything in COMP24112. However, there are some topics where we will focus more than others. In particular, COMP24112 chapters 5 and 6, on loss functions and training schemes. In this chapter we will briefly review this, though not in depth—refer to your COMP24112 notes for full information.

After reviewing this, you should be able to define and use the following concepts correctly in a conversation.

- overfitting versus underfitting,
- the squared loss and cross-entropy loss,
- the gradient descent algorithm, and global vs local minima,
- k-nn and regression tree models, and the effect of their parameters

A.1 Supervised Learning basics: over-fitting and under-fitting

In this module we will focus on *supervised learning* tasks. For any input, we have access to a 'label' (sometimes known as a 'target'), which we regard as the correct response for the input.

Let's look at a concrete (but toy) learning problem, which will illustrate our points. So, here's your *training set*, of $n = 20$ datapoints. Each input is a value on x-axis, the target label you get is the corresponding point on the y-axis. This is a result of some *true* underlying function, plus some random noise.

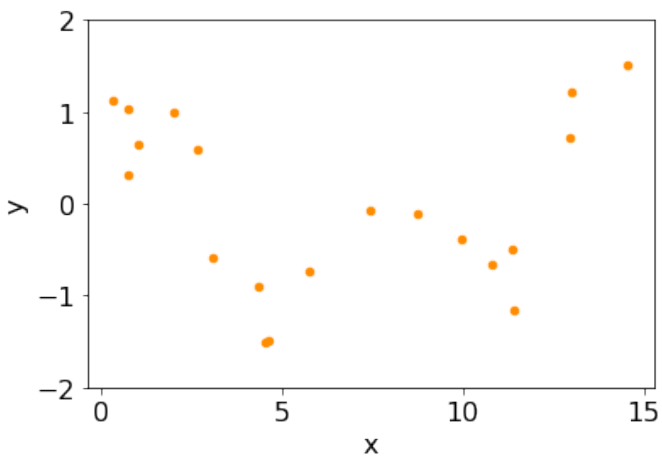
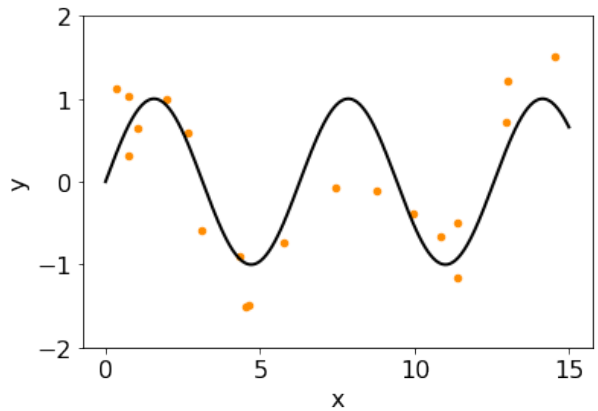


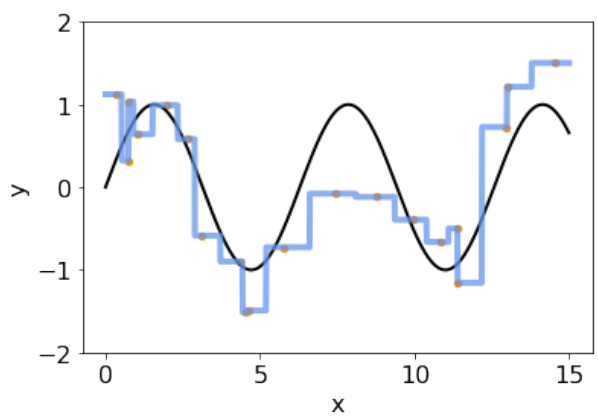
Figure 31: A sample of $n = 20$ points to use as a training set. Can you tell what the true function is?

The task is to learn what the underlying true function is, from this small (and noisy) sample. Can you see what it is? I'd be surprised if you could....

In fact it's just $sin(x)$, shown below. But, the training sample was so small, and so noisy, that it missed the middle peak of the sine wave, not sampling any training data from there.



Let's see what a Machine Learning model can do with this data. We will use a k-nearest neighbour regression, with a squared distance measure. **If you don't remember what k-nn regression is, go back to your COMP24112 notes (Lecture 2A) for some revision.** The fitted model looks like this:



The blue line shows the fitted predictions of the k-nn regression model, with $k = 1$. We fitted very well (perfectly in fact) to this training set. However, we can see visually that performance on the true underlying function (black line, sine wave) was *terrible*.

We used $k = 1$ here—this determines the *fit* of the model. Maybe if I give you more data ($n = 100$) and try different parameters, we can get better performance? Trying $k = 1$, $k = 8$, and $k = 20$, we get the following.

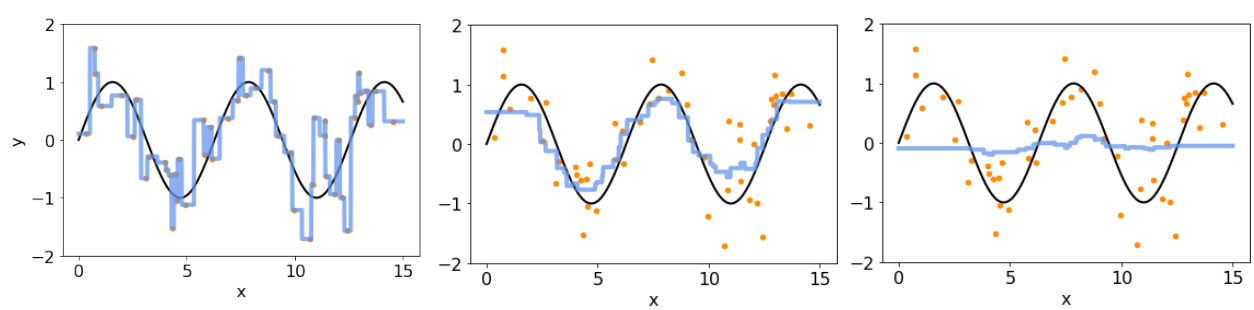


Figure 32: K-nearest neighbor regression, with $k = \{1, 8, 20\}$. Can you guess which figure is which?

Hopefully you are remembering some keywords from COMP24112, **over-fitting** and **under-fitting.** The model for $k = 1$ (left) is **over-fitted** because it is "fine-tuned" to the training data, unable to represent the true function—i.e. it has high testing error. The model for $k = 20$ (right) is **under-fitted**, because it has virtually ignored the training data, and is again unable to represent the function, giving again high testing error. The model for $k = 8$ (middle) is a good fit, at least as can be seen visually. As you can see, the idea of over- and under-fitting are intrinsically linked to the notion of *complexity*. The model on the left is very *complex*. The model on the right is very *simple*.

The k-nn is a *memory-based* algorithm, also known as *instance-based learning*. It just memorises the training data, then returns the average label of the k closest neighbours in the feature space, where 'close' is defined by a distance measure—in our case we chose squared distance. This necessitates storing all the training data in memory. As a consequence, it's not great at generalising beyond this training data to new scenarios. So, let's consider other types of models, which are sometimes known as *function approximators*. You met a couple of these in COMP24112, i.e. linear regression, and neural networks, but there are many alternatives:

- Linear Regression
- Logistic regression
- Support Vector Machines
- Neural networks (e.g. convolutional neural nets, or multilayer perceptrons)
- Naive Bayes (or more generally, probabilistic models)
- Decision Trees (for both regression and classification)

The thing that these models have in common is that they work by trying to minimise **loss functions.** Let's see some details of two very common losses.

A.2 Loss functions: squared and cross-entropy

We can denote a generic model as $f(\boldsymbol{x})$. Here, $\boldsymbol{x}$ is the feature vector, which we will assume is a real-valued vector of length d, or more formally written as $\boldsymbol{x} \in \mathbb{R}^d$. **In this notation, we have left it implicit that the model has some parameters**, which we denote $\boldsymbol{w} \in \mathbb{R}^p$, i.e. there are p parameters. The model might be for example a neural network, but as you see in the list above, there are many other non-neural models that can perform the same job. For a given model f, we evaluate its performance with a loss function.

For *regression* problems, we predict a value in $\mathbb{R}$, and a very common loss is the *squared* loss function:

$$\ell(y, f(\boldsymbol{x})) = (y - f(\boldsymbol{x}))^2. \tag{110}$$

Thus, we have a measure of performance for our model f, in predicting the correct value y based on input $\boldsymbol{x}$. We have a *training dataset* of n points: $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$. We can average the loss over this set. You probably know this as the "training error", but a more conventional name for this is the "risk" of the model, so we use notation R.

$$\hat{R}(f) = \frac{1}{n}\sum_{i=1}^{n}(y_i - f(\boldsymbol{x}_i))^2. \tag{111}$$

We use a 'hat' over the R to denote that it's an estimate, that would change with the size of the sample, n. We want to find the parameters $\mathbf{w}$ that minimise this:

$$\mathbf{w}^* := \arg\min_{\mathbf{w}} \left[\hat{R}(f)\right]. \tag{112}$$

However, our ultimate measure of performance is not the training risk, but the performance on a final testing set, that the model has never seen. You can think about this like some driving lessons (your training set) and a final driving test (your testing set).

For a *classification* problem, our model f will output a vector of predicted class probabilities, and we use the *cross-entropy* as our loss. As a reminder, for two classes, the cross-entropy is:

$$\ell(y, f(\boldsymbol{x})) \;:=\; -\Big[y \ln f(\boldsymbol{x}) + (1-y)\ln(1-f(\boldsymbol{x}))\Big] \tag{113}$$

where $y \in \{0,1\}$ and $f(\boldsymbol{x}) \in (0,1)$. For the general multi-class case, it is:

$$\ell(\boldsymbol{y}, f(\boldsymbol{x})) \;:=\; -\Big[\boldsymbol{y} \cdot \ln f(\boldsymbol{x})\Big] \tag{114}$$

where $\boldsymbol{y}$ is a **one-hot** vector. This is a vector of all zeroes, apart from one, which is 1 in the index of the true class. For example, if we have 3 classes, and the correct class for a given example is 3, then $\boldsymbol{y} = [0,0,1]$.

Our model f returns a vector of probability estimates for each class, e.g. $[0.2, 0.3, 0.5]$. We can visualise the 3-class case as a *probability simplex*, where a single prediction $f(\boldsymbol{x})$ is a point on the simplex.

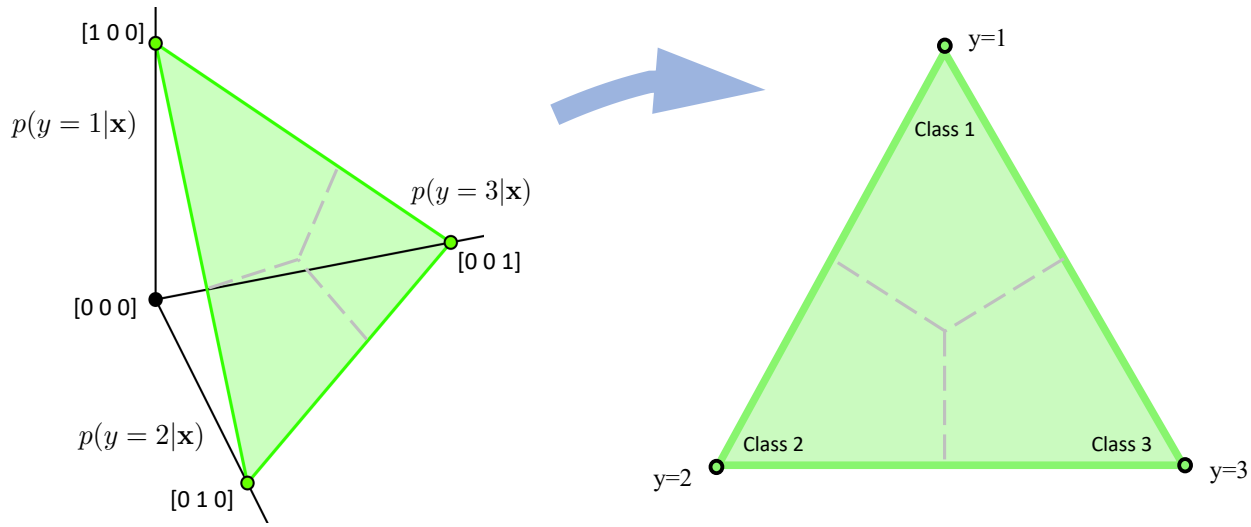


Figure 33: Probability simplex, visualizing the space in which we make predictions for a 3-class problem.

Again, refer back to COMP24112 notes (lectures 5A, 5B, and 5C) for some revision.

Concept Check A.1. I give you an example where the true label is $y = 1$.
My model predicts the probability of $y = 1$ as $f(\boldsymbol{x}) = 0.95$.

Q1. Calculate the cross-entropy loss. Remember to use the **natural** logarithm.

Now, I give you an example $\boldsymbol{x}$, where $\boldsymbol{y} = [1,0,0]$.
My model predicts $f(\boldsymbol{x}) = [0.3, 0.6, 0.1]$.

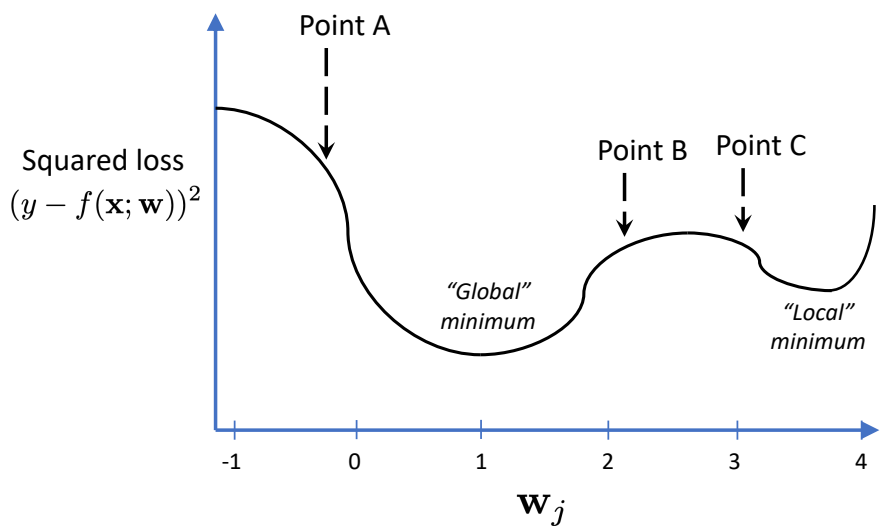
Q2. Calculate the cross-entropy loss again.

Q3. Roughly where in the simplex does this model sit?

A.3 Gradient Descent as a way of minimizing loss

One of the most heavily used principles for a learning algorithm is *gradient descent*. In the final weeks of this module, we will be studying advanced aspects of GD, so it's important you understand the basics.

Imagine we have a squared loss function, applied to the output of a big neural network. Somewhere within that neural network there is a weight, $\mathbf{w}_j$, that we decide to change. We vary $\mathbf{w}_j$ *manually* up and down, and record what the squared loss is. The result looks like this:



But we can't manually plot the loss like this, as it's too computationally expensive. If we could, we'd know immediately to set our parameter to the **'global' minimum**, where $\mathbf{w}_j = 1$. And, we'd want to avoid the inferior **'local' minimum** where $\mathbf{w}_j = 3.7$, or indeed any other place on the landscape. But we can't. Instead we can only guess some initial value for $\mathbf{w}_j$, measure the loss, and then guess how we should change it. Consider the situation if we started from point A, B, or C, above.

The factor that differentiates point A from B is the *gradient*, i.e. the slope of the function. Point A is a negative (downhill) gradient, while point B is a positive (uphill) gradient. The principle of gradient descent is to minimize a loss by examining the sign of the gradient.

> **If the gradient is NEGATIVE, we INCREASE the parameter.**
>
> **If the gradient is POSITIVE, we DECREASE the parameter.**

You can see that by increasing the parameter from point A, we do indeed approach the global minimum. The reverse is true for point B, decreasing the parameter would cause us to minimise the loss. It's important to see that, if we had followed this principle from point C, we would have ended up at a sub-optimal place, the 'local' minimum.

We can calculate the gradient *at any location* by taking derivative with respect to $\mathbf{w}_j$. Remembering that we are using the squared loss,

$$\ell(y, f(\boldsymbol{x})) = (y - f(\boldsymbol{x}))^2, \tag{115}$$

then the gradient is:

$$\frac{\partial \ell(y, f(\boldsymbol{x}))}{\partial \mathbf{w}_j} = \frac{\partial \ell(y, f(\boldsymbol{x}))}{\partial f(\boldsymbol{x})} \frac{\partial f(\boldsymbol{x})}{\partial \mathbf{w}_j} = 2\Big(f(\boldsymbol{x}) - y\Big) \frac{\partial f(\boldsymbol{x})}{\partial \mathbf{w}_j}. \tag{116}$$

If you don't know how this gradient is calculated, you should remind yourself of basic calculus—see also Lecture 6C from COMP24112.

Take a look at the landscape on the previous page. If we were at point A (negative gradient) we would increase $\mathbf{w}_j$, exactly as we decided we needed to do in the plotted landscape, and similarly for point B (positive gradient). This can be built into an algorithm.

Algorithm 4 Gradient Descent for parameter vector $\mathbf{w}$ on a differentiable loss $\ell(y, f(\boldsymbol{x}))$

1: **Choose :** Constant $T > 0$ for the number of steps to take.
2: **Choose :** Constant η, for the size of each step.
3: **Input:** An initial vector $\mathbf{w}$ of length p.
4: **for** $t = 1, \ldots, T$ **do**
5: $\quad \mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla \ell(y, f(\boldsymbol{x}))$
6: **end for**
7: **Output : w**

Notice that here we used the 'nabla' notation ∇, which denotes the fact that $\boldsymbol{w}$ is a vector. For a vector of length d, we have the gradient vector:

$$\nabla \ell\Big(y, f(\boldsymbol{x})\Big) = \left[\frac{\partial \ell(y, f(\boldsymbol{x}))}{\partial \mathbf{w}_1}, \ \frac{\partial \ell(y, f(\boldsymbol{x}))}{\partial \mathbf{w}_2}, \ \ldots, \ \frac{\partial \ell(y, f(\boldsymbol{x}))}{\partial \mathbf{w}_d} \right]^T. \tag{117}$$

Notice also in the above, we are calculating the gradient only for a *single* datapoint $(\boldsymbol{x}, y)$. In reality we will have a full training set to deal with. Look at line 5 in the above algorithm. This is known as the **update rule** for gradient descent. If we had a training set of n datapoints, we could replace this with the following, sometimes known as **batch gradient descent**.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{1}{n} \sum_{i=1}^{n} \left[\nabla \ell(y_i, f(\boldsymbol{x}_i)) \right] \tag{118}$$

Alternatively we can take smaller batches of examples, called 'mini-batch' gradient descent. One reason to use a smaller batch is to have stochastic estimates of the gradient, which can help in escaping local minima.

A.4 Decision trees

An important model family is *decision trees*. These are used widely in industry, as they are fast to both train and deploy. They are excellent models for so-called **tabular** data, which means basically anything that fits in a spreadsheet—not images, or speech signals, or videos.

Decision trees come in two types—for classification, and regression. The basic idea is to recursively split the dataset into subsets based on the values of input features, and then fit a simple model (such as a constant label, or a linear regression) to each subset. Once the tree is built, it can be used to predict the target variable for new samples by traversing the tree from the root to a leaf node, and then using the simple model associated with that node to make the prediction. Below you can see a classification tree, partitioning a 2D space and predicting the probability of a blue circle in each sub-area.

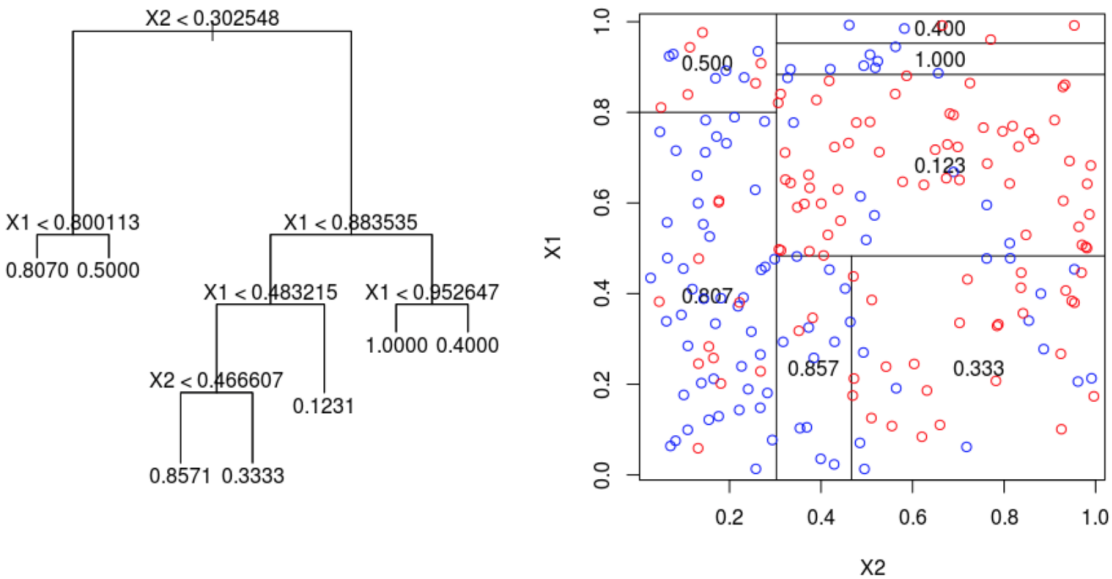


Figure 34: A classification tree.

Regression trees look similar to this, but in each subset of the data they predict a **continuous value**, such as the price of a house. The recursive splits are chosen in such a way as to minimize the variability of the target variable within each subset. We won't go into details of the training procedure here.

With every ML model, there are parameters for you to tune. With trees, the main one is how *deep* it is. This translates to a more fine grained partitioning of the space shown above, equating to a more complex decision boundary. In Figure 36 you can see the decisions from a regression tree as we increase the depth. This is using the sine wave toy problem shown earlier.

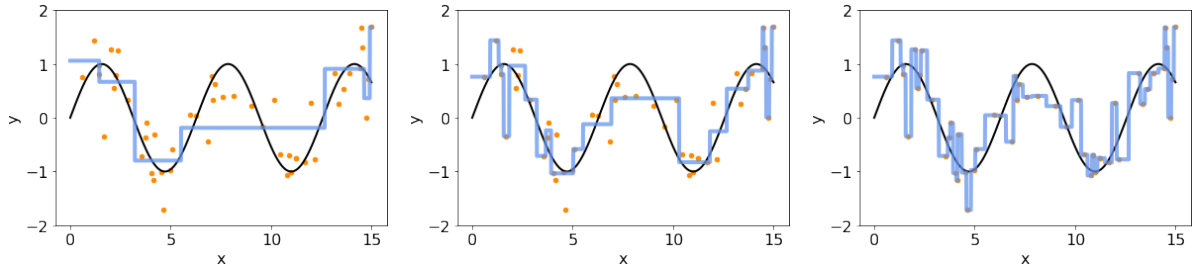


Figure 35: Decision boundaries for trees of increasing depth. From left to right, depth 3, 5, and 16.

Plotting the full range of depth from 1 to 16 reveals that over-fitting occurs around depth 5 or 6. Below we see this, where the darker red lines are averages over 50 repeats of the experiment, and the faint red lines are the 50 individual runs.

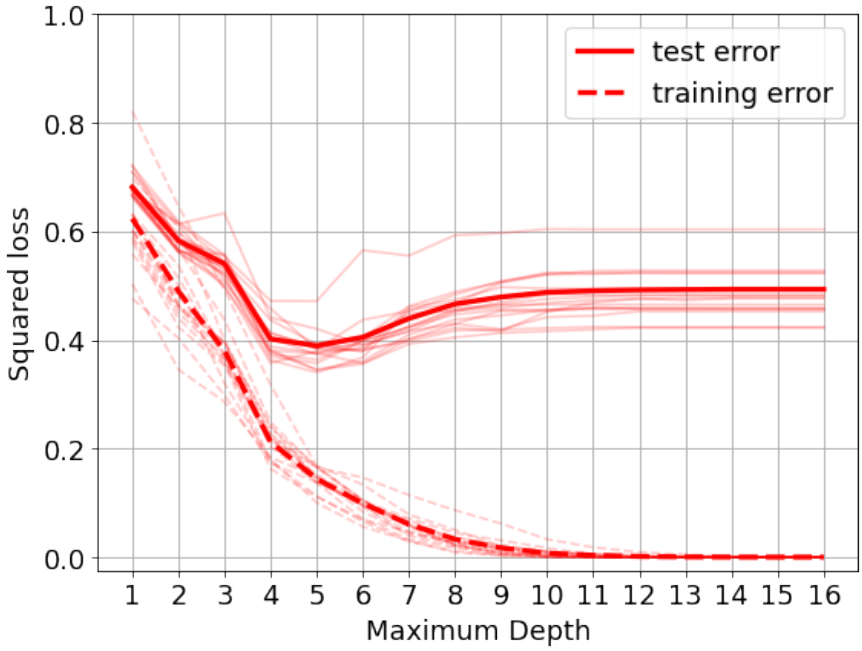


Figure 36: Training and testing loss, as we increase the depth of a regression tree.

A.5 Linear Regression

This section will review some of the key concepts about linear regression that were introduced in Lecture 6C of COMP24112. As alternative reading you may like this: `https://aunmnn.github.io/ml-tutorial/html/blog_content/linear_regression/linear_regression_tutorial.html`. This review will foreshadow some of the major concepts to come later in this module.

Consider the following (fictitious) house price data.

$$\begin{bmatrix} Floor\ Area(in\ m^2) & \#Bedrooms & Cost(in\ 10^3\ GBP) \\ 2104 & 3 & 400 \\ 1600 & 4 & 330 \\ 2400 & 5 & 369 \\ \vdots & \vdots & \vdots \end{bmatrix}$$

Here, the ith housing data can be thought of as a pair $(\boldsymbol{x}_i, y_i)$ where each $\boldsymbol{x}_i$ is the data about the ith house in the first two columns of the ith–row — represented as a two-dimensional vector i.e $\boldsymbol{x}_i \in \mathbb{R}^2$ — and y_i is the corresponding house price given as a real number i.e $y_i \in \mathbb{R}$. More precisely, we identify the coordinates of each $\boldsymbol{x}_i = (x_{i1}, x_{i2})$ as follows : x_{i1}, the first coordinate of $\boldsymbol{x}_i$ is the floor area of the ith house in the data and x_{i2} is its number of bedrooms. In general, when designing a learning problem, it is a very tricky issue to decide what features to choose i.e what should constitute the vectors $\boldsymbol{x}_i$. This design choice is often called "feature selection", but for now let's take the features as given.

Given the data as above, we set ourselves the task of performing "supervised learning" i.e of figuring out a function that evaluates to a good estimate of y_i given $\boldsymbol{x}_i$ as input. Toward this goal we must decide how we will represent functions/hypotheses f in a computer. As an initial easy choice, let's say we decide to

approximate y as a linear function of $\boldsymbol{x}$ i.e as,

$$f(\boldsymbol{x}) = w_1 x_{i1} + w_2 x_{i2} = \langle \boldsymbol{w}, \boldsymbol{x} \rangle = \boldsymbol{w}^\top \boldsymbol{x}$$

In the last two equalities above we have recalled the notation of inner-product between two vectors. The two dimensional vector $\boldsymbol{w} = (w_{i1}, w_{i2})$ here are the parameters (also called "weights") for the linear predictor f. *Note that, the above chosen form of f implies that the predicted house price is* 0 *if its floor area and the number of bedrooms are both* 0 *– which makes sense. But then f is not* 0 *if the floor area alone is* 0 *and hence the model is not that realistic either — thus we can immediately see why modeling via linear functions is quite limiting!*

Now we come to face one of the core questions of machine learning, which is to design a method of using the "training data", the table above, to pick or "learn" a "good" set of parameters $\boldsymbol{w}$. At the very outset, we reckon that there is no obvious or generic or even an optimal way to do so, even when the form of the predictor f has been fixed to the simple setup as above. Hence we choose to proceed with arguably one of the easiest options available.

The simplest reasonable method is to try to choose $\boldsymbol{w}$ s.t $f(\boldsymbol{x})$ very close to y, at least for the training examples we have. (Whether or not this aim is sufficient to ensure proximity on other $(\boldsymbol{x}, y)$ not in the training data is a much harder puzzle — to which we will return in the last lecture of this module!) To formalize this idea, we will define a function that measures for each value of $\boldsymbol{w}$, how close the $f(\boldsymbol{x}_i)$ are to the corresponding y_i *on an average over all data* i.e we define a cost function or training error or "empirical risk" as,

$$\hat{R}(f, \mathcal{S}_n) = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{2} \cdot (y_i - f(\boldsymbol{x}_i))^2 = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{2} \cdot (y_i - \langle \boldsymbol{w}, \boldsymbol{x}_i \rangle)^2 \tag{119}$$

where we have introduced the notation of $\mathcal{S}_n$ as the set of n training data $\{(\boldsymbol{x}_i, y_i) \mid i = 1, \ldots, n\}$ being used.

This training error above is often called the **"Ordinary Least Squares (OLS)"** regression setup.

Linear Regression with Bias First, note that it is often a good model of reality to assume that there exists a function f_*, a label generating function, s.t $y_i = f_*(x_i)$ for all training data $(\boldsymbol{x}_i, y_i)$. Secondly, sometimes it is convenient to consider a more general form of linear regression that allows for a bias variable, b, whereby the empirical risk would be $\frac{1}{n} \sum_{i=1}^{n} \frac{1}{2} \cdot (y_i - b - \langle \boldsymbol{w}, \boldsymbol{x}_i \rangle)^2$. Note that this is tantamount to starting with the predictors f being $f(\boldsymbol{x}) = b + \langle \boldsymbol{w}, \boldsymbol{x} \rangle$. As a quick demonstration of the utility of linear regression, in the experiment shown in Figure 37 we will see an example of how under fortuitous conditions minimizing this empirical risk above, over all choices of parameters $(\boldsymbol{w}, b)$ leads to the discovery of a predictor that very accurately matches the chosen label generating function on the training data — while f_* isn't a linear function.

Concept Check A.2. The reader is encouraged to (a) repeat this experiment in Figure 37 using higher values of angles — and realize how large the angles need to be for the linear regression prediction through a sample of them to visibly start to become a bad predictor. (b) Also, try repeating this experiment with other label-generating functions than the one used here.

Concept Check A.3. Let's take a closer look at the calculation accomplished by the code used to generate the right picture of Figure 37. Let's denote the n data points used therein (the blue dots) to be given as, $\{(x_1, y_1), \ldots, (x_n, y_n)\}$. (That $y_i = \sin(x_i)$ in the particular demonstration above is not relevant for the insight we are aiming for in this exercise.) It follows – from the discussions previously

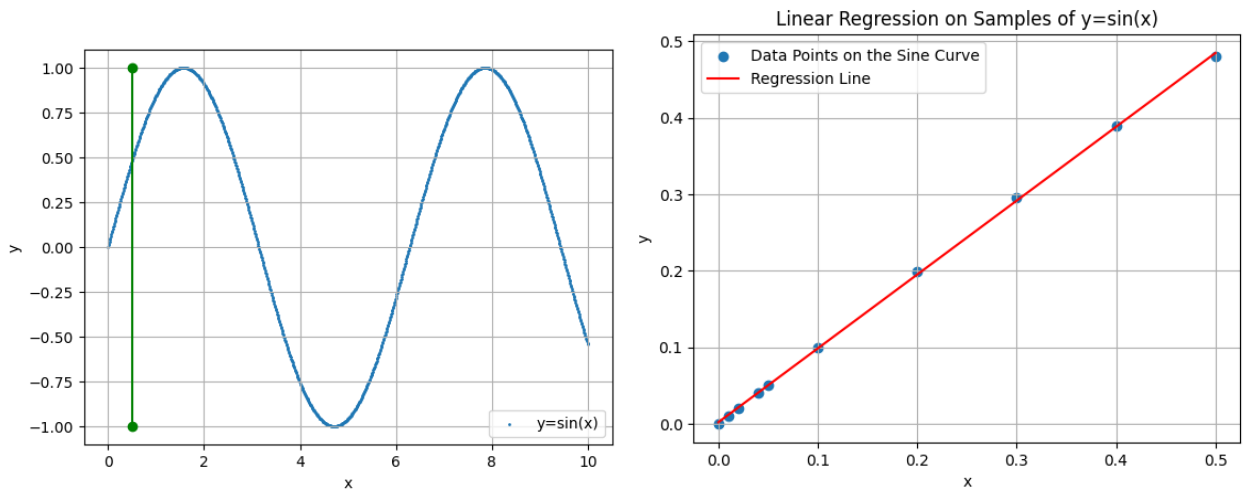


Figure 37: In this example we will see how linear regression can almost accurately recover a complicated label generating function — in the vicinity of the training data – if the data has certain nice properties. **On the left,** we see a plot of the sin function with a green vertical line marked at $x = 0.5$. Visually its already apparent that the sin function is almost linear on the interval $[0, 0.5]$. In the right panel we will concretely demonstrate this intuition by doing a linear regression on a sample of points on the part of the sin curve displayed to the left of the green line. **On the right,** we have *arbitrarily* chosen some labeled data (the blue dots) on the function $y = \sin(x)$ for $0 \leq x \leq 0.5$ and found the linear regression through those samples. The linear regressor (the red line) here is the predictor $f(x) = w \cdot x + b$ where $w \sim 0.995, b \sim 0.002$. The nearly exact match between the blue dots and the red line goes to show how close to linear the sine function is for low values of the angle! (The code to generate this figure can be found at this link.)

— that the empirical risk function to be minimized for doing this linear regression is,

$$\hat{R}(w, b) = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{2} \cdot (y_i - wx_i - b)^2$$

The code used in Figure 37 essentially finds the minimizer (w_*, b_*) of the above empirical risk. Find the analytic expression for this minimizer in terms of the data.

Recall from Lecture 6C of COMP24112, that one way to approach this question would be to take the partial derivatives of $\hat{R}$ w.r.t its variables (w, b) and solve for the (w, b) at which the partial derivatives evaluate to 0 i.e find (w_*, b_*) s.t they solve the following pairs of equations simultaneously, $\frac{\partial \hat{R}}{\partial w} |_{w=w_*} = 0$ and $\frac{\partial \hat{R}}{\partial b} |_{b=b_*} = 0$.

The method of finding minimizers of the empirical risk as given above is not entirely general but it shall suffice for all cases in this course. The reason why this technique of minimizing a function works is that the function $\hat{R}$ above is "convex" — a concept that shall be presented formally in Lecture 7.

Linear Regression by Gradient Descent Thus we come to recognize that our objective is to find $\boldsymbol{w}$ such that the empirical risk $\hat{R}(f, \mathcal{S}_n)$ in equation 119 is the lowest it can be. The demonstration given above is not easy to replicate for more general data and hence there arises the need for other techniques of doing linear regression. Towards this goal we implement the gradient descent iterates — as specified between steps 4 to 6 earlier in Algorithm 4 — on this empirical risk $\hat{R}(f, \mathcal{S}_n)$ to derive the following,

$$
\begin{aligned}
&\text{for } \{t = 0, 1, \ldots\} \\
&\qquad \Big\{ \\
&\qquad w_{1,t+1} = w_{1,t} - \eta \frac{\partial \hat{R}(\boldsymbol{w}_t)}{\partial w_{1,t}} = w_{1,t} - \eta \frac{1}{n} \sum_{i=1}^{n} (y_i - \langle \boldsymbol{w}_t, \boldsymbol{x}_i \rangle) \cdot (-x_{i1}) \\
&\qquad\qquad = w_{1,t} - \frac{\eta}{n} \sum_{i=1}^{n} (\langle \boldsymbol{w}_t, \boldsymbol{x}_i \rangle - y_i) \cdot x_{i1} \\
&\qquad w_{2,t+1} = w_{2,t} - \eta \frac{\partial \hat{R}(\boldsymbol{w}_t)}{\partial w_{2,t}} = w_{2,t} - \eta \frac{1}{n} \sum_{i=1}^{n} (y_i - \langle \boldsymbol{w}_t, \boldsymbol{x}_i \rangle) \cdot (-x_{i2}) \\
&\qquad\qquad = w_{2,t} - \frac{\eta}{n} \sum_{i=1}^{n} (\langle \boldsymbol{w}_t, \boldsymbol{x}_i \rangle - y_i) \cdot x_{i2} \\
&\qquad \Big\}
\end{aligned}
\tag{120}
$$

Note that in above we have denoted the value of the training loss $\hat{R}(f, \mathcal{S}_n)$ at a weight vector $\boldsymbol{w}_t$ as $\hat{R}(\boldsymbol{w}_t)$ to emphasize its dependency on the current "guess" of the good weight value i.e $\boldsymbol{w}_t$. Also recall the "chain rule of differentiation" that has been used to get the second equality in each of the two updates done above.

The above algorithm can be seen as using the updates to (potentially) improve the guess of the best weights from $\boldsymbol{w}_t$ to $\boldsymbol{w}_{t+1}$. We note two salient points about the algorithm specified in equation 120, (A) it is kept implicit that there is a choice of $\boldsymbol{w}_0$ that was made in some way — *and the details about* $\boldsymbol{w}_0$*, the "initialization", will matter when we return to this algorithm in Lecture 9.* (B) the update step in the algorithm above can be written in a vectorial form using the notation of a gradient introduced in the previous section,

$$
\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta \nabla \hat{R}(\boldsymbol{w}_t) = \boldsymbol{w}_t - \eta \left\{ \frac{1}{n} \sum_{i=1}^{n} (\langle \boldsymbol{w}_t, \boldsymbol{x}_i \rangle - y_i) \cdot \boldsymbol{x}_i \right\} \tag{121}
$$

Concept Check A.4. The reader is encouraged to check that, (1) the updates specified in equations 120 and 121 are identical and (2) that the updates specified in equation 121 are well-defined even if $\boldsymbol{x}$ were to be d–dimensional vectors and not just the two-dimensional data as was the motivating example in the beginning.

The Normal Equations Given the simplicity of the empirical risk specified in equation 119, interestingly enough it lends itself to a *direct* evaluation of its global minimum – *under certain conditions.* This is a very instructive special case to understand but to appreciate that we need to introduce a certain amount of "matrix calculus" notation. *We strongly emphasize that this is no different from calculus as you would already be familiar with and that this is merely a better notation.*

Consider a function h which evaluates to a real number when given as input a $n \times d$ matrix. Such a function would be specified as the map, $h : \mathbb{R}^{n \times d} \to \mathbb{R}$. The notation of $\mathbb{R}^{n \times d}$ stands for the set/space of all matrices of dimensions $n \times d$ whose entries are allowed to be any real number. Then we can define the "gradient of h w.r.t its input (say a matrix $\boldsymbol{A}$)" as another $n \times d$ dimensional matrix specified as,

$$
\nabla_{\boldsymbol{A}} h = \begin{bmatrix} \frac{\partial h}{\partial A_{11}} & \cdots & \frac{\partial h}{\partial A_{1d}} \\ \vdots & \cdots & \vdots \\ \frac{\partial h}{\partial A_{n1}} & \cdots & \frac{\partial h}{\partial A_{nd}} \end{bmatrix} \tag{122}
$$

The reader is urged to carefully note that that the gradient matrix $\nabla_{\boldsymbol{A}} h$ is of the same dimensions as the differentiation variable matrix $\boldsymbol{A}$. Keeping an eye out for this can go a long way towards preventing calculation mistakes. Also note that the above includes differentiation w.r.t vectors as a special case corresponding to either n or d being 1. For a quick explicit example consider the function $h : \mathbb{R}^{2\times 2} \to \mathbb{R}$ given as $h(\boldsymbol{A}) = 2A_{11} - 3A_{12} + 5A_{22}^5$. Then we would have its derivative being given as,

$$\nabla_{\boldsymbol{A}} h = \begin{bmatrix} 2 & -3 \\ 0 & 25A_{22}^4 \end{bmatrix} \tag{123}$$

In light of the above let us revisit the OLS empirical risk that was introduced in equation 119,

$$\hat{R}(f, \mathcal{S}_n) = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{2} \cdot (y_i - f(\boldsymbol{x}_i))^2 = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{2} \cdot (y_i - \langle \boldsymbol{w}, \boldsymbol{x}_i \rangle)^2 \tag{124}$$

Now we realize that the above makes sense even if we are not stuck with two dimensional data as was the motivating example therein. We realize that the above notion of empirical risk makes sense even if the data $\boldsymbol{x}$ were d–dimensional vectors and correspondingly the regression weight $\boldsymbol{w}$ too were a d–dimensional vector.

This motivates us to define the following 3 matrices,

$$\boldsymbol{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}, \boldsymbol{X} = \begin{bmatrix} x_{11} & \dots & x_{1d} \\ \vdots & \vdots & \vdots \\ x_{n1} & \dots & x_{nd} \end{bmatrix}, \boldsymbol{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix} \tag{125}$$

Note that the dimensions of the three matrices above are $n \times 1, n \times d$ and $d \times 1$ respectively. Then by doing the matrix multiplications it follows that we have,

$$\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} - \begin{bmatrix} x_{11} \cdot w_1 & \dots & x_{1d} \cdot w_d \\ \vdots & \vdots & \vdots \\ x_{n1} \cdot w_1 & \dots & x_{nd} \cdot w_d \end{bmatrix} = \begin{bmatrix} y_1 - \langle \boldsymbol{w}, \boldsymbol{x}_1 \rangle \\ \vdots \\ y_n - \langle \boldsymbol{w}, \boldsymbol{x}_n \rangle \end{bmatrix} \tag{126}$$

Note that $\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}$ above is a n–dimensional vector or equivalently $n \times 1$ dimensional matrix. Now recall that for any vector $\boldsymbol{z}$, the square of the length of its norm or the its distance from the origin is given as, $\|\boldsymbol{z}\|^2 = \boldsymbol{z}^\top \boldsymbol{z} = \sum_i z_i^2$. Hence the OLS empirical risk in equation 124 — read for data and weights being both d–dimensional — can be re-written as,

$$\hat{R}(\boldsymbol{w}) = \frac{1}{2n} \cdot \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}\|^2 = \frac{1}{2n} \cdot (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})^\top (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}) \tag{127}$$

Note that — as was done when introducing the gradient descent algorithm in equation 121 — we have again written the empirical risk as a function of the weight vector, as that is the dependency we want to focus on. Next, we can compute the gradient of this empirical risk w.r.t its weight vector $\boldsymbol{w}$ as follows. First, we formally denote the derivative as,

$$\nabla \hat{R}(\boldsymbol{w}) = \nabla \left(\frac{1}{2n} \cdot (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w})^\top (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}) \right) \tag{128}$$

Note that in above one might have chosen to write the gradient ∇ as $\nabla_{\boldsymbol{w}}$ to emphasize the variable w.r.t which the gradient is being taken.

Thus opening up the brackets we have,

$$\nabla \hat{R}(\boldsymbol{w}) = \frac{1}{2n} \cdot \nabla \left(\|\boldsymbol{y}\|^2 - \boldsymbol{y}^\top \boldsymbol{X}\boldsymbol{w} - \boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{y} + \boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w} \right) \tag{129}$$

The first term inside the brackets does not have any dependency on the weights and hence its derivate is 0. Also note that we can simplify the third term as, $\boldsymbol{w}^\top(\boldsymbol{X}^\top \boldsymbol{y}) = (\boldsymbol{X}^\top \boldsymbol{y})^\top \boldsymbol{w} = (\boldsymbol{y}^\top \boldsymbol{X})\boldsymbol{w}$. Hence we have,

$$\nabla \hat{R}(\boldsymbol{w}) = \frac{1}{2n} \cdot \nabla \left(-2\boldsymbol{y}^\top \boldsymbol{X}\boldsymbol{w} + \boldsymbol{w}^\top \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w} \right) = \frac{1}{n} \left(\boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w} - \boldsymbol{X}^\top \boldsymbol{y} \right) \tag{130}$$

Recall that you have already seen the above equation in Lecture 6C in COMP24112, except that the loss function $\hat{R}$ was called O in there and it was defined without the factor of $\frac{1}{n}$ as we use here.

Concept Check A.5. The reader is encouraged to convince themselves that the last equality in equation 130 follows from the way matrix derivatives were written down in equation 122. In general the reader is urged to convince themselves of the identities, $\nabla_{\boldsymbol{w}} \boldsymbol{b}^\top \boldsymbol{w} = \boldsymbol{b}$ and $\nabla_{\boldsymbol{w}} \boldsymbol{w}^\top \boldsymbol{A}\boldsymbol{w} = (\boldsymbol{A} + \boldsymbol{A}^\top)\boldsymbol{w}$ for any vector $\boldsymbol{b}$ and matrix $\boldsymbol{A}$.

As done earlier, we can again attempt to determine the global minima of the empirical risk function $\hat{R}(\boldsymbol{w})$ above by trying to determine the $\boldsymbol{w}$ at which its gradient $\nabla \hat{R}(\boldsymbol{w}) = 0$. Note that, such points in the domain at which the gradient of a function is 0 are generally called "critical points". Hence if there is a set of optimal weights $\boldsymbol{w}_*$ as just defined then it follows from equation 130 that,

$$\nabla \hat{R}(\boldsymbol{w}_*) = 0 \implies \boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{w}_* = \boldsymbol{X}^\top \boldsymbol{y} \tag{131}$$

We can recognize that (a) the last equality above specified the critical point $\boldsymbol{w}_*$ as the solution of a set of simultaneous linear equations. And (b) this set of linear equations can immediately be solved if the matrix $\boldsymbol{X}^\top \boldsymbol{X}$ is invertible — and in that case the solution can be written as,

$$\boldsymbol{w}_* = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y} \tag{132}$$

Recall that you have seen the above equation in Lecture 6C in COMP24112 and that this is often called the "normal equation".

At this point, the reader is urged to carefully note that the above determination of optimal weights $\boldsymbol{w}_*$ only applies if the data $\boldsymbol{X}$ satisfies the condition that the $d \times d$ matrix $\boldsymbol{X}^\top \boldsymbol{X}$ is invertible. Playing around with examples – maybe by randomly choosing the matrix $\boldsymbol{X}$ – the reader should be able to convince themselves that its not very hard to satisfy this assumption if $n > d$ i.e there are more data than the data dimensionality.

We will revisit linear regression in much more detail in Lecture 9 and in there we will realize that the above assumption of invertibility is not quite natural in modern ML — because at the cutting edge, we are mostly in the $d > n$ regime - the "big models" which is the theme of this course! We will see how the nature of global minima changes and methods of finding it need to be re-analyzed in response to this change in assumptions.

B Probability Theory Basics

This module is about *Statistical Learning*. For this, a foundation in a few key areas of mathematics is needed—probability theory, matrix algebra, and calculus. This section covers the basics of probability theory, using the notations we will adopt in this course unit.

For some of you, this will require some revision of concepts you may have met at school. In addition to this chapter, you may benefit from some material at https://www.probabilitycourse.com/.

For others who have some experience in probability theory—you can probably skip this section—but we will take a classical view, and focus on foundational concepts *without* the rigor of measure theory.

B.1 Random Variables

You're used to the idea of a *variable* from programming, like `int x = 1`. The value of `x` is known, and fixed, unless you change it. A *random* variable is a quantity where the value is not fixed in advance, but can take on several values, with different probabilities.

There are two types of random variables: *discrete* and *continuous*. We'll start with examples of discrete random variables, and the associated terminology. There are two essential ingredients to define a random variable: the *set of possible outcomes* and the *probability function*.

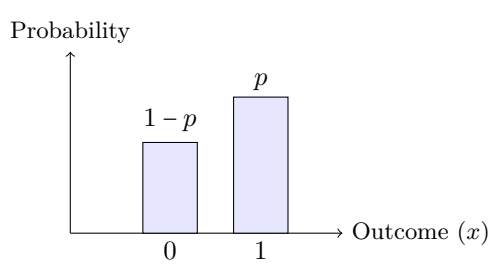
Set of outcomes. To specify a random variable, we first have to state the possible values it can take on, called the *outcomes*. For example $\{1, 2, 3, 4, 5, 6\}$, represents the possible rolls of a die.

Probability function. We also have to specify the probability associated with each possible outcome. In the case of a dice roll, every outcome is equal, i.e. $P(X = 1) = 1/6$, $P(X = 2) = 1/6$, etc. More generally, we might use notation $P(X = x)$ to refer to the probability of the random variable X taking on value x, where the lowercase x is called a *realization* of X. Note that we may also use $P(x)$ as a shorthand for $P(X = x)$.

Different random variables are used to 'describe' real-world events or phenomena. For example, a flip of a coin. Let's define a variable X with outcomes $\Omega = \{0, 1\}$, where $X = 1$ means the coin came up heads and $X = 0$ means tails. The probability of an outcome $X = x$ is given by the probability distribution function:

$$P(X = x) \;=\; p^x (1 - p)^{(1-x)}. \tag{133}$$

Notice that because $x \in \{0, 1\}$, the outcome of this expression is just p or $1 - p$, depending on which value of x you plug in. This is a **Bernoulli**[7] distribution, visualised below for $p = 0.6$.

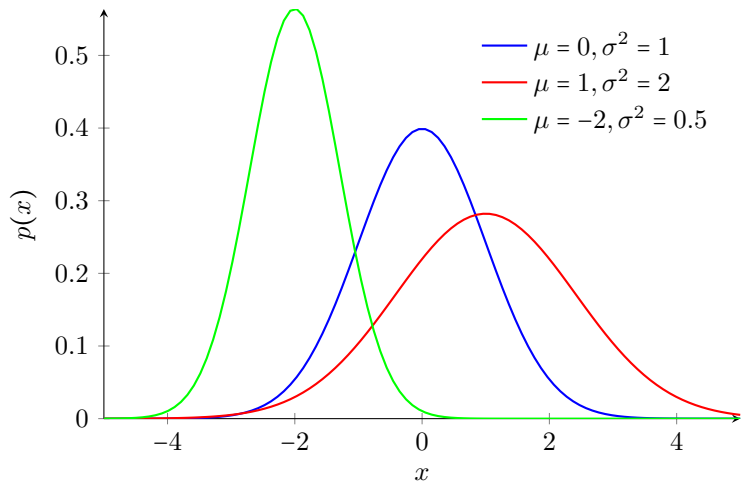


[7] Named after Jacob Bernoulli, an important mathematical figure of the 1600s.

Another important class of random variable for you to understand is the **Gaussian**. This represents continuous (real-valued) outcomes, and the outcomes are the entire real line $\mathbb{R}$: positive/negative numbers, including $\pm\infty$. This implies a subtle change of terminology, accompanied by some mathematics. The Gaussian probability *density* function is

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\Big(- \frac{(x-\mu)^2}{2\sigma^2} \Big). \tag{134}$$

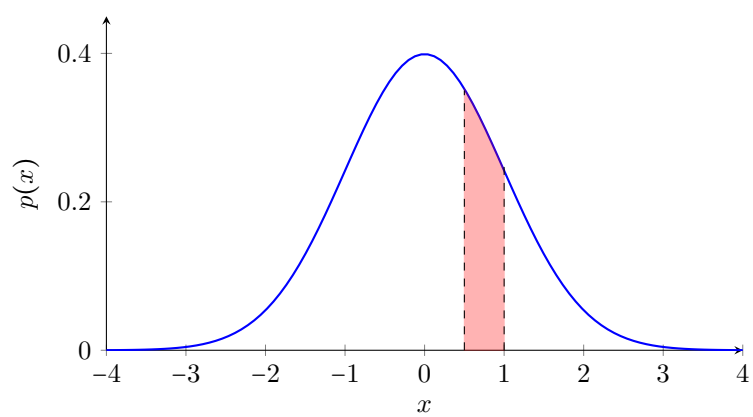
The Gaussian density is fully specified by two parameters: its mean μ, and variance σ^2.



Note that I used lower case p in (134). This distinction is because a *density* does not itself return a probability value, but allows you to calculate one. TO do this, we *integrate* the density function between two points. So the probability of a (Gaussian) random variable X taking a value $x \in [a, b]$ is:

$$P(X = x \in [a, b]) = P(a \leq X \leq b) = \int_a^b p(x)dx. \tag{135}$$

For example, with $\mu = 0,\ \sigma = 1$, the probability $P(0.5 \leq x \leq 1) \approx 0.1498$.



In later topics of this module, we'll be particularly interested in the 'tails' of a distribution—e.g. the area beyond +2/-2 in the above example. The area in the tails quantifies the probability of a rare event.

B.2 Joint and Conditional Random Variables

We'll now cover two important concepts, *joint* and *conditional* random variables. It's easier to discuss this in terms of discrete variables. The same ideas hold for continuous (e.g. Gaussian) random variables, but the mathematics is a bit more involved and not really needed right now.

Joint random variables. These are variables that describe the chances of two (or more) things happening at the same time. They are specified over two (or more) outcome set. We denote a joint r.v. by concatenating the symbols, e.g. XY, or using a comma X, Y. An example might be rolling two dice at the same time, or drawing two cards from a pack. A probability function would specify $P(X = x, Y = y)$ for every possible combination of x, y.

Conditional random variables. These are specified for one event conditional on another having already occurred, with the probability function written as $P(Y \mid X)$. For example, in drawing cards from a pack—the chances of drawing two Kings simultaneously is the *joint* probability $P(X = king, Y = king)$. But, the probability of drawing one King, followed by another (without replacing the first one), is a *conditional* probability with probability function $P(Y = king \mid X = king)$.

Concept Check B.1. You draw two cards, one after the other, from a standard deck of 52 cards **without replacing the first one**. What is the probability that the second card is a king, given that the first card drawn was also a king? Assume that X is the random variable for the first card drawn, and Y is for the second card. If you didn't know, every deck contains 4 kings at the start.

We've discussed two special types of random variable: a *joint* r.v., specified by a joint probability $P(X, Y)$, and *conditional* r.v., specified by a conditional probability $P(X \mid Y)$. A probability of a single variable by itself, like $P(X)$, is referred to as a **marginal** probability.

There are important relationships between these, given by two fundamental rules of probability theory.

Definition 32 (Product rule)**.**

$$\underbrace{P(X, Y)}_{joint\ prob.} = \underbrace{P(Y \mid X)}_{conditional} \times \underbrace{P(X)}_{marginal} \tag{136}$$

To illustrate this... from the concept check above, you should have concluded that $P(Y = king \mid X = king) = 3/51$, i.e. there are only 3 kings left, in the remaining set of 51 cards. If we use this in the product rule above, i.e. $P(Y = king \mid X = king) = 3/51$, and the marginal probability $P(X = king) = 4/52$, we just multiply them to get the joint probability $P(X = king, Y = king) = (3/51) \times (4/52) = 12/2652 = 1/221$, which is the probability of drawing two Kings in succession. Note that there is a symmetry to this, so $P(X, Y) = P(Y \mid X)P(X) = P(X \mid Y)P(Y)$.

The other important rule for you to know is the **sum rule.**

Definition 33 (Sum rule, also known as **Law of Total probability**)**.**

$$P(X) = \sum_{y \in \Omega_Y} P(X, Y = y) \tag{137}$$

where Ω_Y is the set of possible outcomes for r.v. Y.

From this we can also deduce interesting/useful formulae, e.g. by a simple rearrangement of the product rule we have $P(Y|X) = \frac{P(X,Y)}{P(X)}$. If you remember **Bayes' theorem** from previous modules, it is derived by applying the product rule to the numerator in this expression. Give it a go.

B.3 Expectation and Variance

Imagine I give you a set of numbers $\{0, 0, 1, 1, 1\}$. The *arithmetic mean* of this set is $(0+0+1+1+1)/5 = 0.6$.

The arithmetic mean is a special case of a broader concept called the *expected value*. The *expected value* is defined as the sum of each possible outcome in the set Ω, **multiplied** by its probability:

$$\mathbb{E}[X] = \sum_{x \in \Omega} x \times P(X = x) \tag{138}$$

With our example, there are two occurrences of 0 and three of 1. So, $P(X = 0) = 2/5$ and $P(X = 1) = 3/5$. The expected value is calculated as:

$$\mathbb{E}[X] \;=\; \Big(1 \times P(X = 1)\Big) + \Big(0 \times P(X = 0)\Big) \;=\; 3/5 \;=\; 0.6, \tag{139}$$

which is exactly the arithmetic mean we calculated before.

Take another example set of numbers: $\{2, 2, 2, 4, 11\}$. The *arithmetic mean* of this set as $(2+2+2+4+11)/5 = 4.2$. The number 2 occurs 3/5 times, i.e. probability 0.6. The numbers 4 and 11 each occur 1/5 times, so probability 0.2 each. The expected value is therefore $(2 \times 0.6) + (4 \times 0.2) + (11 \times 0.2) = 4.2$.

Concept Check B.2. What's the expected value for the roll of a 6-sided dice?

There are a few simple rules you need to memorize. In future weeks we will make use of these in calculations.

Expectations of Sums of Random Variables.

$$\mathbb{E}_{XY}[X + Y] = \mathbb{E}_X[X] + \mathbb{E}_Y[Y]. \tag{140}$$

Linearity of Expectations. For any constant a, we have:

$$\mathbb{E}[aX] = a\mathbb{E}[X]. \tag{141}$$

Expectations of Functions of Random Variables. For any function $g(\cdot)$, we have:

$$\mathbb{E}[g(X)] = \sum_{x \in \Omega_X} g(x) \times P(X = x). \tag{142}$$

Expectations of Products of Independent Random Variables. If X and Y are independent, then

$$\mathbb{E}_{XY}[XY] = \mathbb{E}_X[X]\mathbb{E}_Y[Y]. \tag{143}$$

Conditional Expectations. We can also talk about *conditional* expectations, e.g. $\mathbb{E}_{Y|X}$, where the probability distribution doing the weighting is the conditional $P(Y|X = x)$.

$$\mathbb{E}_{Y|X=x}[\ldots] = \sum_{y \in \Omega_Y} P(Y = y|X = x) \times [\ldots] \tag{144}$$

Concept Check B.3. Given an arbitary function $g(x)$, is the following expression true or false?

$$\mathbb{E}_{XY}[g(x)] = \mathbb{E}_X[g(x)]$$

The expected value is the *mean* of a random variable. We can also talk about the *variance*, i.e. how spread out the distribution is. You are probably most familiar with the variance written as so:

$$Var(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]. \tag{145}$$

You're probably familiar with this form for a simple Gaussian, where we have:

$$Var(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[(X - \mu)^2] = \sigma^2. \tag{146}$$

However, it can also be written as below, which will turn out easier to work with in many cases:

$$Var(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2 \tag{147}$$

Concept Check B.4. Try proving (147) from (145) yourself.

If you work this through for Bernoulli variable, where $\mathbb{E}[X] = p$, we get:

$$\begin{aligned} Var(X) &= \mathbb{E}[X^2] - \mathbb{E}[X]^2 && (148)\\ &= p - p^2 && (149)\\ &= p(1-p). && (150) \end{aligned}$$

where we note that the expected value of a squared Bernoulli variable is simply an application of Equation 142 where $g(X) = X^2$.

$$\mathbb{E}[X^2] = 0^2 \times P(X = 0) + 1^2 \times P(X = 1) = p \tag{151}$$

Again, there are a few simple rules you need to memorize.

Scaling of variance: For any random variable X, and any constant a, we have:

$$\begin{aligned} Var(aX) &= \mathbb{E}[(aX)^2] - (\mathbb{E}[aX])^2 && (152)\\ &= \mathbb{E}[a^2X^2] - (a\mathbb{E}[X])^2 && (153)\\ &= a^2 Var(X) && (154) \end{aligned}$$

Linearity of variance: For constants a and b,

$$Var(aX + b) = a^2 Var(X). \tag{155}$$

Variance of Averaged Random Variables: For a set of m independent and identically distributed random variables, $X_1, ..., X_m$, where $\bar{X} = \frac{1}{m}\sum_i X_i$ is their sample mean, we have:

$$Var\left(\bar{X}\right) = \frac{1}{m^2}\sum_{i=1}^{m} Var(X_i) \tag{156}$$

Proof is omitted, but you can find it in many standard textbooks (or Wikipedia).

B.4 The Law of Large Numbers and the Central Limit Theorem

There are two important ideas in probability theory that are often conflated and confused. These are the *law of large numbers*, and the *central limit theorem*. We won't go into a lot of detail here but just cover the definitions and important points for our purposes.

Law of Large Numbers (LLN)

The LLN states that as the sample size n increases, the sample mean $\hat{\mu}$ of independent and identically distributed (i.i.d.) random variables converges to the true population mean μ.

Imagine measuring the heights of people in a population. Each height measurement is a random variable X_i, drawn independently from the same population with mean μ (e.g. 180cm), and variance σ^2, (e.g. 20cm). The sample mean is defined as:

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

If n is small, the sample mean $\hat{\mu}_n$ may deviate significantly from the population mean μ. For example, measuring only a few individuals might give a biased estimate. As $n \to \infty$, the sample mean $\hat{\mu}_n$ converges to the true mean μ. Random fluctuations (e.g., unusually tall or short individuals) balance out, and the sample average stabilizes. Mathematically, the LLN guarantees:

$$\lim_{n\to\infty} \hat{\mu}_n = \mu$$

Technically, this statement is the "strong" law of large numbers, in that it states the gap between a sample mean and the true mean will always go to zero. This is called *almost sure* convergence. The "weak" law says the gap will get smaller and smaller, but there's no absolute guarantee it will go to zero. This (weaker) type of convergence is called *convergence in probability.* More here: https://www.probabilitycourse.com/chapter7/7_1_1_law_of_large_numbers.php

The Central Limit Theorem (CLT)

The CLT says something different, but related. It describes the *distribution* of the sample mean $\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} X_i$ for large sample sizes n. It states that, under certain conditions, as the sample size increases, the distribution of the sample mean approaches a **Gaussian**, regardless of the shape of the original distribution of individual measurements.

Let the heights of people in a population be denoted by $X_1, X_2, \ldots, X_n$, which are independent, identically distributed (i.i.d.) random variables with the same μ and σ^2 as above. The CLT states that for sufficiently large n, the distribution of the sample mean $\hat{\mu}_n$ will approximate a normal distribution with:

$$\hat{\mu}_n \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right) \quad \text{for large } n.$$

In other words, as $n \to \infty$, the sample mean $\hat{\mu}_n$ becomes normally distributed with mean μ and variance $\frac{\sigma^2}{n}$. The clever thing is, this applies for all types of variables. Imagine flipping coins. The X_i are now Bernoulli variables. But, the Central Limit Theorem still holds. More here: https://www.probabilitycourse.com/chapter7/7_1_2_central_limit_theorem.php

Summary of the two

- The LLN tells us that large samples give reliable estimates. But, it doesn't tell us anything about the distribution of those estimates for a fixed n. It just says they get closer to the true value as $n \to \infty$.
- The CLT explains that the variability of those estimates around the true value actually behaves predictably: following a normal distribution.

References

Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine learning and the bias-variance trade-off. *arXiv preprint arXiv:1812.11118*, 2018.

Leo Breiman. Bagging predictors. In *Machine Learning*, pages 123–140, 1996.

Gavin Brown and Riccardo Ali. Bias/Variance is not the same as Approximation/Estimation. *Transactions on Machine Learning Research*, 2024. URL https://TO~APPEAR-SOON.

Tom Heskes. Bias-variance decompositions: the exclusive privilege of bregman divergences. *arXiv preprint arXiv:2501.18581*, 2025.

Anders Krogh, Jesper Vedelsby, et al. Neural network ensembles, cross validation, and active learning. *Advances in neural information processing systems*, 7:231–238, 1995.

R Tyrrell Rockafellar. Lagrange multipliers and optimality. *SIAM review*, 35(2):183–238, 1993.

Danny Wood, Tingting Mu, Andrew Webb, Henry Reeve, Mikel Lujan, and Gavin Brown. A unified theory of diversity in ensemble learning. *https://arxiv.org/abs/2301.03962*, 2023.